



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校

河南师范大学

参赛队号

24104760033

队员姓名

1. 贾昊鑫

2. 任永攀

3. 李富坤

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生

数学建模竞赛

题 目： WLAN 组网中网络吞吐量建模

摘 要：

无线局域网（WLAN，Wireless Local Area Network）是一种通过无线通信技术实现的局域网，允许设备在一定范围内（通常是几百米）相互连接并访问网络。它摆脱了传统物理连接（如电缆或光纤）的束缚，使得用户能够灵活地在各种环境中接入网络，支持多种设备的连接，提升了信息获取和交流的便利性。

针对问题一：问题一的核心任务是对无线局域网（WLAN）系统中各接入点（AP）的发送机会进行建模和预测。分析不同因素对 AP 发送机会的影响，并使用这些信息来预测每个 AP 的发送数据帧序列的总时长。分析结果显示，**发送机会与 RSSI 强相关**。通过对题目的分析可知问题一属于回归模型。为了提升预测结果的准确性，我们采用八种回归模型进行预测，从不同角度捕捉 WLAN 系统的复杂特性。并使用 python 绘制图标进行可视化结果、对比、结果显示在 2AP 下，MSE 为 **10.412**，MAE 为 **1.756**，MAPE 为 **0.064**，在 3AP 下，MSE 为 **43.869**，MAE 为 **4.812**，MAPE 为 **0.209**。

针对问题二：问题二的核心任务是预测无线局域网（WLAN）系统中每个无线接入点（AP）发送数据时，选用的调制编码方案（MCS）和空间流数（Number of Spatial NSS）。由于 MCS 和 NSS 的组合数量有限且离散化，因此需要对每个 AP 的信道和网络条件进行特征提取，并将这些输入特征映射到特定的 MCS 和 NSS 组合上。分析结果显示，**MCS 和 NSS 组合与 sin_dB 强相关**。并选择一个最优的 MCS 和 NSS 组合来实现最大的吞吐量，将多种特征映射到有限类别的输出。因此问题属于多分类模型。针对本题的建模采用了传统的逻辑回归、支持向量机（SVM）、K 近邻（KNN）、随机森林等多种经典分类模型。并进行可视化分析、对比，结果显示在 MCS 标签预测目标下，Accuracy 为 **0.84**，F1 Score 为 **0.63**，Precision 为 **0.63**，Recall 为 **0.87**，在 NSS 标签预测目标下，Accuracy 为 **0.98**，F1 Score 为 **0.55**，Precision 为 **0.55**，Recall 为 **0.55**。

针对问题三：问题三的核心任务是根据无线局域网（WLAN）实测数据，构建一个预测系统吞吐量的模型。WLAN 网络的吞吐量受到接收信号强度（RSSI）、信道质量、节点间干扰、等多种因素的影响，选取多种的回归模型进行结果预测。每个模型都有其独特的

数学原理和适用场景，在不同的特征表现和数据分布下表现出不同的性能，这使得所得结果更为全面、准确。经过对结果的可视化分析、对比。相关性分析结果显示，不论是在 2AP 的情况下，还是 3AP 的情况下，吞吐量与协议和信号强相关。验证结果显示在 2AP 下，MSE 为 **10.412**，MAE 为 **1.756**，MAPE 为 **0.064**，在 3AP 下，MSE 为 **10.905**，MAE 为 **1.814**，MAPE 为 **0.091**。

关键词： 无线局域网 系统吞吐量 AP 发送机会 多分类模型 回归模型

目录

1	问题背景	5
2	问题重述	9
2.1	问题一重述	9
2.2	问题二重述	9
2.3	问题三重述	9
3	模型假设	10
4	符号说明	11
5	问题分析	12
5.1	问题一分析	12
5.2	问题二分析	14
5.3	问题三分析	16
6	模型的建立与求解	18
6.1	问题一模型建立与求解	18
6.1.1	数据预处理	18
6.1.2	特征提取	19
6.1.3	模型构建与优化	19
6.1.4	模型预测与评估	22
6.2	问题二模型建立与求解	24
6.2.1	信号干扰与 SINR 建模	24
6.2.2	AMC 算法	27
6.2.3	丢包率与 PHY 层速率的关系	27
6.2.4	吞吐量	28
6.2.5	多分类模型的建立	28
6.2.6	多分类模型的预测与评估	34
6.3	问题三模型建立与求解	34
6.3.1	PHY 速率	35
6.3.2	有效数据速率	35
6.3.3	接收信号强度 (RSSI) 提取	35

6.3.4	AP 间互听性	36
6.3.5	干扰 RSSI 提取	36
6.3.6	机器学习模型建立	37
6.3.7	模型预测与评估指标	37
7	实验结果分析	39
7.1	问题一结果分析	39
7.1.1	变量间的相关性分析	39
7.1.2	验证结果分析	40
7.1.3	测试结果分析	44
7.2	问题二结果分析	45
7.2.1	变量间的相关性分析	45
7.2.2	训练过程分析	48
7.2.3	验证结果分析	52
7.2.4	测试结果分析	56
7.3	问题三结果分析	59
7.3.1	变量间的相关性分析	59
7.3.2	验证结果分析	60
7.3.3	测试结果分析	62
8	模型评价	63
8.1	问题一模型评价	63
8.2	问题二模型评价	63
8.3	问题三模型评价	63
9	未来展望	65
	参考文献	66
	附录 A Python 源程序	67
A.1	第 1 问程序	67
A.2	第 2 问程序	93
A.3	第 3 问程序	122

1 问题背景

无线局域网（Wireless Local Area Network, WLAN）是一种利用无线通信技术建立局域网的方式，其核心是通过无线信道将多个终端设备互联，从而实现信息的无线传输与共享。WLAN [1] 自其提出以来，基于 IEEE 802.11 标准 [2] 不断发展和演进，从早期的 IEEE 802.11a/b/g [3] 演变到最新的 IEEE 802.11ax（Wi-Fi 6）及即将到来的 Wi-Fi 7[4]，速率和带宽显著提升。WLAN 在全球范围内的广泛部署，涵盖了家庭、办公、教育、工业、医疗等各类场景 [5]，逐渐成为人们日常生活中不可或缺的网络基础设施。

WLAN 吞吐量的瓶颈与挑战尽管 WLAN 技术取得了显著进步，峰值速率已经达到了数 Gbps，未来的 Wi-Fi 7 [6] 更是有望支持高达 30 Gbps[7] 的传输速率，但在实际部署中，WLAN 系统的性能往往无法达到理论上的最大值。尤其是在高密度的使用场景下 [8]，如现代化办公室、智慧工厂、医院等，由于节点密集度增加、覆盖区域相互重叠等原因，干扰和碰撞问题日益严重，导致网络吞吐量和传输速率大幅下降。图1.1 展示了一个简单的 WLAN 组网架构 [9]。

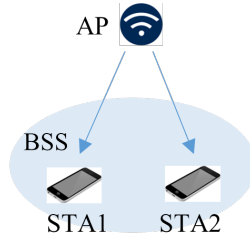


图 1.1 WLAN 组网架构

影响 WLAN 吞吐量的关键因素包括信道接入机制、节点间的信号干扰、业务流量模式等。WLAN 使用载波侦听多址接入/冲突避免（Carrier Sense Multiple Access with Collision Avoidance, CSMA/CA）的机制来实现信道接入，多个节点在同一信道上进行数据传输时，必须首先判断信道是否可用，以避免数据帧的碰撞。然而，在节点密集度较高的环境中，多节点的频繁竞争会导致信道接入效率下降，进而影响网络的整体吞吐量。

此外，信道环的动态变化也是影响 WLAN 吞吐量的重要因素 [10]。在无线通信中，信道条件随时可能发生波动，干扰、阻塞、反射等环境因素都会影响信号的传输质量。这种情况下，基于固定参数的仿真模型通常无法准确反映实际部署中的网络性能变化，导致仿真结果与实际网络性能之间存在较大偏差 [11]。因此，基于仿真训练的预测模型在实际网络优化中应用时往往难以满足精度和适用性要求。两个 STA 进行通信时，无线设备在接收数据帧和发送数据帧之间会存在时间间隔，也称短帧帧间距 SIFS(Short inter-frame space)[12]。接收站在收到数据帧并且确认接受无误时，会发送 ACK(Acknowledge character) 字符表示确认。当发送站收到接收站反馈回的 ACK 字符时，表明发送成功。其他情况如：接收站为未接收到数据帧、接收站反馈 ACK 字符失败或发送站未收到 ACK 字符，都代表

数据传输失败。数据传输失败的情况下，发送节点会在等待超时后重新传数据。STA 通信流程如图1.2所示。

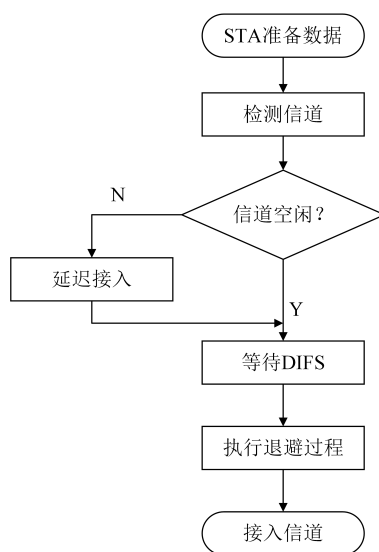


图 1.2 DCF 流程示意图

为提升 WLAN 系统的性能，如何对吞吐量进行精确预测是一个关键问题。吞吐量通常指节点在单位时间内成功发送的比特数，它是衡量网络性能的核心指标。精确的吞吐量预测能够帮助设计者优化 WLAN 系统的配置，并对潜在的性能瓶颈进行预判，进而提升系统的稳定性和效率。随着 WLAN 部署场景的日益复杂，特别是在大规模、多节点的部署中，吞吐量预测面临的挑战愈发严峻 [13]。WLAN 中工作在单一信道的各个节点共享该信道，并且采用载波侦听多址接入/退避 (CSMA/CA, carrier sense multi-access and collision avoidance) 的机制 [12] 避免冲突，即分布式协调功能 (DCF, distributed coordination function)。CSMA/CA 的工作原理如图1.3所示。

在两个节点进行数据传输时，若 $RSSI > ED$ ，则意味着一个节点传输数据时，另一节点检测信道为繁忙，称两个节点能“听”到。在这种情况下二者的数据传输将交替进行，只在偶然同时回退到 0 时同时开始数据传输。这种在节点间可侦听情况下发生的传输称为同步传输。然而在实际部署 WLAN 中，受覆盖范围和可用信道数等约束，异步传输的情况更为普遍。AP 之间的 RSSI 处于 $[PD, ED]$ 之间，无法检测到彼此的前导序列，导致 AP 无法确认信道的使用状态，从而出现异步传输。图1.4展示了异步传输的示意图。

当前一些研究开始尝试使用机器学习 [14] 等技术手段，通过对网络的基础特征进行提取和分析，进行吞吐量预测。常见的网络特征包括信道带宽、业务流量、发送功率、信道接入机制、节点之间的接收信号强度 (RSSI)、信噪比 (SINR) 等。此外，还可以结合节点的动态位置和干扰情况，进行更加精准的建模和预测。然而，这类方法大多基于仿真数据进行模型训练，虽然在仿真环境中能够取得较好的预测精度，但由于真实场景中的通信行为复杂多变，仿真数据无法全面覆盖所有可能的网络情况，导致模型在实际应用中的

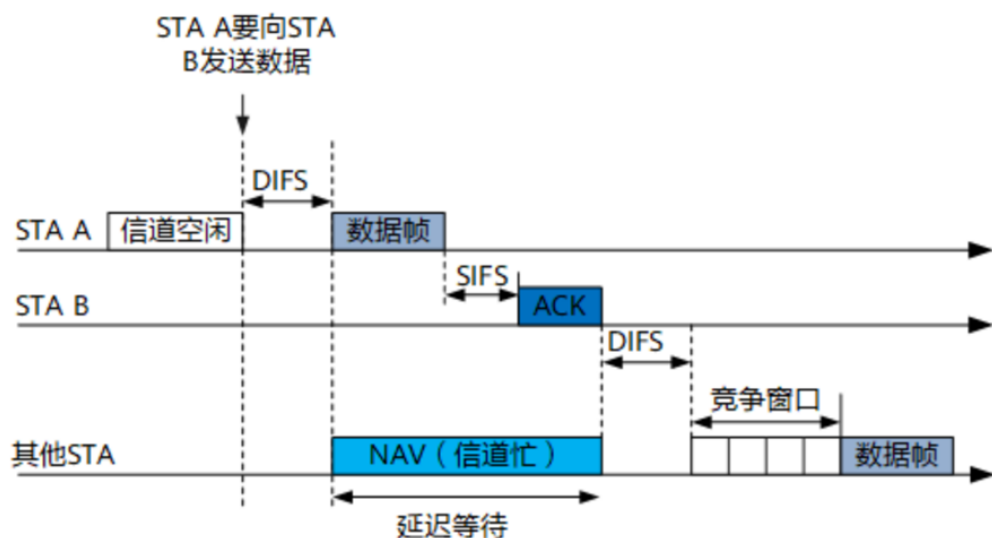


图 1.3 CSMA/CA 的工作原理图



图 1.4 一次异步传输和异步传输状态示意图

效果往往不如预期。为解决这一问题，准确捕捉 WLAN 系统中的实际部署数据，并结合无线通信中的关键影响因素，进行数据驱动的吞吐量预测，成为当前 WLAN 优化的一个重要研究方向。通过分析不同 AP（无线接入点）之间的 RSSI、信道接入机制、信道干扰等因素对吞吐量的影响，建立更加符合实际场景的吞吐量预测模型，有望进一步提升 WLAN 系统的性能。此外，在 WLAN 吞吐量建模过程中，需要考虑的因素非常多，主要包括以下几个方面：

信道接入机制的复杂性：WLAN 节点采用 CSMA/CA 机制进行信道接入，不同节点之间的信道竞争、随机回退等过程对吞吐量有着直接影响。尤其在多节点场景下，如何建模节点之间的竞争关系，精确估算各节点获得信道的机会，是吞吐量建模的一个难点。

节点间干扰的影响：节点之间的干扰是 WLAN 网络中的常见问题，尤其是在高密度部署场景中，多个节点的传输信号会相互干扰，导致信号解码失败或数据传输错误。因此，在吞吐量建模时，需要充分考虑节点之间的 RSSI 及其对数据传输成功率的影响。

动态网络环境的建模：无线信道环境具有高度的动态性，不同时间、不同位置的信号强度、干扰水平都会发生变化。建模时，需要考虑到这种动态性，合理引入时变因素进行预测，以提高模型的适用性和鲁棒性。

业务流量的复杂性：不同类型的业务流量对网络资源的需求各不相同。例如，TCP 和 UDP 协议的行为模式存在较大差异，TCP 的拥塞控制机制会动态调整传输速率，而 UDP

则是固定速率发送。因此，建模时需要区分不同的业务流量特征，以准确反映其对吞吐量的影响。

PHY 速率的选择：WLAN 节点使用自适应调制编码（AMC）算法，根据信道条件动态调整物理层（PHY）速率。在信号质量较好时，节点会选择较高的调制编码方案（MCS）和空间流数（NSS），以提高数据传输速率；反之，则选择较低的 MCS 和 NSS。

因此，在吞吐量预测时，还需要考虑 PHY 速率的自适应变化。总的来说，WLAN 系统吞吐量建模是一个复杂的多因素综合问题，涉及信道接入、节点干扰、动态信道环境、业务流量模式等多个方面。通过对 WLAN 实测数据的深入分析，建立准确的吞吐量预测模型，对于优化 WLAN 系统性能具有重要意义。

2 问题重述

2.1 问题一重述

在无线局域网（WLAN）中，不同接入点（AP）之间的信道竞争、干扰和节点间的信号强度（RSSI）等因素会影响各 AP 的发送机会，即每个 AP 在特定时间段内能够成功发送数据的总时长（seq_time）。WLAN 通过载波侦听多址接入/冲突避免（CSMA/CA）的机制，AP 在信道可用时竞争进行数据发送，而信道可用性受节点间 RSSI、CCA 门限、随机回退等多种因素的影响。因此，准确建模和预测 AP 的发送时长，能够帮助优化信道资源的分配，并最大化 WLAN 系统的吞吐量。问题 1 的目标是通过分析 WLAN 实测数据中网络拓扑、业务流量、信道接入机制等特征，找出影响 AP 发送机会的关键因素，并根据这些特征建立模型，预测 AP 在不同网络条件下的发送机会。通过对 2AP 和 3AP 测试集的分类建模和分析，最终为复杂场景下 WLAN 网络优化提供参考依据。

2.2 问题二重述

在 WLAN 系统中，AP 发送数据时会根据信号干扰情况和信道条件，自适应地选择合适的调制编码方案（MCS）和空间流数（NSS），从而确定物理层速率（PHY Rate）。这些选择不仅受信号干扰的影响，还取决于节点间的信号强度（RSSI）和网络的传输方式（同步传输或异步传输）。问题 2 的核心是根据实测数据中的 RSSI 信息和门限信息，结合对 AP 发送机会的分析，构建预测 AP 选用最多次数（MCS, NSS）组合的模型。这不仅能帮助预测 WLAN 中的 PHY 速率，还能为进一步优化网络性能提供依据。通过在测试集中分析不同的 AP 传输情况，该模型可以预测在不同 RSSI 条件下 AP 所采用的最优速率参数组合，以提高网络传输效率。

2.3 问题三重述

WLAN 系统的吞吐量是衡量网络性能的关键指标之一，受多种因素影响，包括 AP 的发送机会和所选用的调制编码方案（MCS）及空间流数（NSS）。问题 3 要求结合前两个问题的分析结果，建立一个完整的系统吞吐量预测模型。首先，需要综合考虑影响 AP 发送机会的因素，例如信道接入机制、节点间 RSSI 等。其次，基于这些因素，进一步预测每个 AP 采用的（MCS, NSS）组合。最后，将 AP 的发送时长和物理层速率结合，计算出 WLAN 系统的实际吞吐量。该模型不仅可以通过训练集评估其性能，还可以通过测试集预测在不同网络条件下的系统吞吐量。通过精确的吞吐量建模，可以为 WLAN 的部署和优化提供有效指导，确保在复杂、动态的无线环境下实现高效的数据传输。

3 模型假设

1. 针对两个 AP 传输总是不互听情况，求解两个节点间 RSSI 时忽略环境底噪；
2. AP 发送数据时选用最多次数数的 (MCS, NSS) 反映 SINR 水平；
3. 实测中统计的数据帧真实 (MCS, NSS) 可作为高精度模型输入变量；
4. 在建模过程中，WLAN 网络环境稳定，测得数据不会出现多数差异大的离群值。
5. 回归模型和分类模型中使用的各个影响因子相互独立。

表 4.1 变量符号及其说明

序号	符号	表达含义	序号	符号	表达含义
1	P_error	丢包率	24	λ	正则化系数
2	P_ACK	ACK 确认概率	25	β_0	截距项
3	T_seq	发送数据帧总时长	26	$\beta_1, \beta_2, \dots, \beta_n$	回归系数
4	RSSI	信号强度	27	C	正则化参数
5	CU	信道利用率	28	M	决策树的数量
6	NL	噪声水平	29	T_m	第 m 棵决策树的预测结果
7	X	输入特征矩阵	30	y_i	第 i 个最近邻居的值
8	y	目标变量	31	$l(y_i, \hat{y}_i)$	损失函数
9	N	样本总数	32	$\Omega(hm)$	正则化项
10	w	权重向量	33	P_signal	目标 AP 发送信号的功率
11	b	偏置项	34	P_interference	邻近 AP 产生的干扰功率
12	σ	标准差	35	P_noise	背景噪声的功率
13	μ	均值	36	P_AP_i	第 i 个 AP 的发射功率
14	ε	误差	37	L_i	第 i 个 AP 到 STA 的路径损耗
15	μ_{RSSI}	信号强度的均值	38	d_i	AP 到 STA 的距离
16	σ_{RSSI}	信号强度的标准差	39	d_0	参考距离
17	μ_X	X 的均值	40	α	路径损耗指数
18	σ_X	X 的标准差	41	N_pkt	时间 t 内传输的包数
19	X_RSSI	X 某一列的信号强度	42	N_error	传输失败的包数
20	R_SSI 异常	异常值	43	R_phy	PHY 层速率
21	P_received	接收到的信号功率	44	T_throughput	吞吐量
22	N_int	邻近 AP 的数量	45	N_ant	AP 或 STA 的天线数量
23	RSSI_j	第 j 个 AP 的干扰 RSSI	46	RSSI_i	第 i 根天线接收的信号强度

4 符号说明

我们对计算当中使用的符号进行简单说明，便于读者理解，如表4.1所示。

5 问题分析

5.1 问题一分析

问题一的核心是对无线局域网（WLAN）系统中各接入点（Access Point, AP）的发送机会进行建模和预测。具体而言，问题要求根据网络拓扑、业务流量、门限、节点间接收信号强度指示（RSSI）等输入特征，分析不同因素对 AP 发送机会的影响，并使用这些信息来预测每个 AP 的发送数据帧序列的总时长。

从问题的性质来看，问题一属于回归问题，因为目标是预测一个连续值（AP 的发送时长 seq time）。预测任务涉及将多维输入特征映射到一个连续的输出目标值，这与经典的回归问题特征一致。因此，本问题可以归类为机器学习中的监督学习任务，特别是针对时间序列或负载预测的回归问题。具体流程如图5.5所示。

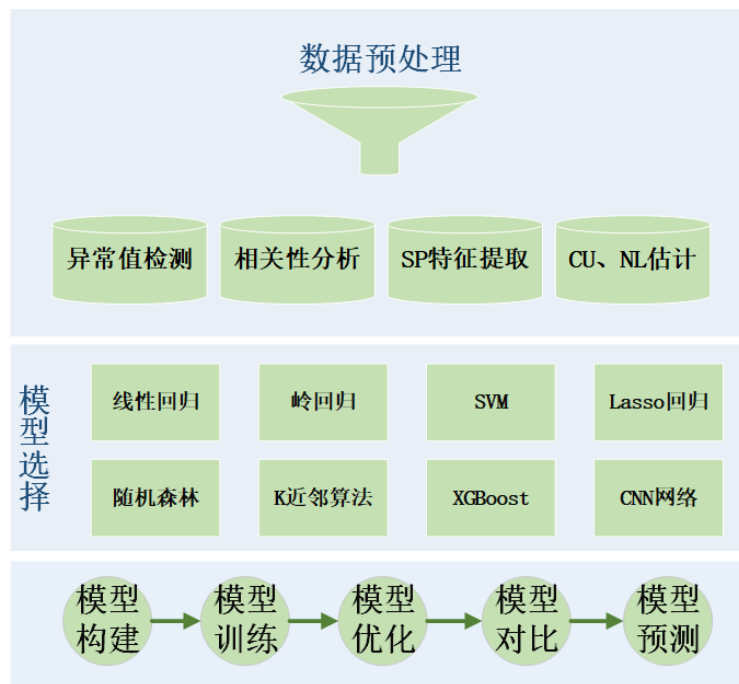


图 5.5 问题一分析流程

此外，问题一是一个具有一定复杂度的多变量预测问题。影响发送机会的因素很多，包括网络中的信号强度（RSSI）、干扰强度（SINR）、信道利用率（CU）等多种因素，而这些因素之间存在复杂的非线性关系。因此，简单的线性回归模型可能不足以准确捕捉这些复杂的关系，必须结合适当的特征工程和非线性模型来进行建模与预测。

为了对问题进行求解，我们使用了多种机器学习方法进行建模与预测。这些手段包括数据预处理、特征提取、模型选择与优化等。下面从各个方面对所使用的手段进行分析：

1. 数据预处理是机器学习中的关键步骤，确保模型能够处理高质量的数据。对于 WLAN 系统中的实测数据，我们进行了多种预处理操作。首先，通过异常值检测与处理，

去除了数据集中可能影响模型性能的异常数据。通常，这些异常数据会显著偏离数据的主流分布，对模型的泛化能力带来负面影响。其次，我们应进行了数据标准化操作。由于数据集中不同特征的量级差异较大，如 RSSI 是以 dBm 表示的接收信号强度，而信道利用率 (CU) 是一个介于 0 和 1 之间的比例值，因此需要对特征进行归一化或标准化处理。这样可以避免某些特征由于数值量级过大而对模型训练过程中的梯度产生不良影响。最后，针对数据集中可能存在的缺失值，通过均值插补或插值法对缺失数据进行了填充。这保证了在模型训练时，数据完整性不会对训练效果造成不利影响。

2. 特征提取是问题一建模的重要步骤。在 WLAN 系统中，多个变量共同影响 AP 的发送机会，包括 RSSI、信道利用率 (CU)、噪声水平 (Noise Level, NL) 等。我们通过提取这些核心特征，将输入特征矩阵 X 作为预测模型的输入。

RSSI (接收信号强度指示): RSSI 是衡量无线信号质量的关键指标，它代表了接收信号的功率，通常用于判断信号强度和链路质量。较高的 RSSI 值意味着信号较强，可以提高 AP 的数据传输速率，从而增加 AP 的发送机会。**SINR (信号干扰噪声比):** SINR 是信号与干扰和噪声的比值，反映了无线通信的干扰状况。高 SINR 通常意味着干扰较小，通信链路质量较高，因此可以提高发送机会。**CU (信道利用率):** 信道利用率反映了信道被占用的程度，较高的信道利用率意味着信道竞争更加激烈，从而可能降低 AP 的发送机会。通过对这些特征的深入分析，我们能够进一步理解不同特征对 AP 发送机会的影响，并为后续的模型训练奠定基础。

3. 针对问题一的建模任务，我们采用了多种机器学习模型进行求解，目的是选择最合适的模型以获得最佳的预测性能。不同模型的选用依据是各自的特点和优势，以应对问题一中输入特征和目标变量之间复杂的非线性关系。常用的机器学习模型包括线性模型、决策树模型、集成学习模型和神经网络模型等。

4. 为了保证模型的预测精度，我们引入了多种模型优化策略。通过正则化技术 (如 L2 正则化)，防止模型在训练过程中出现过拟合现象，从而提高模型的泛化能力。此外，我们采用了交叉验证的方式对模型进行评估，确保模型能够在不同的数据集上表现稳定。

5. 在模型评估阶段，采用了多种评估指标，包括均方误差 (MSE)、平均绝对误差 (MAE) 和累积分布函数 (CDF) 等。这些评估指标能够帮助我们全面了解模型的预测性能，找到最优的模型。

总之，问题一属于典型的多因素非线性回归问题，解决该问题需要通过数据预处理、特征提取和多种机器学习模型的构建与优化。通过分析不同特征对发送机会的影响，我们能够找到影响发送时长的关键因素，并通过合适的机器学习算法对发送时长进行准确预测。未来可以结合更多的网络特征以及实时监控数据，进一步提升模型的预测精度和适应性。

5.2 问题二分析

问题二的核心任务是预测无线局域网（WLAN）系统中每个无线接入点（Access Point, AP）发送数据时，选用的调制编码方案（Modulation and Coding Scheme, MCS）和空间流数（Number of Spatial Streams, NSS）。WLAN 中的吞吐量直接受到 MCS 和 NSS 的影响，它们决定了 PHY 层的传输速率，因此准确预测 AP 所选用的 MCS 和 NSS 组合对于网络性能优化至关重要。本质上，问题二是一个多分类问题。由于 MCS 和 NSS 的组合数量有限且离散化，因此我们需要对每个 AP 的信道和网络条件进行特征提取，并将这些输入特征映射到特定的 MCS 和 NSS 组合上。每一个 AP 都会选择一个最优的 MCS 和 NSS 组合来实现最大的吞吐量，从而将多种特征映射到有限类别的输出。这符合机器学习中的分类问题的定义，且是一个多类别分类问题。

在实际的无线网络通信中，AP 的 MCS 和 NSS 选择受多种因素的影响，如信道条件（RSSI、SINR 等）、干扰水平、丢包率、业务流量类型和信道利用率等。这些特征决定了 AP 在发送数据时需要使用的调制和编码方案，以确保在当前的网络条件下获得最大的传输速率。因此，问题二不仅是一个简单的多分类问题，还涉及到如何基于动态和多变的信道环境，结合这些复杂的输入特征来选择最佳的 MCS 和 NSS 组合。求解具体流程如图5.6所示：

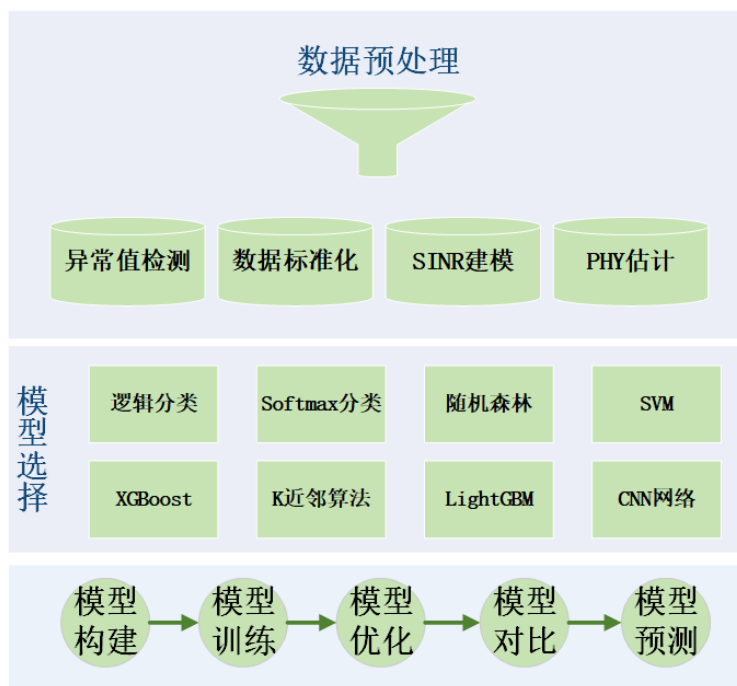


图 5.6 问题二分析流程

求解手段分析针对问题二的求解，我们采用了多种机器学习模型来进行建模和分类。由于 MCS 和 NSS 的选择是一个多分类任务，因此主要采用了多种分类模型进行预测。这些模型包括传统的逻辑回归、支持向量机（SVM）、K 近邻（KNN）、随机森林等经典分类器，还包括神经网络、XGBoost 和 LightGBM 等先进的机器学习方法。以下是我们具体使

用的求解手段和建模过程。

1. 在机器学习建模过程中，数据预处理是至关重要的第一步。通过对数据集中的各种特征进行清理、标准化和转换，我们可以保证分类模型能够稳定、准确地对 MCS 和 NSS 组合进行预测。

2. 异常值处理：在处理信道数据（如 RSSI 和 SINR）时，可能会存在一些极端的值或异常数据点。这些异常值可能是由于信号检测过程中的误差或者其他外部干扰导致的。因此，我们采用了基于均值和标准差的异常值检测方法，去除掉那些偏离主要分布的异常数据点，以确保数据的稳定性。缺失值填补：由于测试数据的采集过程中可能出现一些数据缺失的情况（例如某些信号强度测量未成功），因此我们对缺失值进行了合理的填补。对于 RSSI 等数值型特征，我们采用了均值填补的方法，以减少数据的不完整性对模型训练的影响。特征标准化：特征标准化的目的是将不同量级的特征转化为相同的尺度，避免某些特征（例如 RSSI 的值通常较大）在训练过程中对模型产生过大的影响。通过将各个特征标准化为零均值和单位方差，我们能够使模型在训练时对每个特征进行公平的处理。2. 特征提取是本问题求解的关键步骤。要准确预测每个 AP 在当前网络条件下选用的 MCS 和 NSS 组合，需要从原始数据集中提取出能够反映信道和网络状态的有效特征。

3. 问题二是一个典型的多分类问题，因此我们选择了多种分类算法进行建模和预测。不同的分类模型适用于不同的数据分布和特征模式，因此我们在建模过程中尝试了多种算法，以确保我们能够找到最佳的模型解决方案。

4. 模型优化与评估在模型训练过程中，我们使用了交叉验证方法对模型进行调优，以确保模型能够在不同的数据集上具有良好的泛化能力。此外，我们采用了常见的评估指标如分类准确率和交叉熵损失来评估模型的性能。

5. 为了防止模型出现过拟合现象，我们在某些模型（如逻辑回归和神经网络）中引入了正则化项，例如 L2 正则化，通过对模型参数施加惩罚项来控制模型的复杂度。在神经网络和 XGBoost 等复杂模型中，我们还通过调整超参数（如学习率、树的深度等）来优化模型性能。通过对多种模型的性能比较，最终选择出最适合问题二求解的模型，并利用该模型对每个 AP 的 MCS 和 NSS 组合进行预测。

综上，问题二的本质是一个多分类问题，其中每个 AP 需要根据当前的信道和网络条件选择最适合的调制编码方案（MCS）和空间流数（NSS）组合。通过对网络拓扑、信道质量（RSSI、SINR）、信道利用率、丢包率等特征的提取，我们构建了一个多维特征矩阵，并利用多种分类模型进行建模和预测。我们采用了多种机器学习模型，包括逻辑回归、支持向量机、K 近邻、随机森林、神经网络、XGBoost 和 LightGBM。这些模型各有优势，能够分别处理不同的数据特性和场景。通过合理的特征工程和模型优化，我们能够有效预测 AP 在不同网络条件下最可能选择的 MCS 和 NSS 组合，从而提高 WLAN 系统的吞吐量和性能提供支持。

在实际的模型选择过程中，我们通过交叉验证和多种评估指标（如分类准确率和交叉熵损失）来评估模型的性能，最终选择表现最优的模型用于实际的预测任务。与此同时，我们引入了正则化技术来防止模型过拟合，确保模型在测试集上的泛化能力。问题二的求解体现了典型的机器学习分类问题的建模流程：从数据预处理、特征提取到模型选择与优化，再到模型评估与预测。这一系列步骤不仅确保了模型能够准确预测 MCS 和 NSS 组合，也为无线网络优化提供了一个数据驱动的解决方案。

5.3 问题三分析

问题三的核心任务是根据无线局域网（WLAN）实测数据，构建一个预测系统吞吐量的模型。WLAN 网络的吞吐量是指在特定时间内，网络成功传输的比特数。这是一个典型的回归问题，因为目标变量吞吐量是一个连续数值。相比于问题二中的 MCS 和 NSS 组合选择属于分类任务，吞吐量的预测需要根据一系列特征来估算一个数值，因此我们需要通过机器学习中的回归算法来完成这一任务。

WLAN 网络的吞吐量受到多种因素的影响，包括接收信号强度（RSSI）、信道质量、节点间干扰、物理层速率、丢包率、信道竞争特性等。这些复杂的因素相互影响、相互制约，导致吞吐量的变化。因此，我们需要对这些特征进行详细分析，并通过建立复杂的回归模型，将特征与吞吐量的关系拟合出来。具体流程如图5.7所示。

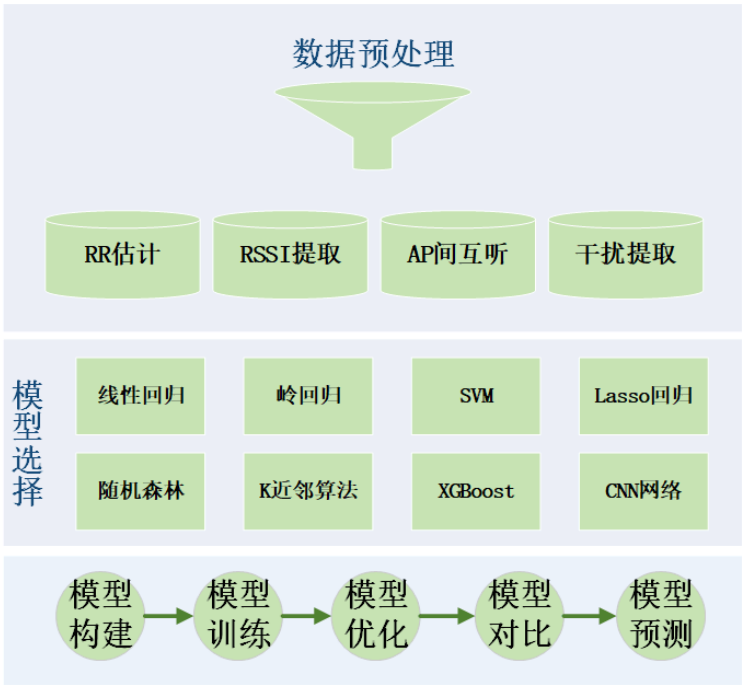


图 5.7 问题三分析流程

首先我们需要解决以下问题：

- 1. **多特征非线性关系：** 吞吐量不仅受到单一特征的影响，而是多个特征的共同作用。

例如，RSSI 较好时，物理层速率可以较高，但如果同时存在较强的干扰和高丢包率，吞吐量依然会受到限制。因此，吞吐量预测是一个典型的多维特征非线性回归问题。

2. 时变性与干扰性：WLAN 网络是一个动态的系统，信道竞争、节点干扰以及链路状态都会随着时间变化。因此，在建模过程中，需要考虑系统的时变性以及这些干扰因素带来的随机波动。

3. 特征间的共线性问题：如 RSSI、SINR 等特征之间存在较强的相关性。这种多重共线性会影响模型的稳健性和泛化能力，需要通过特定的建模手段（如岭回归）来解决。

为了解决这个回归问题，我们采用了机器学习的回归建模方法，基于数据驱动的方法来进行预测。具体来说，我们通过以下几个步骤来求解这个问题：

1. 特征提取是构建有效机器学习模型的核心步骤。在本问题中，WLAN 网络的吞吐量受多种因素影响，因此我们从网络实测数据中提取了多个关键特征，包括：接收信号强度（RSSI）、调制编码方案（MCS）和空间流数（NSS）、信号干扰噪声比（SINR）、丢包率（PER）。通过这些特征的提取，我们构建了一个多维特征矩阵 X ，用于后续的回归建模。特征提取过程中，我们特别注意了对干扰 RSSI 的处理以及邻近 AP 之间的互听性判断，以捕捉节点间的信道竞争和干扰特性。

2. 根据问题三的性质，我们选择了多种回归模型来建立吞吐量的预测模型。这些模型包括线性回归、岭回归、Lasso 回归、决策树回归、随机森林回归、梯度提升回归（GBRT）、XGBoost、支持向量回归（SVR）和神经网络回归。这些模型能够分别处理不同的回归场景和特征模式。

3. 为了保证模型的泛化能力，我们在训练过程中采用了交叉验证和正则化方法，防止模型过拟合。在正则化方面，岭回归和 Lasso 回归通过在损失函数中加入 L_2 和 L_1 正则化项，防止模型参数过大，进而提高泛化能力。神经网络回归中，我们通过引入丢弃层（dropout）来抑制过拟合现象。

在模型评估过程中，我们使用了均方误差（MSE）、平均绝对误差（MAE）和平均绝对百分比误差（MAPE）等指标来衡量模型的性能。此外，通过绘制累积分布函数（CDF），我们能够更直观地分析预测误差的分布情况，从而判断模型在不同误差区间的表现。

通过本问题的求解，我们能够基于 WLAN 的网络特征，精确预测系统的吞吐量。我们采用了多种回归模型，并结合特征工程、正则化与模型调优，构建了一个高效的吞吐。

6 模型的建立与求解

6.1 问题一模型建立与求解

WLAN（无线局域网）在现代通信中扮演着重要角色，发送机会是影响 WLAN 性能的重要因素。通过准确预测 WLAN 系统的发送机会，我们可以优化网络资源，提升通信效率。本章节采用多种机器学习算法，通过数据预处理、特征提取和模型评估等步骤，对 WLAN 系统的发送机会进行精确预测。

6.1.1 数据预处理

数据预处理是机器学习项目的基础步骤。它确保数据的质量，去除噪声和异常值，并将特征标准化，以保证模型的稳定性和泛化能力。假设我们有来自 WLAN 系统的一个多维数据集 $\{\mathbf{X}, \text{seq_time}\}$ ，其中 $\mathbf{X}$ 是输入特征矩阵， seq_time 是目标变量（即发送机会）。我们通常会进行如下几步预处理：

异常值检测与处理

异常值是指显著偏离其他数据的值，可能会对模型训练造成负面影响。假设特征矩阵 $\mathbf{X}$ 的某一系列表示信号强度 (RSSI)，记为 $\mathbf{X}_{\text{RSSI}}$ 。我们可以使用以下公式来检测异常值：

$$\text{RSSI}_{\text{异常}} = \mathbf{X}_{\text{RSSI}}[|\mathbf{X}_{\text{RSSI}} - \mu_{\text{RSSI}}| > 3\sigma_{\text{RSSI}}] \quad (6.1)$$

其中， μ_{RSSI} 是信号强度的均值， σ_{RSSI} 是信号强度的标准差。异常值通常通过设置阈值来检测，这里使用 3 倍标准差的方式。

数据标准化

为了使模型能够更好地训练，我们需要对数据进行标准化处理。标准化将每个特征归一化到零均值和单位标准差，避免特征值在数量级上差异过大而导致模型不稳定：

$$\mathbf{X}_{\text{标准化}} = \frac{\mathbf{X} - \mu_{\mathbf{X}}}{\sigma_{\mathbf{X}}} \quad (6.2)$$

其中， $\mu_{\mathbf{X}}$ 是特征矩阵 $\mathbf{X}$ 的均值， $\sigma_{\mathbf{X}}$ 是特征矩阵 $\mathbf{X}$ 的标准差。

数据缺失处理

现实中的数据往往会存在缺失项。我们可以通过均值填补法或插值法对缺失数据进行处理。假设 $\mathbf{X}_{i,j}$ 表示矩阵中的第 i 行第 j 列，若其值缺失，可以使用以下公式填充：

$$\mathbf{X}_{i,j} = \frac{1}{N-1} \sum_{k \neq i} \mathbf{X}_{k,j} \quad (6.3)$$

其中， N 是样本总数。

6.1.2 特征提取

特征提取在建模过程中起着至关重要的作用。通过提取有用的特征，我们可以使模型更好地理解并预测目标变量。针对本题的建模，我们使用了包括信号强度 (RSSI)、信道利用率 (Channel Utilization, CU)、噪声水平 (Noise Level, NL) 在内的多种常见特征。接下来对选取这三种特征的原因进行简单阐述，并介绍其计算原理。

信号强度 (RSSI)

信号强度 (RSSI) 是接收信号强度的直接测量值，能够很好地反映出接收端设备感知的信号质量。通过它模型可以评估通信链路的质量，从而有效预测网络的性能，如吞吐量、延迟等。其公式计算如下：

$$\text{RSSI} = 10 \log_{10} \left(\frac{P_{\text{received}}}{1 \text{ mW}} \right) \text{ dBm} \quad (6.4)$$

其中， P_{received} 是接收到的信号功率，单位为毫瓦 (mW)。

信道利用率 (CU)

信道利用率通过计算信道占用时间相对总时间的占比 (如公示 4.5 所示)，可直接反映出无线信道的拥塞程度。如果信道利用率过高，说明多个设备正在争用相同的信道，可能会导致冲突、传输延迟增加和吞吐量下降。因此，通过信道利用率建模，可以帮助网络系统预测拥塞，并进行资源分配和负载均衡，减轻网络瓶颈。

$$\text{CU} = \frac{\text{信道占用时间}}{\text{总时间}} \times 100\% \quad (6.5)$$

噪声水平 (NL)

噪声水平直接影响信噪比 (SNR, Signal-to-Noise Ratio) 的计算。SNR 是衡量信号质量的关键指标，高 SNR 意味着更好的通信性能。通过将噪声水平作为特征，可以帮助模型更加精确地估计 SNR，从而提高对链路质量、误码率 (BER) 和吞吐量的预测。其计算方式如下：

$$\text{NL} = 10 \log_{10} \left(\frac{P_{\text{noise}}}{1 \text{ mW}} \right) \text{ dBm} \quad (6.6)$$

其中， P_{noise} 是噪声功率，单位为毫瓦 (mW)。

6.1.3 模型构建与优化

使用预处理后的数据和提取的特征，我们可以构建机器学习模型来预测 WLAN 系统的发送机会。通过对其合理建模和预测，可以提升无线局域网的整体传输效率，减少干扰和拥塞，提高网络的吞吐量。在构建初步模型之后，我们需要进行优化，以确保模型在实际应用中的性能达到最佳。

(一) 模型构建

为了确保全面和准确的分析，我们将采用以下八种不同的机器学习模型进行研究，这些模型各具特点，能够从不同角度捕捉 WLAN 系统的复杂特性：

1. 线性回归模型

线性回归模型通过假设特征与目标变量之间是线性关系，提供直观的理解。每个特征的回归系数可以直接表示该特征对目标变量的影响大小。通过对线性回归结果的系数解读，可以帮助我们快速直接找到对发送机会影响性强弱的顺序。线性回归模型假设因变量 y 和自变量 x 之间的关系是线性的，其数学表达式如下：

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon \quad (6.7)$$

其中， β_0 为截距项， $\beta_1, \beta_2, \dots, \beta_n$ 为回归系数， ϵ 为误差项。目标是通过最小化均方误差来估计回归系数：

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (6.8)$$

2. 岭回归

岭回归是一种线性回归的改进，旨在解决多重共线性问题。在提供的 WLAN 数据中，多个特征可能高度相关。岭回归通过引入 L2 正则化项，有效地减少了多重共线性对模型的影响，从而提高了模型的稳定性和可靠性。其损失函数为：

$$L = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n \beta_j^2 \quad (6.9)$$

其中， λ 为正则化系数，通过控制模型的复杂度，防止过拟合。

3. 支持向量机 (SVM)

支持向量机 (SVM) 是一种用于分类和回归的监督学习模型，主要通过找到最优的决策平面来区分数据。SVM 对噪声和异常值具有较强的鲁棒性。其目标是最大化支持向量与决策边界之间的间隔 (margin)，这使得模型在面对数据中的噪声时仍能保持良好的性能。在 WLAN 环境中，信号强度、干扰和其他因素可能导致数据波动，SVM 能够有效抵御这些干扰，从而提供更准确的预测结果。SVR 寻找最佳超平面是通过最小化以下损失函数来实现的：

$$L = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \max(0, |y_i - \hat{y}_i| - \epsilon) \quad (6.10)$$

其中， $\mathbf{w}$ 为超平面的权重向量， C 为正则化参数， ϵ 为误差容忍度。SVR 能够在高维空间中拟合复杂的非线性关系。

4. Lasso 回归

Lasso 回归是一种 L_1 正则化的回归模型。通过 L_1 正则化，能够将一些回归系数压缩至零，从而实现自动特征选择。这有助于识别对 WLAN 性能影响最大的因素，从而对模型进行简化，对不必要的冗余特征进行过滤。其损失函数如下：

$$L = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^n |\beta_j| \quad (6.11)$$

5. 随机森林

随机森林是由多棵决策树组成的集成模型。它结合了多个树的预测结果，能够有效减少模型的方差，提高预测准确性。相比于其他模型，随机森林使用了随机抽样和特征选择，模型对训练数据的依赖性较低，抗过拟合能力强。这对于 WLAN 建模尤为重要，因为数据集中可能存在异常值和噪声。随机森林可以有效利用多样本数据，降低对特定样本的过拟合风险。其计算方式如下：

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M T_m(\mathbf{x}) \quad (6.12)$$

其中， M 是决策树的数量， T_m 表示第 m 棵决策树的预测结果。

6. K 近邻算法 (KNN)

K 近邻算法是一种简单而直观的监督学习算法，常用于分类和回归任务。它基于“相似性”原则，认为相近的数据点在特征空间中的目标值也相似。KNN 是非参数模型，不需要对数据进行假设。它根据训练数据直接进行预测，保证了用于模型训练的数据绝对符合现实情况，预测结果更贴合生活实际。在 WLAN 环境中，信号强度和干扰等因素可能复杂多变，KNN 能够灵活适应这些变化。在计算过程中，KNN 通过找到最相似的 K 个邻居，通过它们的平均值来进行预测：

$$\hat{y}_{(x)} = \frac{1}{K} \sum_{i=1}^K y_i \quad (6.13)$$

其中， y_i 是第 i 个最近邻居的值。

7. XGBoost

XGBoost 通过集成多棵决策树，采用加权投票机制，能够有效提高预测准确性。该模型能够自动处理缺失值，而无需对数据进行额外预处理。同时，XGBoost 引入了正则化项，有效抑制过拟合现象。通过控制模型的复杂度，提升了模型的泛化能力。其目标函数如下：

$$L(\theta) = \sum_{i=1}^N l(y_i, \hat{y}_i) + \sum_{m=1}^M \Omega(h_m) \quad (6.14)$$

其中, $l(y_i, \hat{y}_i)$ 为损失函数, $\Omega(h_m)$ 为正则化项, 用于控制树的复杂度。通过最小化该目标函数, XGBoost 能够在保持高预测精度的同时防止过拟合。

8. 神经网络回归

神经网络是一种模拟生物神经网络的模型, 通过多层神经元的连接, 实现对复杂非线性关系的拟合。神经网络可以从原始数据中自动学习和提取特征, 无需手动特征工程。这对于 WLAN 建模中的高维数据尤其有利, 可以减少在数据处理时人为的干预和偏差, 更有助于确保结果的准确性。每个神经元的输出由以下公式计算:

$$z = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (6.15)$$

其中, $\mathbf{w}$ 为权重向量, $\mathbf{x}$ 为输入向量, b 为偏置, σ 为激活函数。通过最小化损失函数, 神经网络能够自适应调整权重和偏置, 从而拟合训练数据。

(二) 模型优化

损失函数是衡量模型预测结果与真实值之间差距的标准。通过优化损失函数, 可以明确评估模型的性能, 并确定改进的方向。选择合适的损失函数能够更好地反映 WLAN 特定应用场景中的需求, 本论文选择均方误差 (MSE) 来优化预测准确性, 其描述如下:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\text{seq_time}_i - \text{seq_}\hat{\text{time}}_i)^2 \quad (6.16)$$

其中, N 是样本总数, seq_time_i 是第 i 个样本的真实值, $\text{seq_}\hat{\text{time}}_i$ 是第 i 个样本的预测值。

为了防止过拟合, 我们可以引入正则化项, 常见的有 L2 正则化:

$$\text{Loss} = \text{MSE} + \lambda \|\mathbf{w}\|_2^2 \quad (6.17)$$

其中, λ 是正则化参数。

6.1.4 模型预测与评估

模型建立后, 对其性能进行评估是必不可少的。评估模型的性能可以帮助我们判断模型的预测结果是否可靠。在 WLAN 环境中, 准确的预测可以直接影响到网络的吞吐量、延迟和用户体验。通过评估, 我们能够识别模型在不同场景下的表现, 确保其在实际应用中具有有良好的准确性。为了评估模型的性能, 我们通常使用多种评估指标, 包括均方误差 (Mean Squared Error, MSE)、平均绝对误差 (Mean Absolute Error, MAE)、平均绝对百分比误差 (Mean Absolute Percentage Error, MAPE) 和累积分布函数 (Cumulative Distribution Function, CDF)。

均方误差 (MSE)

均方误差是通过计算预测值与真实值之间差的平方的平均值来衡量模型性能的指标。MSE 对大误差给予了更高的惩罚，因此适合用于需要严格控制大误差的场景。在 WLAN 建模中，较低的 MSE 意味着模型的预测结果更接近真实情况，从而反映出网络性能的准确性。均方误差 (MSE) 用于度量预测值与真实值之间的差异，其公式为：

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

其中， $\hat{y}_i$ 为第 i 个样本的预测值， y_i 为第 i 个样本的真实值， N 为样本数量。MSE 反映了误差的平方和平均，较大的误差对 MSE 的贡献较大，因此该指标对异常值较为敏感。

平均绝对误差 (MAE)

平均绝对误差 (MAE) 表示预测值与真实值之间的平均绝对差。与 MSE 相比，MAE 对异常值的敏感性较低，能够提供更为直观的误差测量。在 WLAN 建模中，MAE 可以帮助我们了解模型在不同条件下的实际预测能力，尤其在处理有噪声的环境时，其优势尤为明显。MAE 的计算方式如下：

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

MAE 比 MSE 对异常值不敏感，能够更直接地反映预测误差的平均水平。

平均绝对百分比误差 (MAPE)

平均绝对百分比误差 (MAPE) 将绝对误差与真实值进行比较，通常以百分比形式表示。这一指标特别适用于需要考虑预测结果相对真实值的情况。在复杂的 WLAN 系统中，可能存在多个性能目标（如吞吐量、延迟等）。MAPE 可以同时评估不同指标的相对表现，为综合优化提供数据支持。其计算方法为：

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{\hat{y}_i - y_i}{y_i} \right| \times 100$$

MAPE 表示预测值相对于真实值的平均百分比误差，适合用于度量误差的相对大小。

累积分布函数 (CDF)

累积分布函数 (CDF) 是概率论中的一个基本概念，用于描述随机变量取值的累积概率分布。CDF 可以直观地展示模型在不同阈值下的性能表现，使得用户能够明确在特定条件下的预测准确性。这对于评估 WLAN 中流量波动和信号强度等动态因素的影响尤为重要。给定一个随机变量 X ，其累积分布函数 $F(x)$ 定义为：

$$F(x) = P(X \leq x)$$

即 $F(x)$ 表示随机变量 X 小于或等于某个特定值 x 的概率。CDF 的值范围为 $[0, 1]$ ，表示在 x 之前的所有概率之和。CDF 的特点如下：

- 当 $x \rightarrow -\infty$ 时， $F(x) \rightarrow 0$ ，表示随机变量不可能取到非常小的值。
- 当 $x \rightarrow \infty$ 时， $F(x) \rightarrow 1$ ，表示随机变量几乎肯定小于或等于非常大的值。

CDF 的导数是概率密度函数（PDF），因此 CDF 表示了累积概率，而 PDF 则表示特定区间内的概率密度。

为了更好地分析预测误差的分布情况，我们可以计算误差的累积分布函数（CDF）。通过绘制 CDF 曲线，可以展示误差的累积概率分布，分析出模型在不同误差区间上的表现。

具体步骤如下：

- 计算每个样本的预测误差：Error = $\hat{y} - y$
- 计算误差百分比：Error Percentage = $\frac{\hat{y} - y}{y} \times 100$
- 将误差百分比按升序排列，并计算其 CDF 值：CDF 的值为当前误差百分比的排序位置除以总样本数，即 $F(x) = \frac{k}{N}$ ，其中 k 是样本在排序中的位置， N 是总样本数。

通过分析 CDF 曲线，我们可以直观了解大部分预测误差的范围，并计算模型的精度。

对于问题一采用了多种机器学习模型（包括线性回归、多层感知器、支持向量机、决策树、随机森林、KNN、XGBoost 和 LSTM）对 WLAN 系统的发送机会进行预测。每种机器学习模型都有其独特的特征和优点。多种机器学习模型，多方面的分析视角，能够提升我们结果分析的准确性，更贴合实际生活。并且，通过详细的数据预处理、特征提取和模型构建过程，我们能够有效地训练和优化各类模型。未来研究可以进一步探讨模型在实际网络环境中的可移植性和稳定性，并结合更多的网络特征和优化策略，以提高模型的预测精度和鲁棒性。

6.2 问题二模型建立与求解

在 WLAN 网络中，接入点（AP）选择合适的调制编码方案（MCS）和空间流数（NSS）对于提升网络吞吐量至关重要。问题二的目标是基于实测数据，利用网络拓扑、节点间 RSSI、门限等信息，建立模型预测 AP 发送数据时选用最多次数（MCS, NSS）。我们将对物理层模型、SINR 的计算以及 AMC 算法进行简单介绍，并采用多种分类模型进行建模。

6.2.1 信号干扰与 SINR 建模

SINR 是影响无线通信质量的重要指标，定义为信号功率与干扰加噪声功率的比值。从计算方法上看，SINR 不仅考虑了接收到的信号强度，还考虑了干扰和噪声的影响，贴合实际 WLAN 信道中存在的多干扰事实，能够更全面地反映无线信道的质量。可以表示为：

$$SINR = \frac{P_{signal}}{P_{interference} + P_{noise}} \quad (6.18)$$

其中, P_{signal} 是目标 AP 发送信号的功率, $P_{interference}$ 是邻近 AP 产生的干扰功率, P_{noise} 是背景噪声的功率。

假设干扰功率主要由其他 AP 对目标 STA 的干扰造成, 那么:

$$P_{interference} = \sum_{i=1}^N P_{AP_i} \cdot L_i \quad (6.19)$$

算法 1 Calculate SINR for a given data row

Input: Data row containing RSSI values and identifiers

Output: Calculated SINR in dB

```
1: function CalculateSINR(row)
2: Extract the AP ID from the ap_id field
3: Extract the STA ID from the sta_id field
4: Compute own_ap_rssi_col  $\leftarrow$  'sta_from_ap_{ap_id}_sum_ant_rssi'
5: Retrieve own_rssi_dBm from row[own_ap_rssi_col]
6: Initialize interference_mW  $\leftarrow$  0
7: for col in row.columns do
8:   if col starts with 'sta_from_ap_' and ends with '_sum_ant_rssi' then
9:     Extract ap_x from col
10:    if ap_x  $\neq$  ap_id then
11:      Retrieve interference_rssi_dBm from row[col]
12:      if interference_rssi_dBm is not NaN then
13:        if interference_rssi_dBm is a string then
14:          Parse interference_rssi_dBm using ast.literal_eval
15:          Compute mean value of interference_rssi_dBm
16:          Convert interference_rssi_dBm to mW and add to interference_mW
17: if own_rssi_dBm is a string then
18:   Parse own_rssi_dBm using ast.literal_eval
19:   Compute mean value of own_rssi_dBm
20: if own_rssi_dBm is not NaN then
21:   Compute desired_mW  $\leftarrow 10^{\left(\frac{\text{own\_rssi\_dBm}-30}{10}\right)}$ 
22: else
23:   return NaN
24: Set noise_floor_dBm  $\leftarrow$  -95
25: Compute noise_mW  $\leftarrow 10^{\left(\frac{\text{noise\_floor\_dBm}-30}{10}\right)}$ 
26: Compute sinr_linear  $\leftarrow \frac{\text{desired\_mW}}{\text{interference\_mW} + \text{noise\_mW}}$ 
27: Compute sinr_dB  $\leftarrow 10 \times \log_{10}(\text{sinr\_linear})$ 
28: return sinr_dB
```

其中, P_{AP_i} 是第 i 个 AP 的发射功率, L_i 是第 i 个 AP 到目标 STA 的路径损耗。路径损耗可以用对数距离模型表示为:

$$L_i = \left(\frac{d_i}{d_0} \right)^\alpha \quad (6.20)$$

其中, d_i 为 AP 到 STA 的距离, d_0 为参考距离, α 为路径损耗指数。

背景噪声 P_{noise} 通常认为是一个常量, 可以根据实际场景设定。关于 SINR 的实现如算法1所示。

6.2.2 AMC 算法

自适应调制编码 (AMC) 算法根据当前的信道质量 (通过 SINR 评估) 选择合适的 MCS 和 NSS 组合, 从而优化数据传输速率。当 SINR 值高时, 信道质量良好; 反之, 若 SINR 值低, 则表明信道质量较差。基于 SINR 的评估结果, AMC 选择合适的 MCS 和 NSS 组合, 以实现最大化系统吞吐量。例如, 当 SINR 超过某一阈值时, 算法会降低 MCS 和 NSS, 以实现更高的数据传输速率; 反之则提高 MCS 和 NSS, 以保证信号的可靠传输。假设有 M 种不同的 MCS 组合, 每种 MCS 组合都有其最低所需的 SINR 门限值 γ_m , 那么 AMC 算法可以表示为:

$$\text{Select}(MCS, NSS) = \arg \max_m \{m \mid SINR \geq \gamma_m\} \quad (6.21)$$

总之, AMC 算法具有动态性。当无线信道的质量变化时, AMC 能够迅速作出反应, 及时调整 MCS 和 NSS 的组合, 从而确保系统始终在最优状态下运行。

6.2.3 丢包率与 PHY 层速率的关系

根据 AMC 算法, 选定的 MCS 和 NSS 决定了物理层的传输速率 R_{phy} 。物理层的传输速率通常以比特每秒 (bps) 为单位, 表示在特定时间内可以成功发送的比特数。传输速率越高, 单位时间内成功传输的数据比特数越多, 但同时也意味着对信道质量要求更高。

假设在时间 t 内, 传输的包数为 N_{pkt} , 则丢包率 P_{error} 可以表示为:

$$P_{error} = \frac{N_{error}}{N_{pkt}} \quad (6.22)$$

其中 N_{error} 为传输失败的包数。 P_{error} 与 PHY 层速率 R_{phy} 的关系可以用一个经验公式表示为:

$$P_{error} = f(R_{phy}, SINR) \quad (6.23)$$

当 SINR 较高时, 系统可以选择较高的 PHY 层速率 R_{phy} , 但此时丢包率也会上升。因此, AMC 算法需要在传输速率与丢包率之间找到一个平衡点。在信道质量良好时, 算法可以选择更高的 MCS 和 NSS, 以实现更快的传输速率, 增加用户的体验感; 而在信道

质量下降时，算法会自动调整，选择适当的 MCS 和 NSS 组合，降低传输速率以保证数据的可靠传输。

6.2.4 吞吐量

WLAN（无线局域网）系统的吞吐量是一个关键性能指标，它被定义为单位时间内成功传输的比特数。具体而言，这一指标衡量的是在特定时间段内，网络能够有效传输的数据量，通常以比特每秒（bps）为单位表示，记作 $T_{throughput}$ 。高吞吐量是我们布局 WLAN 网络时的重要目的之一。吞吐量越高，用户可以更快地下载文件、观看高清视频或进行在线游戏，更好的满足用户对日常办公、生活的需求。对于一个 AP，其吞吐量可以表示为：

$$T_{throughput} = (1 - P_{error}) \cdot R_{phy} \quad (6.24)$$

这里， P_{error} 表示传输失败的概率， R_{phy} 表示物理层传输速率。

进一步的， R_{phy} 与所选用的 MCS 和 NSS 相关，可以表示为：

$$R_{phy} = f(MCS, NSS) \quad (6.25)$$

结合丢包率的表达式，最终的吞吐量可以表示为：

$$T_{throughput} = (1 - f(R_{phy}, SINR)) \cdot f(MCS, NSS) \quad (6.26)$$

6.2.5 多分类模型的建立

为了准确预测 AP 发送数据时选用最多的 (MCS, NSS) 组合，我们将使用多种分类模型进行建模和预测。每个模型都有其独特的数学原理和适用场景，因此在不同的特征表现和数据分布下，不同的分类模型可能表现出不同的性能。以下将详细介绍每种分类模型的理论基础、公式推导以及模型中的参数含义。我们将分别讨论以下 8 种模型：Logistic 回归、Softmax 回归、随机森林、支持向量机（SVM）、K-近邻（KNN）、神经网络（Neural Networks）、XGBoost、LightGBM。通过对不同分类结果的分析，选择适合的分类型模型，我们可以更有效地预测 AP 在不同条件下的最优选择，从而提升无线通信系统的整体性能。

1. Logistic 回归分类器

Logistic 回归是一种经典的分类方法，最常用于二分类问题。其基本思想是通过线性模型来估计某个事件发生的概率，得到一个实数值，并使用逻辑函数（Logistic Function，也称为 sigmoid 函数）将线性模型的输出值映射到 $[0, 1]$ 区间，从而得到事件发生的概率。sigmoid 函数的形状类似于 S 型曲线，使得无论输入值多大或多小，输出值都能有效地限制在 $[0, 1]$ 之间。使用 Logistic 回归进行建模能够输出每个 (MCS, NSS) 组合的概率，便于分析在特定条件下选择某个发送速率的可能性，并且可以清晰地看到 RSSI 和门限信息如何影响 (MCS, NSS) 的选择。

对于二分类问题，Logistic 回归的公式为：

$$P(y = 1|X) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{X} + b)} \quad (6.27)$$

其中：

- $P(y = 1|X)$ ：表示输入特征 $\mathbf{X}$ 属于类别 $y = 1$ 的概率。
- $\mathbf{w}$ ：权重向量，用于控制特征对分类结果的影响。
- $\mathbf{X}$ ：输入特征向量，包括 RSSI、SINR 等特征。
- b ：偏置项。

为了扩展到多分类问题，我们可以采用“一对多”的策略，针对每个类别 m 训练一个二分类模型，并输出每个类别的概率：

$$P(y = m|X) = \frac{1}{1 + \exp(-\mathbf{w}_m^T \mathbf{X} + b_m)} \quad (6.28)$$

然后将所有类别的概率进行比较，选择概率最大的类别作为最终分类结果。这种方式将 Logistic 回归从二分类扩展到多分类任务中。尽管 Logistic 回归的模型较为简单，但在数据呈线性可分时，通常能取得较好的效果。

2. Softmax 回归模型

Softmax 回归是一种用于多分类问题的通用分类器。与 Logistic 回归类似，它也依赖于线性模型，但其输出不再是一个二值概率，而是多个类别的概率分布。Softmax 函数能够将线性模型的输出值转化为一组概率值，且这些概率值的和为 1。在 WLAN 网络中，中常用的 (MCS, NSS) 组合可以有多个，Softmax 回归能够同时处理这些多类别选择，更好地适应自适应速率控制需求。并且对于多输入特征（如 RSSI、信道状态、用户密度等，Softmax 回归能有效处理这些高维数据，提升模型的表现。

Softmax 回归的模型公式如下：

$$P(y = m|X) = \frac{\exp(\mathbf{w}_m^T \mathbf{X} + b_m)}{\sum_{j=1}^C \exp(\mathbf{w}_j^T \mathbf{X} + b_j)} \quad (6.29)$$

其中：

- $P(y = m|X)$ ：表示输入特征 $\mathbf{X}$ 属于类别 m 的概率。
- $\mathbf{w}_m$ ：类别 m 对应的权重向量。
- b_m ：类别 m 的偏置项。
- C ：类别的总数，即不同的 (MCS, NSS) 组合数量。
- $\mathbf{X}$ ：输入的特征向量。

Softmax 回归的目标是通过最大化输入特征属于某个类别的概率来进行分类。与 Logistic 回归类似，Softmax 回归同样可以通过最大化似然估计来学习模型参数 w_m 和 b_m 。在训练过程中，通常使用交叉熵损失函数作为优化目标：

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(P(y_i = c|X_i)) \quad (6.30)$$

其中：

- N ：训练样本的数量。
- C ：类别的数量。
- $y_{i,c}$ ：第 i 个样本属于类别 c 的真实标签（采用 one-hot 编码）。
- $P(y_i = c|X_i)$ ：第 i 个样本的预测概率。

通过最小化交叉熵损失，Softmax 回归能够学习出最佳的模型参数，以便更好地分类。

3. 随机森林分类器

随机森林（Random Forest）是一种集成学习方法，基于多个决策树（Decision Tree）的集成进行预测。它通过生成多个不同的决策树，并通过对每棵树的预测结果进行投票，得到最终的分类结果。相比单一的决策树模型，随机森林具有更强的鲁棒性和泛化能力。随机森林不依赖于特定的数据分布假设，不需要对数据进行复杂的预处理，可以直接使用 WLAN 网络中收集到的各个特征数据准确值。

随机森林的核心思想是：每棵树使用的是训练集的不同子集，并且在每棵树的节点划分过程中，随机选择一部分特征进行判断。这样做不仅减少了模型的过拟合，还增强了模型对数据变化的适应性。尤其是针对 WLAN 网络出现的噪声大、干扰强等问题。

随机森林的分类结果可以表示为：

$$P(y = m|X) = \frac{1}{T} \sum_{t=1}^T P_t(y = m|X) \quad (6.31)$$

其中：

- T ：决策树的数量。
- $P_t(y = m|X)$ ：第 t 棵决策树对类别 m 的预测概率。

随机森林的训练过程中，每棵树都会学习不同的特征子集，从而保证了模型的多样性。

4. 支持向量机分类器 (SVM)

支持向量机（SVM）是一种用于分类的监督学习模型，其核心思想是寻找一个最优的超平面，将不同类别的样本点最大程度地分开。SVM 模型通过最大化支持向量间的“间隔”（Margin）来提高分类的准确性，这在动态和变化的 WLAN 环境中能减少分类错误。

另一方面，SVM 通过核函数能够有效捕捉 AP 发送机会与输入特征（如 RSSI、SINR 等）之间的非线性关系，适应复杂的无线信号环境。

对于线性可分的数据，SVM 的优化目标可以表示为：

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \max(0, 1 - y_i(w^T X_i + b)) \quad (6.32)$$

其中：

- $\|w\|^2$ ：权重向量的二范数，用于最大化分类间隔。
- C ：正则化参数，用于平衡分类间隔与误分类的惩罚。
- y_i ：样本的真实标签， $y_i \in \{-1, 1\}$ 。
- X_i ：第 i 个样本的特征向量。
- b ：超平面的偏置项。

对于非线性可分的数据，SVM 通过引入核函数 $K(X_i, X_j)$ 将输入空间映射到高维特征空间，从而在高维空间中找到线性可分的超平面。常用的核函数包括：

- 线性核： $K(X_i, X_j) = X_i^T X_j$
- 多项式核： $K(X_i, X_j) = (X_i^T X_j + c)^d$
- 高斯核： $K(X_i, X_j) = \exp\left(-\frac{\|X_i - X_j\|^2}{2\sigma^2}\right)$

其中：

- σ 是高斯核函数的参数，控制核函数的带宽；
- $K(X_i, X_j)$ 是核函数的输出值，用于度量样本 X_i 和 X_j 在映射空间中的相似度；
- X_i 和 X_j 是输入特征向量。

SVM 模型的最终目标是找到一个最优的超平面，使得数据在高维空间中可以尽可能地被分隔开。对于多分类问题，SVM 可以通过“一对一”或“一对多”的方式进行扩展，将多个二分类问题组合起来实现多类别的分类。

5. K-近邻分类器 (KNN)

K-近邻分类器是一种基于距离度量的非参数分类方法，其基本思想是：对于一个未分类的样本，找到训练集中距离它最近的 k 个样本，根据这些样本的类别进行投票，选择出现频率最高的类别作为该样本的分类结果。KNN 通过考虑数据点之间的距离关系，能够充分利用邻近点的信息，这在 WLAN 环境中能够有效反映出相似条件下的 (MCS, NSS) 选择。

KNN 的公式表示如下：

$$P(y = m|X) = \frac{1}{k} \sum_{i=1}^k I(y_i = m) \quad (6.33)$$

其中：

- $P(y = m|X)$: 表示输入样本 X 属于类别 m 的概率；
- k : 邻居的数量，即选择与输入样本最接近的 k 个样本；
- y_i : 第 i 个邻居的类别标签；
- $I(y_i = m)$: 指示函数，若 $y_i = m$ 则 $I = 1$ ，否则 $I = 0$ 。

KNN 分类的过程可以分为以下步骤：

1. 计算测试样本与训练集中每个样本的距离，常用的距离度量包括欧氏距离、曼哈顿距离等；
2. 根据距离从小到大排序，选取距离最近的 k 个邻居；
3. 对这 k 个邻居的类别进行投票，选择票数最多的类别作为分类结果。

6. 神经网络分类器 (Neural Networks)

神经网络是一种由大量神经元组成的网络结构，能够通过多个隐层进行特征提取和非线性映射。神经网络的核心思想是：输入特征通过多个隐藏层的加权求和和激活函数处理，最终输出一个分类结果。通过多层结构，神经网络能够学习到高维特征的抽象表示，有效捕捉输入特征之间的复杂非线性关系。这在 WLAN 环境中尤其重要，因为 (MCS, NSS) 选择常常受到多种因素（如 RSSI、干扰、用户数量等）的影响。并且，神经网络能够轻松扩展，随着收集到的 WLAN 实测数据量的增加，可以增加更多的层或节点来提高模型性能。

神经网络中每个神经元的计算公式为：

$$z_i^{(l)} = \sigma \left(\sum_j W_{ij}^{(l)} z_j^{(l-1)} + b_i^{(l)} \right) \quad (6.34)$$

其中：

- $z_i^{(l)}$ 是第 l 层中第 i 个神经元的输出；
- $W_{ij}^{(l)}$ 是第 l 层中第 i 个神经元与第 $(l-1)$ 层中第 j 个神经元之间的权重；
- $z_j^{(l-1)}$ 是第 $(l-1)$ 层中第 j 个神经元的输出；
- $b_i^{(l)}$ 是第 l 层中第 i 个神经元的偏置项；
- $\sigma(\cdot)$ 是激活函数，如 ReLU、sigmoid 等。

在多分类问题中，神经网络的输出层通常会使用 Softmax 激活函数，将神经元的输出值映射到概率分布。网络的训练过程采用反向传播算法 (Backpropagation)，通过计算损失函数（通常是交叉熵损失）并使用梯度下降法更新参数。

7. XGBoost 分类器

XGBoost (Extreme Gradient Boosting) 是基于梯度提升决策树 (GBDT) 的一种高效实现。其基本思想是：通过加权组合多个弱分类器（通常是决策树），逐步减少模型的误差，

从而提高整体的分类性能。XGBoost 能够提供每个特征对预测结果的重要性评分，有助于分析在 AP 发送机会中哪些因素（如 RSSI、用户密度等）对（MCS, NSS）的选择影响最大，这对我们的结果分析提供明确的方向。

XGBoost 的预测公式为：

$$P(y = m|X) = \sum_{k=1}^K f_k(X), \quad f_k \in \mathcal{F} \quad (6.35)$$

其中：

- $P(y = m|X)$ ：表示输入特征 $\mathbf{X}$ 属于类别 m 的预测结果；
- K ：弱分类器的数量；
- $f_k(X)$ ：第 k 个弱分类器的预测输出；
- $\mathcal{F}$ ：所有可能的决策树集合。

XGBoost 通过最小化目标函数来优化决策树的结构和参数。目标函数包括两部分：损失函数和正则化项。损失函数用于度量模型的误差，正则化项用于防止模型过拟合。XGBoost 的训练过程使用贪心算法逐步构建决策树，并通过梯度提升的方法不断优化。

8. LightGBM 分类器

LightGBM（Light Gradient Boosting Machine）是另一种基于梯度提升的决策树模型。LightGBM 能够直接处理类别特征，而不需要额外的独热编码，简化了数据预处理流程，提高了建模效率。并且，其通过高效的贪心算法构建决策树，能够生成更深和更复杂的树结构，尤其在复杂信号环境下，能够更准确地预测（MCS, NSS）组合。

LightGBM 的基本公式与 XGBoost 类似：

$$P(y = m|X) = \sum_{i=1}^L g_i(X), \quad g_i \in \mathcal{G} \quad (6.36)$$

其中：

- $P(y = m|X)$ ：输入特征 $\mathbf{X}$ 属于类别 m 的预测概率；
- L ：树的数量；
- $g_i(X)$ ：第 i 棵树的预测结果；
- $\mathcal{G}$ ：所有可能的树模型集合。

LightGBM 采用了基于直方图的决策树学习算法，通过将特征值离散化，减少了计算复杂度。它还使用了叶子级别的增长策略（Leaf-wise Growth Strategy），通过在信息增益最大的叶子节点继续生长，进一步提升了模型的性能，尤其在处理不平衡数据时具有显著优势。在 LightGBM 的训练过程中，同样通过最小化损失函数来调整决策树的结构和参数。损失函数由两部分组成：模型误差和正则化项。正则化项可以有效防止模型过拟合。

6.2.6 多分类模型的预测与评估

模型预测

在测试阶段，对于每个 AP，我们输入测试集中的特征，利用训练好的分类器预测其最有可能的 (MCS, NSS) 组合。通过计算分类器的概率输出，我们选择概率最大的类别作为预测结果。

假设 Softmax 分类器的输出为 $P(y = i|X)$ ，则预测的 (MCS, NSS) 组合为：

$$(MCS, NSS) = \arg \max_i P(y = i|X) \quad (6.37)$$

类似地，对于其他分类器，我们也可以采用类似的策略选择最优的 (MCS, NSS) 组合。

本次建模的目标是预测 WLAN 系统中每个 AP 发送数据时使用最多的 (MCS, NSS) 组合。通过分析 SINR、RSSI 等信道特征，并结合 AMC 算法的工作原理，我们建立了多个分类模型来解决该问题。最终的预测结果将基于不同分类模型的性能表现，选择最优的模型进行预测。

模型评估

在训练完成后，我们可以使用验证集进行模型评估。对于每个分类器，我们计算其分类准确率和损失函数。假设分类结果为 $\hat{y}$ ，真实标签为 y ，则分类准确率可以表示为：

$$Accuracy = \frac{1}{N} \sum_{i=1}^N I(\hat{y}_i = y_i) \quad (6.38)$$

其中， $I(\cdot)$ 为指示函数，当预测结果与真实标签相同时， $I(\cdot) = 1$ ，否则为 0。

此外，我们还会计算交叉熵损失函数，表示为：

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(P(\hat{y}_i = c|X_i)) \quad (6.39)$$

其中， $y_{i,c}$ 为真实类别标签的 one-hot 表示， $P(\hat{y}_i = c|X_i)$ 为模型预测的第 c 类的概率。

6.3 问题三模型建立与求解

无线局域网（WLAN）在高密度场景下，由于节点间的干扰、信道竞争以及异步传输等问题，导致网络的吞吐量大幅下降。为了优化网络性能并提高吞吐量的准确预测，我们基于 WLAN 的实测数据，使用机器学习方法构建网络吞吐量预测模型。目标是通过分析网络拓扑、信道竞争接入机制、节点间的 RSSI、干扰等特征，精确预测 WLAN 的系统吞吐量。WLAN 的系统吞吐量受到 PHY 速率、传输过程中的丢包现象、接收信号强度以及邻近 AP 之间的互听、干扰等因素都有关系。接下来将对这几个影响因素进行简单的介绍。

6.3.1 PHY 速率

在精确预测 WLAN 的系统吞吐量过程中,PHY 速率的计算是至关重要的。因为 WLAN 的吞吐量与所选用的调制和编码方案 (MCS) 以及空间流数 (NSS) 直接相关。PHY 速率的高低将影响数据传输的效率,从而影响整体吞吐量。在高干扰环境中,适应性调制与编码 (AMC) 算法会根据实际的信道条件动态调整所选用的 (MCS, NSS), 确保在尽可能高的 PHY 速率下进行数据传输。我们使用以下公式计算 PHY 速率, 基于 MCS 和 NSS 的组合查表获得实际速率值:

$$\text{PHY Rate} = f(\text{MCS}, \text{NSS})$$

对于常见的 MCS 和 NSS 组合, 其 PHY 速率如下表所示:

表 6.2 PHY 速率表 (单位: Mbps)

MCS	NSS=1	NSS=2
0	8.6	17.2
1	17.2	34.4
2	25.8	51.6
3	34.4	68.8
4	51.6	103.2

6.3.2 有效数据速率

PHY 速率并不代表最终的数据速率。在数据传输过程中, 如果发生丢包, 需进行重传, 这会消耗额外的带宽和时间, 导致有效传输速率降低。同时, 数据包的有效到达率减少, 吞吐量也随之下降。有效数据速率可以通过以下公式计算:

$$\text{Effective Rate} = \text{PHY Rate} \times (1 - P_{\text{error}})$$

其中, P_{error} 表示丢包率, 反映了数据传输过程中因干扰、竞争等导致的失败帧比例。

6.3.3 接收信号强度 (RSSI) 提取

WLAN 的吞吐量与接收信号强度 (RSSI) 密切相关。在实际应用中, WLAN 环境通常动态多变, 例如设备移动、障碍物变化或网络负载波动等, 都可能导致 RSSI 的波动。RSSI 的波动会影响 AP 和 STA (终端设备) 之间的连接质量, 进而影响吞吐量。我们通过以下公式提取 RSSI 值:

$$\text{RSSI}_{\text{AP-STA}} = \sum_{i=1}^{N_{\text{ant}}} \text{RSSI}_i$$

其中， N_{ant} 表示 AP 或 STA 的天线数量， RSSI_i 为第 i 根天线接收到的信号强度。更高的 RSSI 通常对应更好的信号质量和更高的吞吐量。

6.3.4 AP 间互听性

为了判断 AP 之间是否能够侦听对方的信号，我们将 AP 之间的 RSSI 与不同的检测门限（如物理层侦听门限 PD、能量检测门限 ED、网络分配向量 NAV）进行对比是：

$$\text{Can Hear}_{\text{PD}} = \begin{cases} 1, & \text{if } \text{RSSI}_{\text{AP}} \geq \text{PD} \\ 0, & \text{if } \text{RSSI}_{\text{AP}} < \text{PD} \end{cases} \quad (6.40)$$

$$\text{Can Hear}_{\text{ED}} = \begin{cases} 1, & \text{if } \text{RSSI}_{\text{AP}} \geq \text{ED} \\ 0, & \text{if } \text{RSSI}_{\text{AP}} < \text{ED} \end{cases} \quad (6.41)$$

$$\text{Can Hear}_{\text{NAV}} = \begin{cases} 1, & \text{if } \text{RSSI}_{\text{AP}} \geq \text{NAV} \\ 0, & \text{if } \text{RSSI}_{\text{AP}} < \text{NAV} \end{cases} \quad (6.42)$$

当 AP 之间的 RSSI 高于相应的检测门限时，它们能够相互侦听，从而减少信道冲突和干扰，提升 WLAN 整体吞吐量。然而，如果 RSSI 低于门限，AP 可能无法检测到对方的信号，导致潜在的信道争用和数据包冲突，网络的吞吐量就会降低。

6.3.5 干扰 RSSI 提取

邻近 AP 之间的干扰会显著影响网络的吞吐量。这一现象在高密度无线网络环境中尤为明显。当多个 AP 在相近的频率范围内工作时，它们之间的信号会互相干扰，导致接收信号质量下降。这种干扰会直接影响接收信号强度指示（RSSI）和信噪比（SINR），进而影响设备所能选择的调制和编码方案（MCS）。为了量化干扰，我们提取了 AP 之间的干扰 RSSI 值，包括均值和最大值：

$$\text{Interfering RSSI}_{\text{mean}} = \frac{1}{N_{\text{int}}} \sum_{j=1}^{N_{\text{int}}} \text{RSSI}_j$$

$$\text{Interfering RSSI}_{\text{max}} = \max(\text{RSSI}_1, \text{RSSI}_2, \dots, \text{RSSI}_{N_{\text{int}}})$$

其中， N_{int} 表示邻近 AP 的数量， RSSI_j 为第 j 个 AP 的干扰 RSSI。

6.3.6 机器学习模型建立

根据对题目分析可知，问题三和问题一样都是回归问题。在问题一当中我们使用了八种机器学习模型，不同模型可以从不同的角度理解数据。使用多种模型有助于捕捉更多的数据特征和潜在模式，从而提供更全面的视角，进一步保障了结果的准确性。在第三题我们使用同样的机器学习模型，模型细节在问题一已经详细描述过，这里只做简单介绍：

- 线性回归：线性回归通过拟合一条直线来描述自变量与因变量之间的线性关系。
- 岭回归：岭回归是线性回归的一种扩展，加入 L2 正则化项以减少多重共线性和过拟合风险。
- 支持向量机：SVM 是一种强大回归模型，通过寻找最佳超平面来最大化分类边界。
- Lasso 回归：Lasso 回归通过 L1 正则化实现特征选择，能够将某些回归系数压缩为零，实现模型复杂度的降低并提高解释性。
- 随机森林：随机森林是一种集成学习方法，通过构建多棵决策树并对其结果进行平均来提高预测准确性。
- K 临近算法：KNN 是一种基于实例的学习方法，通过计算新样本与训练样本的距离，选择最近的 K 个邻居进行预测。
- XGBoost：XGBoost 是一种高效的梯度提升树（GBDT）算法，结合了决策树的优点，具有高效、灵活、准确的特点。
- 神经网络回归：神经网络通过模拟生物神经元的工作方式，通过构建多层网络进行非线性映射来处理复杂的回归问题。

6.3.7 模型预测与评估指标

同样的，由于问题三和问题一样都是回归问题，使用了同样的机器学习模型，我们对模型的预测和评估指标也采用同样的方法。评估模型性能是确保其有效性和可靠性的关键步骤。通过使用验证集进行评估，我们可以量化模型在未知数据上的表现，识别潜在的过拟合或欠拟合问题。评估方法以及具体实现细节我们已经在问题一中详细阐述，在此只做简单的回顾：

- 均方误差（MSE）：MSE 衡量预测值与真实值之间的平方差的平均值。它对较大的误差赋予更高的权重，对离群值较敏感。
- 平均绝对误差（MAE）：MAE 计算预测值与真实值之间绝对差的平均值。它对所有误差的权重相同，提供了一个更直观的误差度量。
- 平均绝对百分比误差（MAPE）：MAPE 衡量预测值与真实值之间绝对误差占真实值的百分比的平均值。它提供了一个相对误差的度量，便于在不同规模的预测任务之间进行比较。
- 误差的累积分布函数（CDF）：CDF 用于展示误差的分布情况，显示小于或等于某一特

定值的误差的概率。这可以分析模型在不同误差范围内的表现，识别误差的集中程度和极端情况。

7 实验结果分析

7.1 问题一结果分析

首先对训练数据集中各个特征之间的相关性进行可视化及分析，如图7.54所示，eirp与seq_time（发送数据帧序列的总时长）相关性最强，其次是nav，protocol与loc_id。

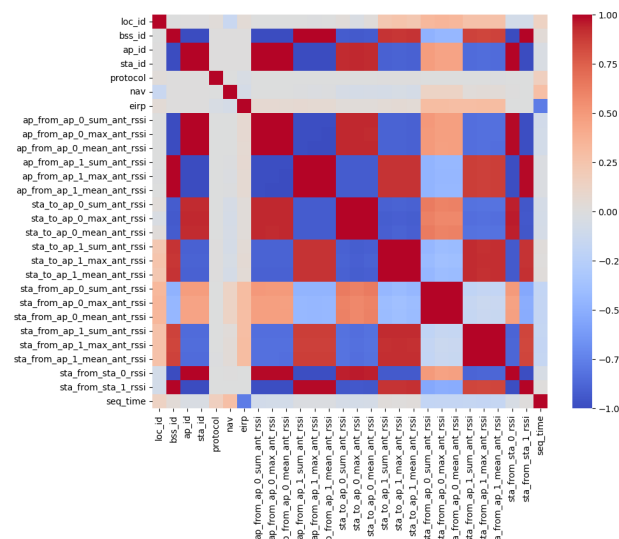


图 7.8 2AP 热力图

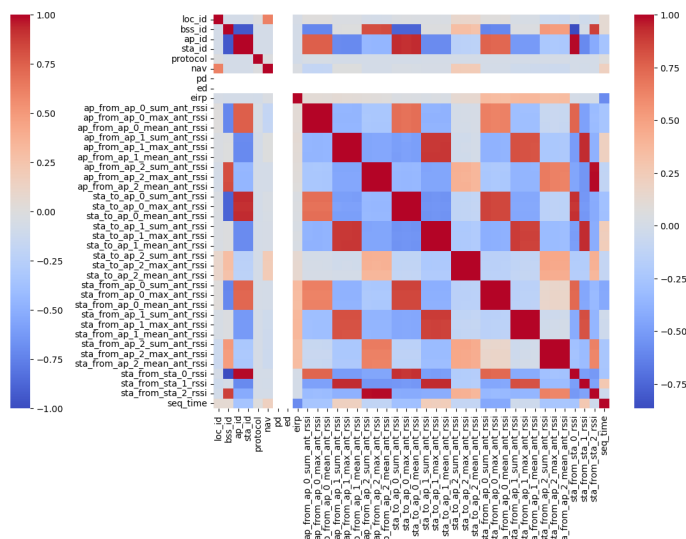


图 7.9 3AP 热力图

分析特征相关性热力图可以发现，在 2AP 结构下，与 seq_time 相关性最强的几个特征为 eirp, nav, protocol 和 loc_id，在 3AP 结构下，与 seq_time 相关性最强的几个特征为 eirp, nav, protocol 和 loc_id。

7.1.1 变量间的相关性分析

为进一步比较相关性，我们通过随机森林和 Xgboost 两种方法对特征相关性进行进一步评估，结果如图7.62和7.65所示。

可以进一步观察到，eirp 是相关性最强的特征，其次是 protocol, nav 与 loc_id。

可以进一步观察到,3AP 结构下参数之间的相关更为复杂，但 eirp 仍然是相关性很强的特征。

因为 AP 的发送功率（eirp）增加会直接导致其发射的信号强度（RSSI）提升。RSSI 是衡量信号强度的关键指标，高 RSSI 意味着信号强、干扰小，从而增加了数据传输成功的概率。即同步传输的次数更多，而同步传输时，每个 AP 交替抢到信道，偶然同时发送，发送时长基本上各占一半。因此，eirp 与 seq_time 存在较强的负相关。这也验证了结果的合理性。

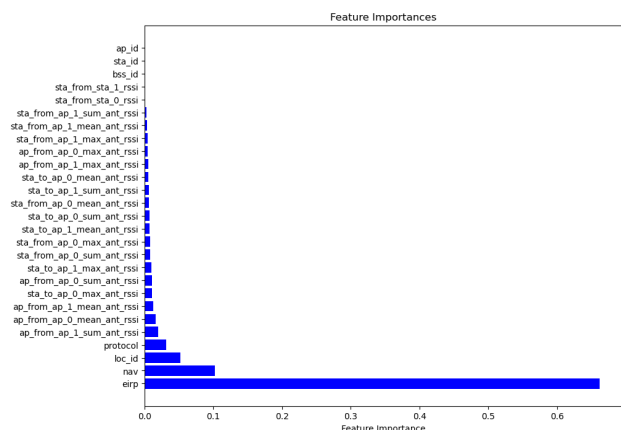


图 7.10 2AP 随机森林变量重要性排序

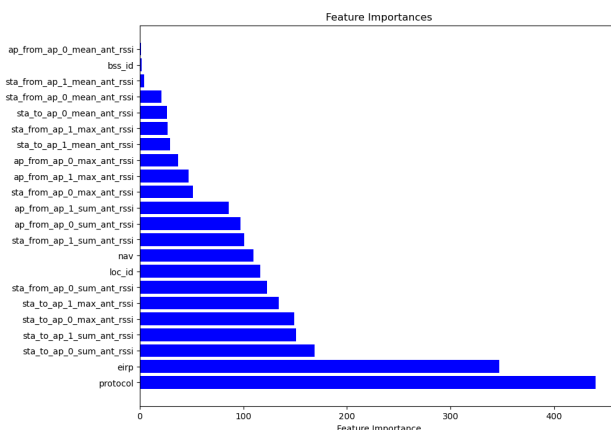


图 7.11 2AP xgboost 变量重要性排

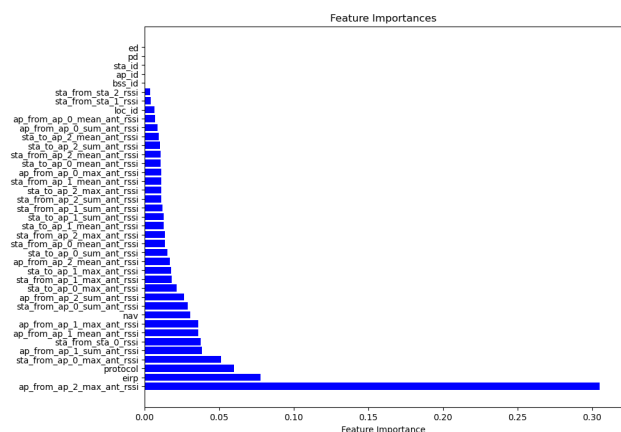


图 7.12 3AP 随机森林变量重要性排序

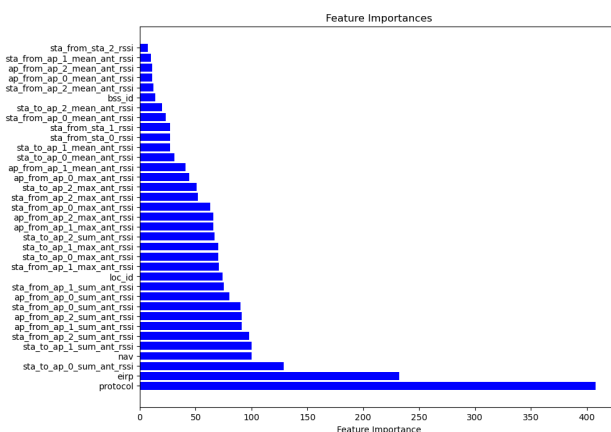


图 7.13 3AP xgboost 变量重要性排

7.1.2 验证结果分析

本节进一步对 AP 发送数据帧序列的总时长预测模型，在训练过程中的验证结果进行分析，共对比了 9 种模型：线性回归，岭回归，lasso 回归，Knn，决策树，随机森林，xgboost，svm 和 RegressionNet。

- 线性回归：是一种通过最小二乘法建立自变量（X）和因变量（Y）之间线性关系的统计方法。它尝试找到一条最佳的直线（ $y = wx + b$ ），使得所有观测值到这条直线的垂直距离（即残差）的平方和最小。线性回归可分为一元线性回归和多元线性回归，前者涉及一个自变量和一个因变量，后者涉及多个自变量和一个因变量。线性回归在趋势预测、经济学等领域有广泛应用。
- 岭回归：是一种专门用于处理共线性数据的回归分析方法。它通过向最小二乘法的损失函数中添加一个 L2 正则化项（即系数的平方和）来防止过拟合，并提高模型的泛化能力。岭回归牺牲了最小二乘法的无偏性，但换来了模型系数的稳定性和较小的方差。在处理病态数据和多重共线性数据时，岭回归的表现通常优于普通的最小二乘回归。

- **Lasso 回归**：是一种线性回归的变种，它通过向损失函数中添加 L1 正则化项（即系数的绝对值之和）来实现模型的稀疏化。Lasso 回归不仅能够像岭回归一样防止过拟合，而且能够自动进行特征选择，将那些对目标变量影响不大的特征的系数压缩到 0，从而简化模型并提高解释性。这使得 Lasso 回归在特征选择、变量筛选等方面具有独特的优势。
- **KNN**：是一种基于实例的学习方法，其基本思想是：对于给定的测试样本，找出与其距离最近的 K 个训练样本，然后根据这 K 个训练样本的类别来预测测试样本的类别（对于回归问题，则是计算这 K 个训练样本的目标值的平均值作为预测值）。KNN 方法简单直观，但计算量较大，特别是当训练样本数量较多时。此外，KNN 的预测结果受 K 值选择和距离度量方式的影响较大。
- **决策树**：是一种树形结构的分类与回归方法。它通过递归地将数据集划分为若干个纯度更高（对于分类问题）或更一致（对于回归问题）的子集来构建决策树。决策树的每个内部节点表示一个属性上的测试，每个分支代表一个测试结果的输出，每个叶节点代表一个类别（对于分类问题）或一个具体的数值（对于回归问题）。决策树易于理解和实现，但容易出现过拟合现象。
- **随机森林**：是一种集成学习方法，它通过构建多个决策树并将它们的预测结果进行平均或投票来得到最终的预测结果。随机森林在构建每棵决策树时都会随机选择部分样本和特征进行训练，这使得每棵决策树都具有一定的差异性。通过集成多棵决策树的预测结果，随机森林能够显著降低模型的方差并提高预测的准确性。随机森林在处理高维数据、缺失数据以及具有复杂相互作用的变量时表现出色。
- **XGBoost** 是一种高效的集成学习算法，属于梯度提升树（Gradient Boosting Decision Tree）的变种。它通过迭代地训练一系列的决策树模型并将它们组合成一个强大的模型来实现预测。XGBoost 在损失函数中加入了正则化项以控制模型的复杂度并防止过拟合。同时，XGBoost 还采用了多种优化技术来提高训练速度和预测精度。XGBoost 在机器学习竞赛和工业应用中均取得了优异的性能表现。
- **SVM** 是一种基于监督学习的分类与回归方法。对于分类问题，SVM 通过寻找一个超平面来最大化不同类别之间的间隔；对于回归问题，SVM 则通过寻找一个超平面来使得所有训练样本到该平面的距离（即误差）尽可能小且均匀分布。SVM 在处理高维数据和小样本数据方面具有独特的优势，但其计算复杂度较高且对参数的选择较为敏感。
- **RegressionNet** 并不是一个标准的机器学习算法，但可以从字面意思上理解为一种用于回归任务的神经网络模型。神经网络（特别是深度神经网络）通过构建多层神经元之间的连接来学习输入数据与输出数据之间的复杂关系。RegressionNet（或类似的回归神经网络）可以用于处理各种回归问题，如预测股票价格、房价等连续值输出的问题。然而，需要注意的是，由于神经网络的高度复杂性和非线性特性，其训练过程可能需要

表 7.3 2AP 结构下验证结果

	线性回归	岭回归	Lasso 回归	Knn	决策树	随机森林	xgboost	svm	RegressionNet
MSE	29.515	28.265	26.929	17.209	28.246	10.412	10.905	23.054	34.824
MAE	4.467	4.372	4.272	2.683	2.369	1.836	1.756	3.231	4.463
MAPE	0.133	0.129	0.124	0.086	0.081	0.065	0.064	0.100	0.129

表 7.4 3AP 结构下验证结果

	线性回归	岭回归	Lasso 回归	Knn	决策树	随机森林	xgboost	svm	RegressionNet
MSE	72.412	72.412	72.412	62.713	75.505	43.869	46.096	78.333	67.813
MAE	6.705	6.705	6.705	5.758	5.711	4.812	5.054	6.522	6.542
MAPE	0.296	0.296	0.296	0.265	0.260	0.209	0.211	0.308	0.291

大量的数据和计算资源，并且容易出现过拟合现象。在实际应用中，需要根据具体问题和数据集的特点来选择合适的模型架构和训练策略。

表7.60和7.61分别为 9 种模型在 2AP 结构和 3AP 结构下的 MSE，MAE 以及 MAPE 指标的对比：

从表7.60可以发现，随机森林和 XGBoost 的测试结果都较为优秀，随机森林的 MSE 最低为 10.421，XGBoost 的 MAE 和 MAPE 都最低，分别为 1.756 和 0.064。因此，在测试集的预测种，我们选用 XGBoost 来对 2AP 结构下的 seq_time 进行预测。

此外，我们还对验证结果的 CDF 进行了可视化，CDF 图如下所示：

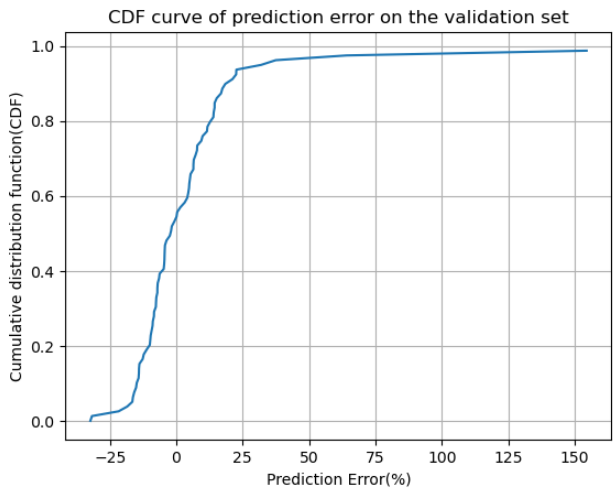


图 7.14 2ap 线性回归 CDF 曲线

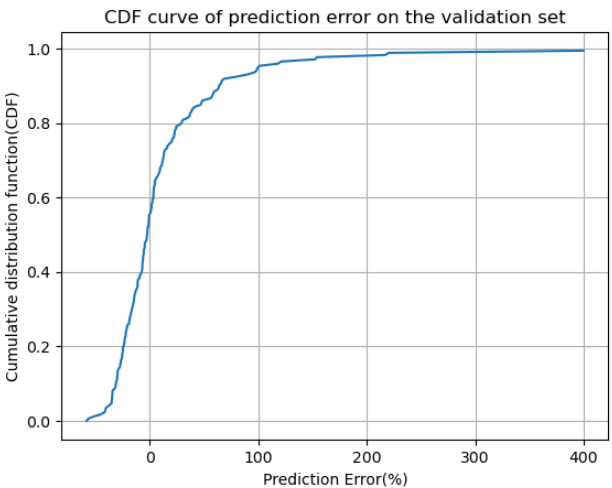


图 7.15 3ap 线性回归 CDF 曲线

CDF 曲线（Cumulative Distribution Function Curve），即累积分布函数曲线，是统计学和概率论中用于描述随机变量概率分布的一种图形表示方法，描述了一个随机变量 X 小

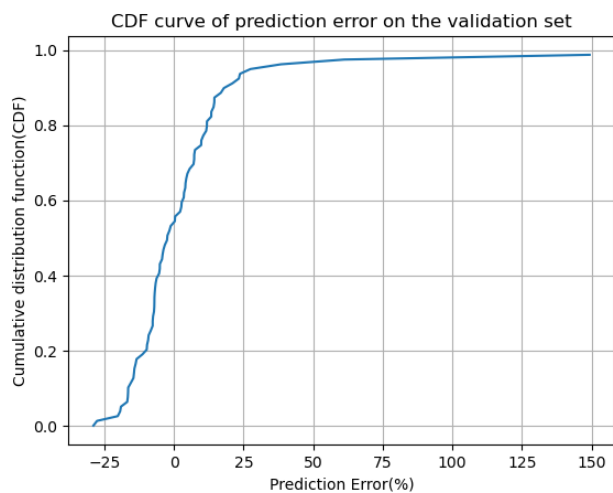


图 7.16 2ap 岭回归 CDF 曲线

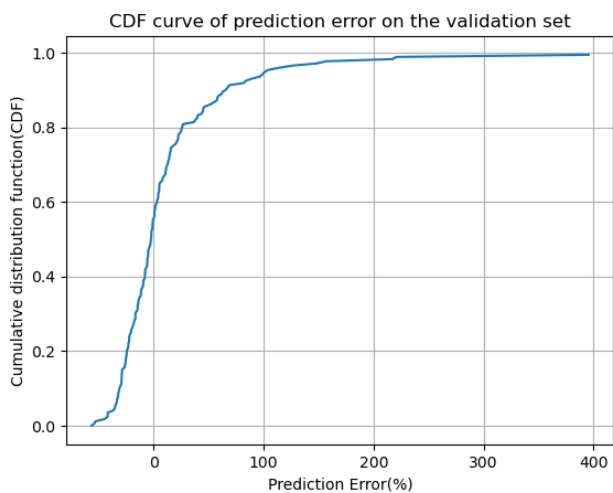


图 7.17 3ap 岭回归 CDF 曲线

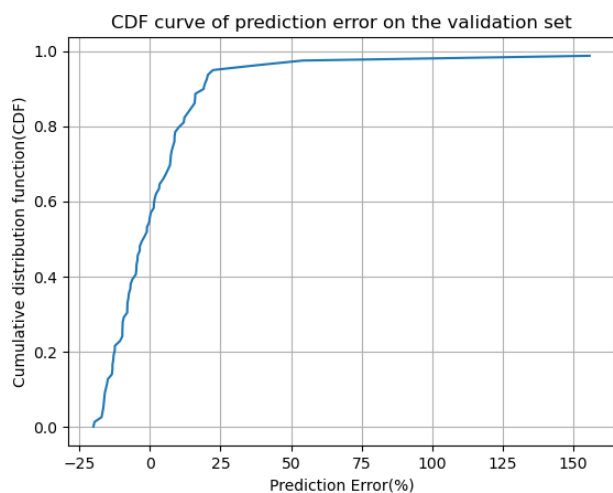


图 7.18 2aplasso 回归 CDF 曲线

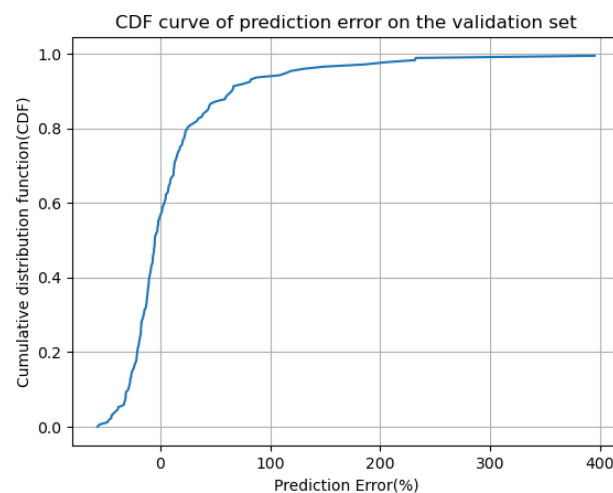


图 7.19 3aplasso 回归 CDF 曲线

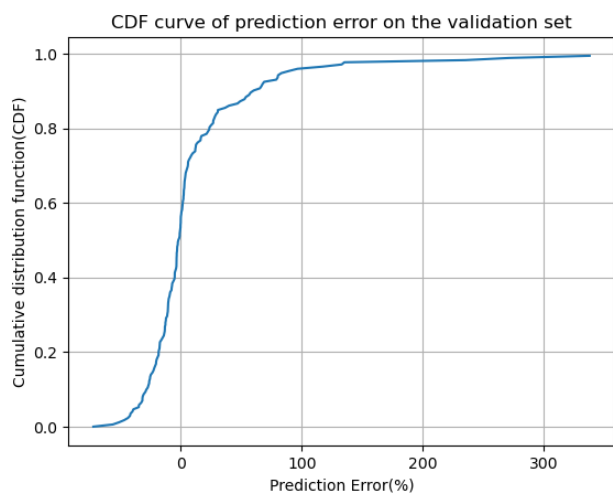


图 7.20 2qpKNN-CDF 曲线

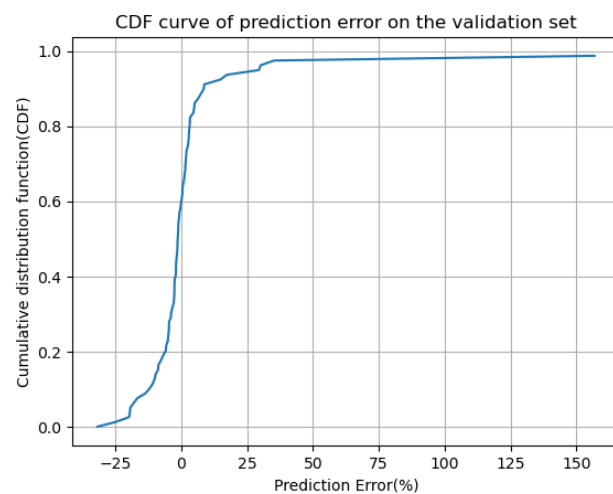


图 7.21 3qpKNN-CDF 曲线

于或等于某一特定值 x 的概率。绘制测试集中吞吐量预测误差 $error$ 的 CDF 曲线，取 $error$ 小于或等于某一特定值 $ERROR$ 的概率为 90% 时对应的特定值 $ERROR$ ，则模型精度为 1 与 $ERROR$ 的差值。请分别提供预测每 AP 吞吐的精度，和预测系统吞吐（所有 AP 吞吐之和）的精度。

在 CDF 曲线中，曲线越陡峭表示数据集中存在着更多较大或较小的数值。在机器学习和统计建模中，CDF 曲线常用于评估模型的预测性能。通过比较模型预测结果的 CDF 曲线与实际观测数据的 CDF 曲线，可以评估模型的准确性和可靠性。

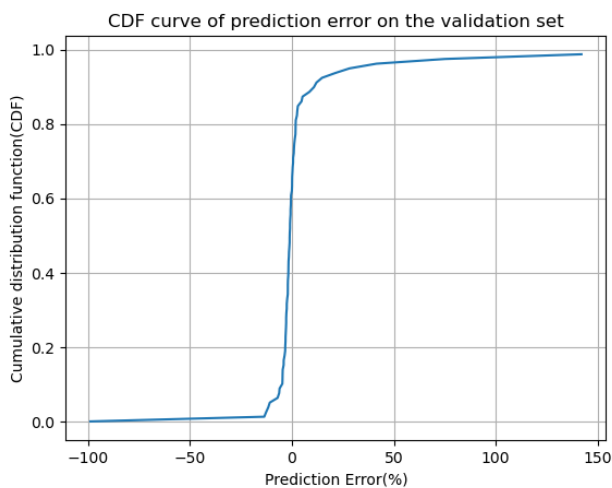


图 7.22 2ap 决策树 CDF 曲线

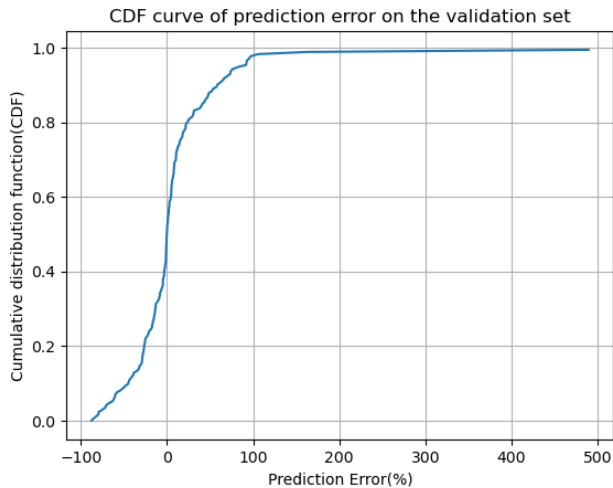


图 7.23 3ap 决策树 CDF 曲线

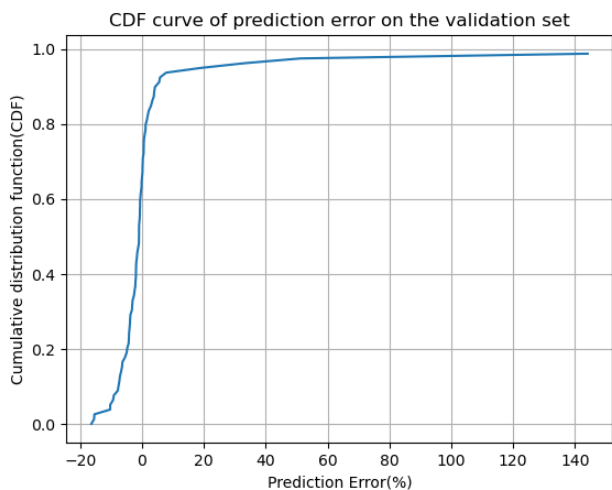


图 7.24 2ap 随机森林 CDF 曲线

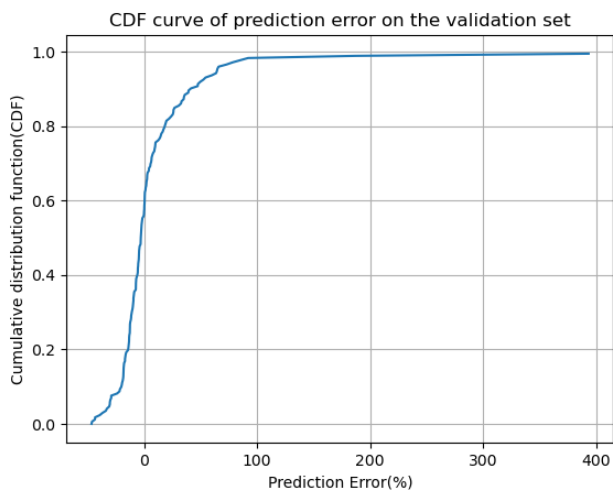


图 7.25 3ap 随机森林 CDF 曲线

7.1.3 测试结果分析

测试结果分别保存在附件：test_set_1_2ap.csv 以及 test_set_1_3ap.csv 当中。

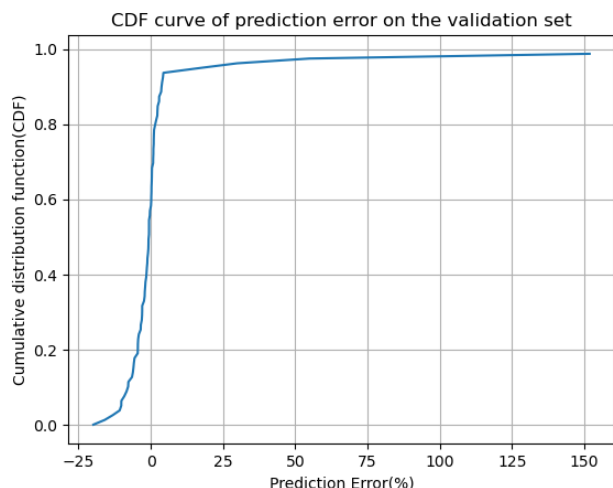


图 7.26 2apXGBoost CDF 曲线

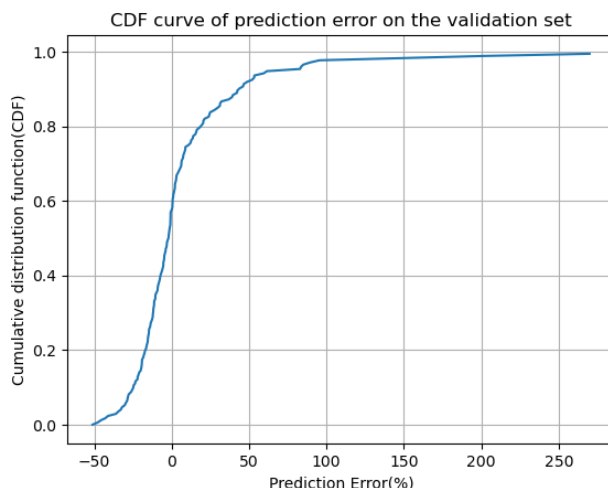


图 7.27 3apXGBoost CDF 曲线

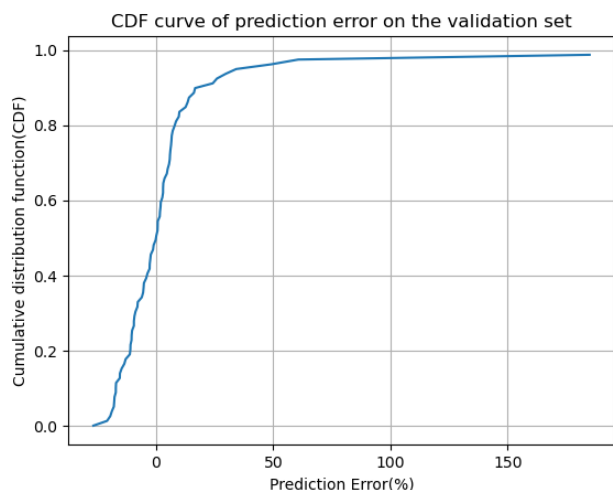


图 7.28 神经网络 CDF 曲线

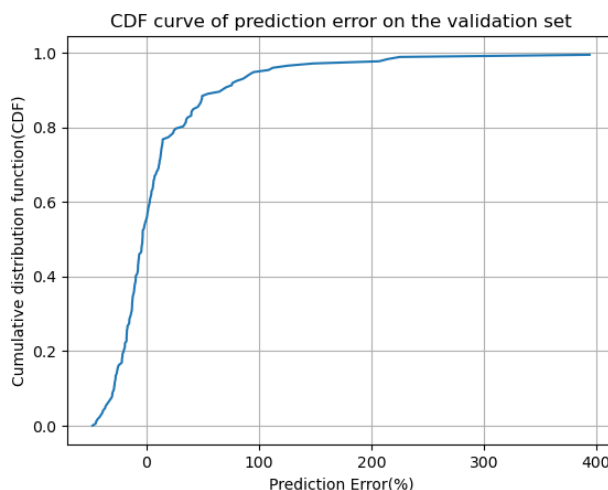


图 7.29 神经网络 CDF 曲线

7.2 问题二结果分析

7.2.1 变量间的相关性分析

针对问题二, 通过对图7.30 pairplot 的分析, 我们可以看到不同变量之间的关系。整体上, 我们计算出的 $\sin r_d B$ 与其他变量之间有显著的线性相关性, 数据点在图中显得较为集中。此外, $eirp$ 数据中存在较多重复值, 这可能是由于该变量经过离散化处理。 nav 变量的数据分布显示出明显的集中现象, 表明某些特定值出现的频率较高。同时, $bss_i d$ 和 $ap_i d$ 似乎是离散型的分类变量, 它们在 pairplot 中的分布也显示出这种特性。热力图则揭示了变量之间的相关性。

从图7.31中, 我们可以更直观地看到各个特征之间的相关性矩阵。 $\sin r_d B$ 与 mcs 之间的相关系数为 0.58, 表明它们之间存在很强的正相关, 而 $eirp$ 与 mcs 的相关系数为 0.50,

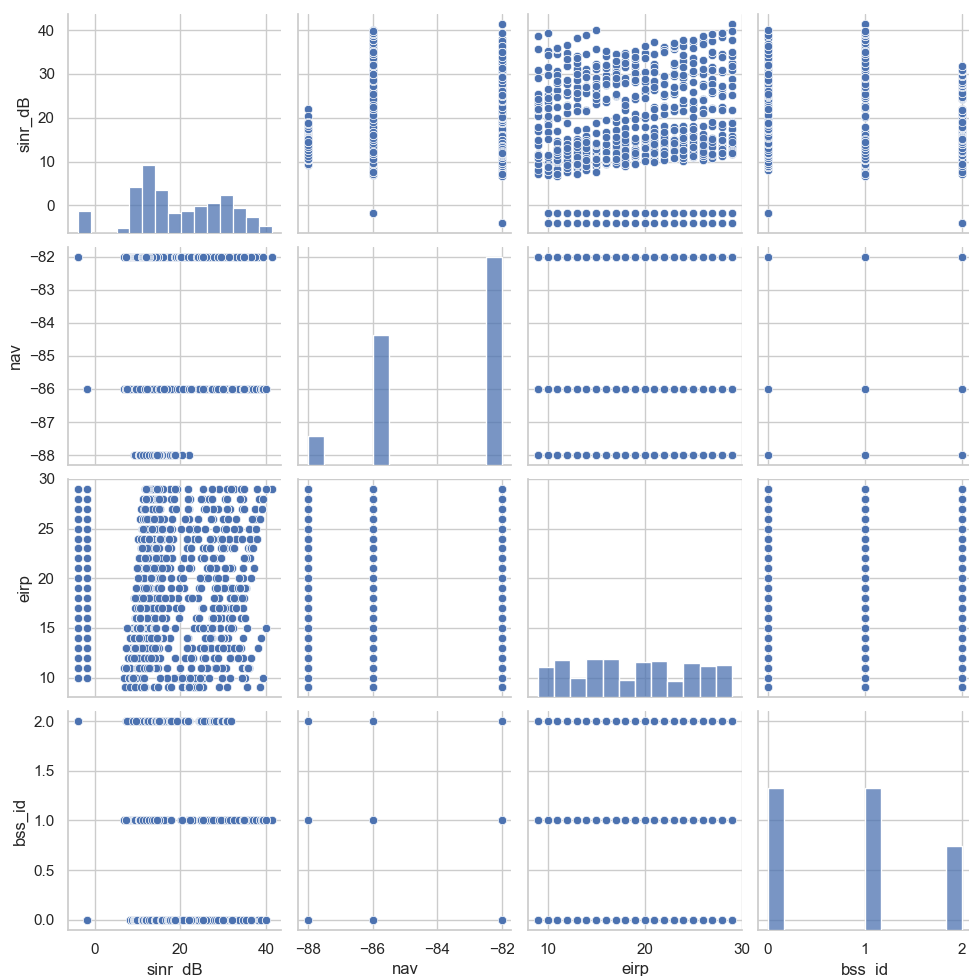


图 7.30 pairplot 图

也显示了明显的正相关性。 ap_id 与 bss_id 之间存在很强的负相关，相关系数为-0.82，这可能是由于它们之间存在某种逻辑上的互斥关系。相比之下， loc_id 、 $protocol$ 与其他变量的相关性较低，说明它们对目标变量的影响较小。

通过以上分析，我们可以更好地理解各个特征变量与目标变量之间的关系。从图7.32可以看出，特征 mcs 与 nss 的相关性最高，相关系数约为 0.25，说明 mcs 对 nss 有一定的正向影响。其次， $eirp$ 和 $sinr_dB$ 的相关性也较高，相关系数分别为 0.15 和 0.10，表示它们与 nss 之间存在一定的正相关性。其他特征如 ap_id 、 nav 等的相关性则较弱，甚至呈现出负相关。

从柱状图7.33可以看到， $sinr_dB$ 对 mcs 的影响最大，相关系数为 0.58，说明信号与干扰噪声比是影响 mcs 的关键因素。 $eirp$ 紧随其后，与 mcs 的相关系数为 0.50，表示发射功率对 mcs 也有显著影响。 nss 与 mcs 之间的相关性为 0.25，虽然比前两者低，但仍然是一个重要的特征。而 ap_id 、 $protocol$ 等变量对 mcs 的相关性则很小，甚至呈现负相关，表明它们对 mcs 的影响较为有限。

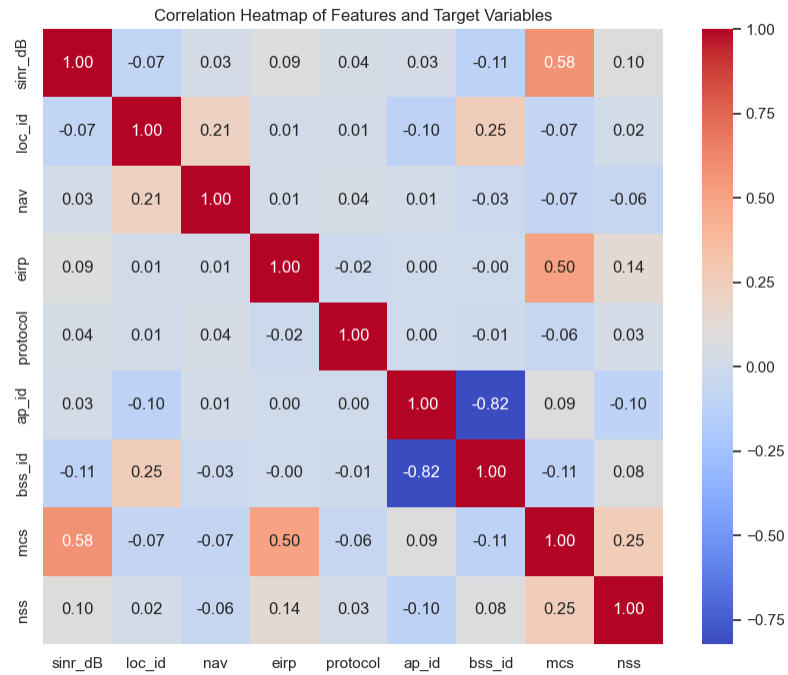


图 7.31 热力图

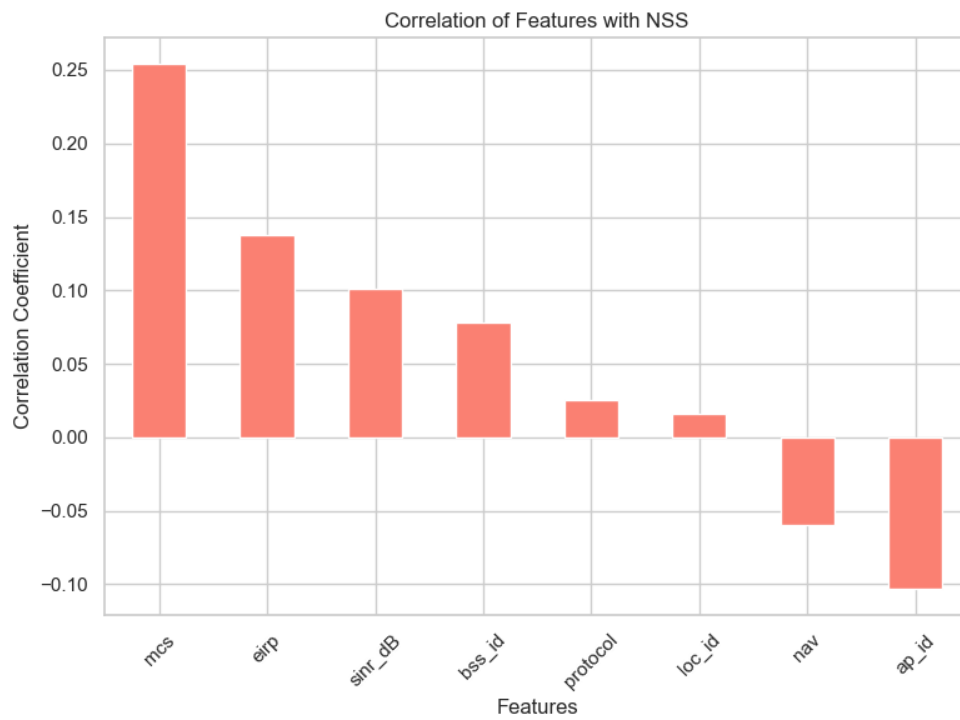


图 7.32 NSS 相关性分

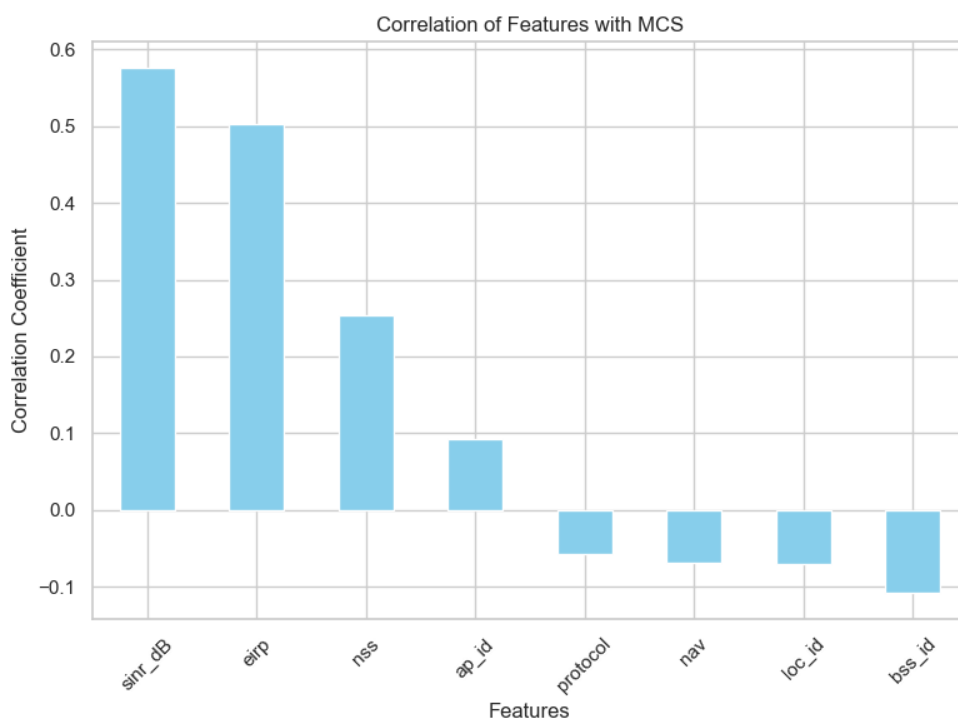


图 7.33 MCS 相关性分析

总之，从上面的分析可以看出，通过我们所提出的 *sinrB* 算法计算得出值，与 *MCS* 和 *NNS* 具有极强的相关性，且相比较于其他的变量他的相关性远远高于其他变量。

7.2.2 训练过程分析

对于 *MCS* 和 *NNS* 两个目标，我们分别建立了两个模型，一个叫 *MCS* 分类模型，另一个叫 *NNS* 分类模型。由于我们采用了八种不同的模型分别进行了训练和评估，下面我们将展示，在八个模型上训练的基本过程。

1. Logistic 回归:

在 *MCS* 与 *NSS* 模型中，Logistic 回归 适用于线性可分问题。尽管模型简单易于解释，但由于 *MCS* 与 *NSS* 之间的关系可能存在非线性，Logistic 回归 在处理这些问题时的表现相对较差。从图7.34, 图7.35中可以看到，对于 *MCS* 和 *NSS*，Logistic 回归 模型的表现较为稳定，但精度有限。

2. Softmax 回归:

适合多分类问题，尤其在多个类别时表现不错。然而，从图7.36,7.37 *MCS* 与 *NSS* 模型的结果来看，虽然 Softmax 回归 在某些分类任务中表现出色，但随着数据复杂度的增加，训练时间明显增多，特别是 *NSS* 模型中的精度有所下降。

3. 随机森林:

通过集成学习增强模型的稳定性。从图7.38, 图7.39 *MCS* 与 *NSS* 模型的结果中可以看

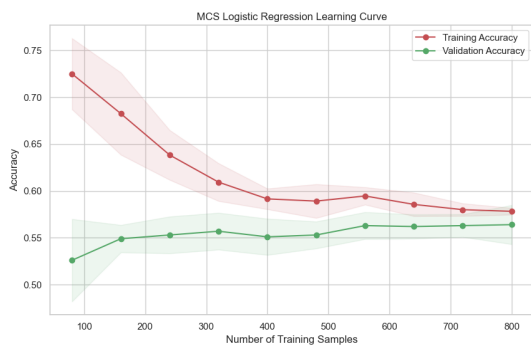


图 7.34 MCS 逻辑回归学习曲线

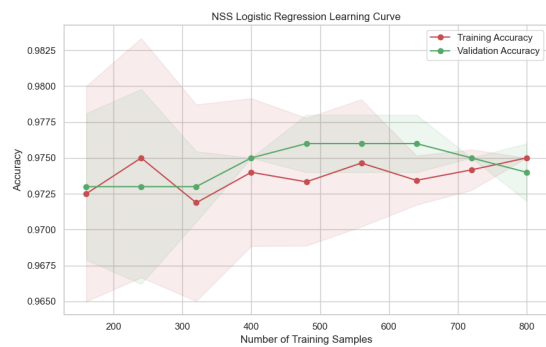


图 7.35 NSS 逻辑回归学习曲线



图 7.36 MCS SoftMax 回归学习曲线

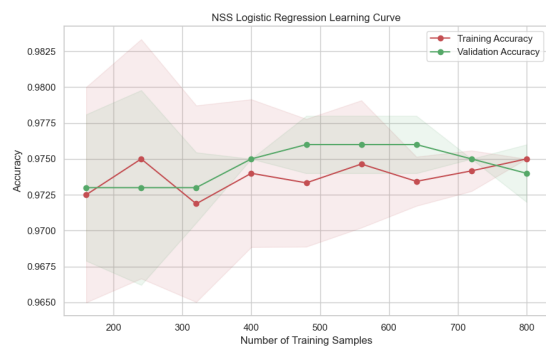


图 7.37 NSS SoftMax 回归学习曲线

出，随机森林 的表现较为优越，特别是在处理高维复杂数据时，模型的泛化能力显著提高。然而，由于树的数量较多，训练速度和预测速度在 *NSS* 模型上相对较慢。

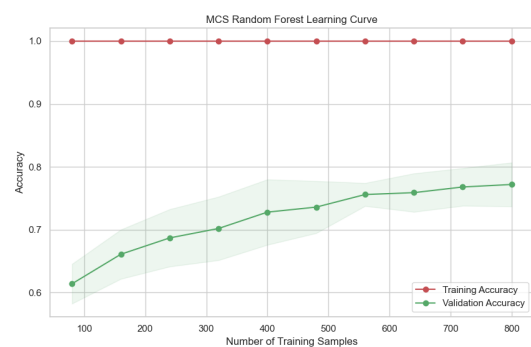


图 7.38 MCS 随机森林学习曲线

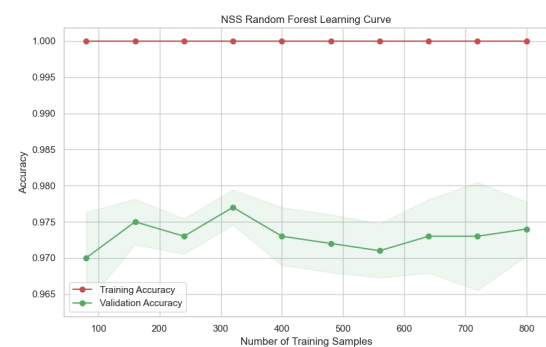


图 7.39 NSS 随机森林学习曲线

4. 支持向量机 (SVM):

图7.40,7.41, 显示，对于高维数据的表现良好，尤其是当 *MCS* 和 *NSS* 之间的关系不是线性可分时，*SVM* 通过核函数提升了模型的分类能力。图中显示，*SVM* 在处理 *MCS* 模型时表现优秀，但在 *NSS* 模型中的计算复杂度较高，处理大规模数据时效率有所下

降。



图 7.40 MCS SVM 学习曲线



图 7.41 NSS SVM 学习曲线

5. K-近邻分类器 (KNN):

图7.42,7.43 KNN 对小样本数据集表现良好，但在处理 MCS 与 NSS 模型中的大规模数据时，计算复杂度迅速增加，尤其在 NSS 模型中，计算开销显著，导致表现不如其他模型。

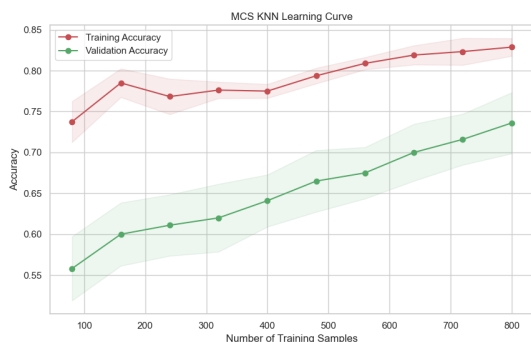


图 7.42 MCS KNN 学习曲线

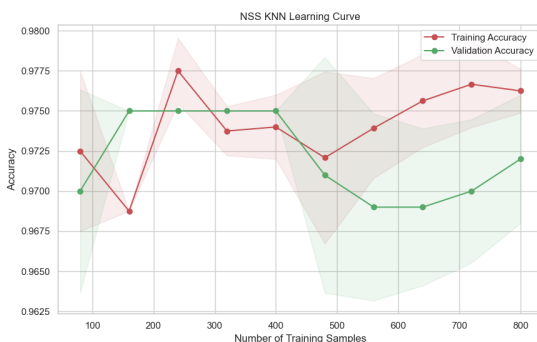


图 7.43 NSS KNN 学习曲线

6. 神经网络 (Neural Networks):

图7.44,7.45 中显示，通过多层神经元处理复杂的非线性关系，神经网络在 MCS 与 NSS 模型中的表现相当优异，能够捕捉复杂特征间的关系。然而，神经网络容易过拟合，并且需要较长的训练时间。特别是在 NSS 模型中，过拟合问题更加明显。

7. XGBoost:

从图7.46,7.47中可以看出，XGBoost 在 MCS 与 NSS 模型中的表现提升了决策树的性能，尤其在中等规模的数据集上表现非常出色。通过调参，模型在精度上取得了较高的水平，但调参过程较为复杂，尤其在 NSS 模型中对参数敏感。

8. LightGBM:

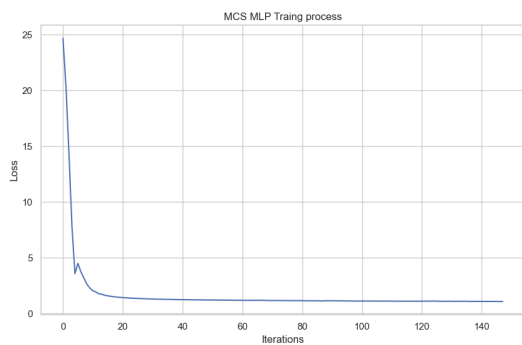


图 7.44 MCS 神经网络学习曲线

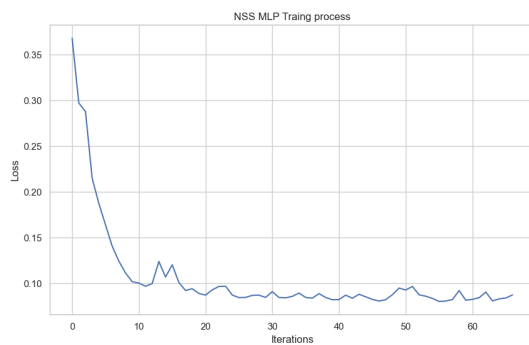


图 7.45 NSS 神经网络学习曲线

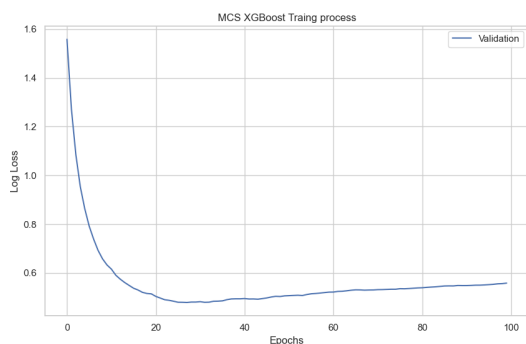


图 7.46 MCS XGBoost 学习曲线

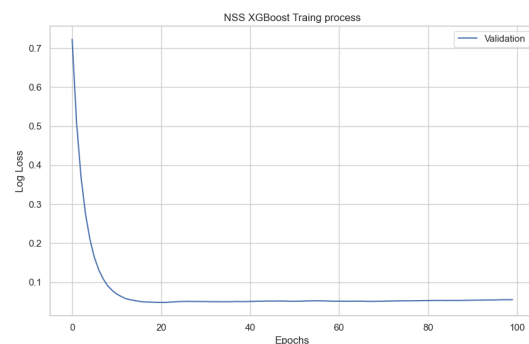


图 7.47 NSS XGBoost 学习曲线

LightGBM 在处理大规模数据集时表现出色，从图7.48,7.49中 *MCS* 与 *NSS* 模型的结果来看，它的训练速度快，内存占用低。尤其在 *NSS* 模型中，对于高维稀疏数据的处理效果非常好。此外，*LightGBM* 在类别不平衡的数据集上表现优异。

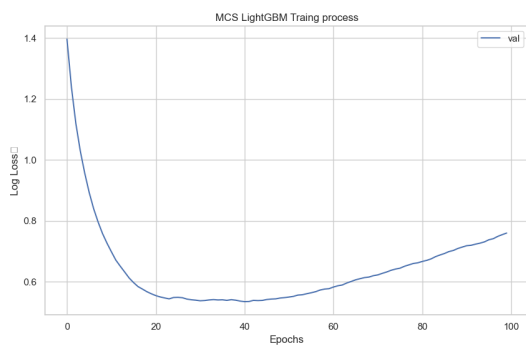


图 7.48 MCS LightGBM 学习曲线

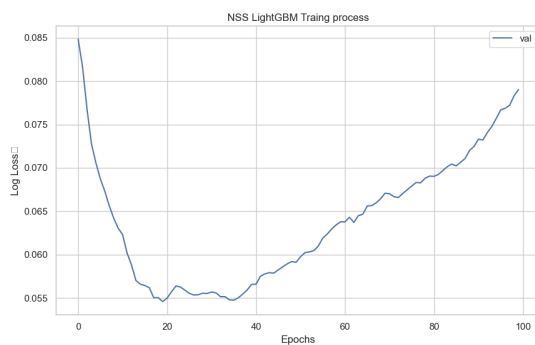


图 7.49 NSS LightGBM 学习曲线

从上面的实验训练过程分析来看，对于少量数据而言，神经网络并没有特别高的优势。反而是随机森林算法等基于传统的机器学习方法在其中表现出了优异的结果。相比较于神经网络，反而少了过拟合的情况。

7.2.3 验证结果分析

在本次实验中，我们针对两个不同的标签预测目标——MCS (Monte Carlo Sampling) 和 NSS (Neural Sampling Strategy)，分别构建并验证了八种不同的机器学习模型。这些模型包括 Logistic Regression、Softmax Regression、Random Forest、SVM (Support Vector Machine)、KNN (K-Nearest Neighbors)、MLP (Multi-Layer Perceptron)、XGBoost 以及 LightGBM。为了全面评估这些模型的性能，我们采用了四项关键评价指标：准确率 (Accuracy)、F1 值 (F1 Score)、精度 (Precision) 和召回率 (Recall)。本文将分别从 MCS 和 NSS 两个部分，对这八个模型的实验结果进行详细的对比分析，并探讨各模型在不同标签预测目标下的表现差异及其可能原因。

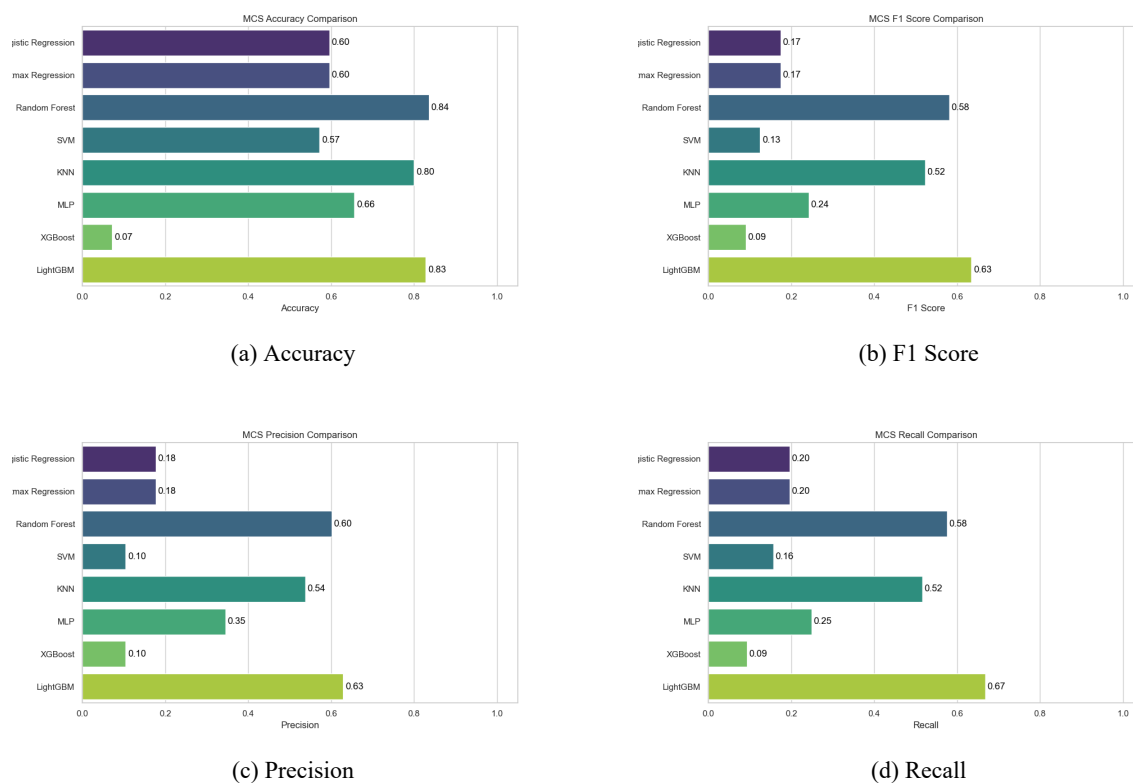


图 7.50 MCS 标签预测目标下的模型表现

1.MCS 标签预测目标下的模型表现

在 MCS 标签预测目标下，八个模型的表现存在显著差异。以下将从准确率、F1 值、精度和召回率四个维度，对各模型的性能进行详细分析。

准确率分析:

在 MCS 目标下，所有模型的准确率从 0.0720 (XGBoost) 到 0.8360 (Random Forest) 不等。具体而言，XGBoost 模型的准确率极低，仅为 0.0720，表明其在此任务中几乎无法正确分类。而 Random Forest 则表现最佳，准确率达到 0.8360，显著高于其他模型。KNN

和 LightGBM 的准确率分别为 0.8000 和 0.8280，表现亦较为优秀。相比之下，Logistic Regression、Softmax Regression、SVM 和 MLP 的准确率分别为 0.5960、0.5960、0.5720 和 0.6560，表现较为一般。

F1 值分析

F1 值综合考虑了模型的精度和召回率，是衡量模型在不平衡数据集上表现的重要指标。在 MCS 下，Random Forest 的 F1 值最高，达到 0.5806，显示出其在平衡精度与召回率方面的优势。KNN 和 LightGBM 的 F1 值分别为 0.5224 和 0.6347，同样表现优异。Logistic Regression、Softmax Regression 和 SVM 的 F1 值分别为 0.1748、0.1748 和 0.1252，明显低于其他模型，表明这些模型在处理 MCS 目标时存在较大的分类困难。MLP 的 F1 值为 0.2418，虽然低于 Random Forest 和 KNN，但相比于 Logistic Regression 等模型有所提升。

精度分析

精度衡量的是模型预测为正类的样本中实际为正类的比例。在 MCS 下，Random Forest 的精度最高，达到 0.6011，显示出其在减少假阳性方面的有效性。LightGBM 的精度为 0.6291，略高于 Random Forest，表明其在识别正类样本上具有较高的准确性。KNN 的精度为 0.5375，表现中等。相比之下，Logistic Regression、Softmax Regression 和 SVM 的精度分别为 0.1774、0.1774 和 0.1045，极低的精度表明这些模型在 MCS 任务中容易产生大量假阳性，限制了其实际应用价值。MLP 的精度为 0.3453，较其他简单模型有所提升。

召回率分析

召回率反映了模型能够识别出的实际正类样本的比例。在 MCS 下，Random Forest 的召回率为 0.5754，表现优异，显示出其在识别正类样本方面的高效性。LightGBM 的召回率更高，达到 0.6676，表明其在 MCS 任务中能够捕捉到更多的正类样本。KNN 的召回率为 0.5159，表现中等。Logistic Regression、Softmax Regression 和 SVM 的召回率分别为 0.1965、0.1965 和 0.1569，较低的召回率表明这些模型在 MCS 任务中存在较多的假阴性，无法有效捕捉所有的正类样本。MLP 的召回率为 0.2489，虽有所提升，但仍有较大的改进空间。

综合分析

综合上述四项指标，Random Forest 和 LightGBM 在 MCS 标签预测目标下表现最为优异。Random Forest 凭借其高准确率、F1 值、精度和召回率，显示出在该任务中的综合优势。LightGBM 尽管在准确率和召回率上表现突出，但其 F1 值略低于 Random Forest，可能是由于在精度和召回率之间的权衡导致的。KNN 和 MLP 在中等水平上表现，而 Logistic Regression、Softmax Regression 和 SVM 则在各项指标上均表现不佳，显示出在 MCS 任务中的局限性。XGBoost 由于其极低的准确率和 F1 值，基本无法在 MCS 任务中发挥作用，可能是由于该模型在当前数据集上的过拟合或欠拟合导致的。

2. NSS 标签预测目标下的模型表现

在 NSS 标签预测目标下，八个模型的表现总体优于 MCS 目标下的表现，尤其是在准

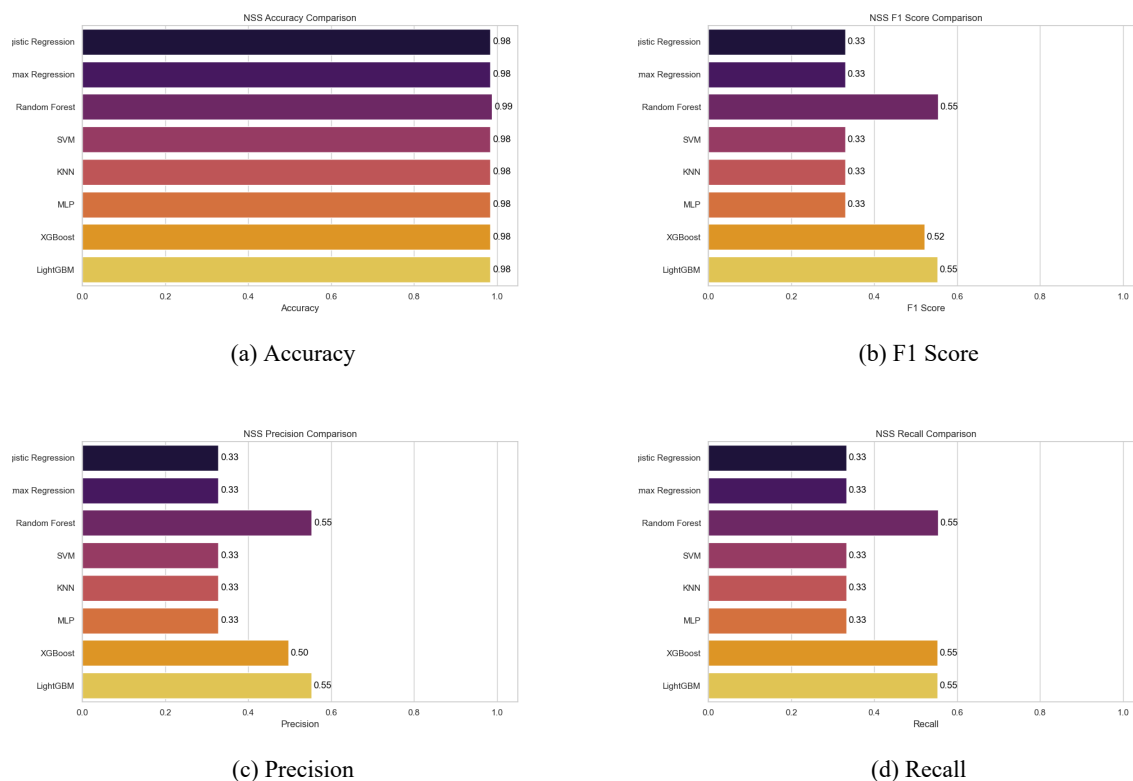


图 7.51 NSS 标签预测目标下的模型表现

准确率方面。然而，不同模型在其他评价指标上的表现仍存在一定差异。以下将从准确率、F1 值、精度和召回率四个维度，对各模型的性能进行详细分析。

准确率分析

在 NSS 目标下，所有模型的准确率均达到了 0.9840，除了 Random Forest（0.9880）和 LightGBM（0.9840），基本上所有模型的准确率都接近于 1。这表明在 NSS 任务中，数据分布可能更为平衡，或模型更容易捕捉到数据中的模式，从而实现高准确率。

F1 值分析

F1 值在 NSS 下的表现有所不同。Random Forest 的 F1 值为 0.5535，XGBoost 为 0.5211，而其他模型（Logistic Regression、Softmax Regression、SVM、KNN、MLP）均为 0.3306。LightGBM 的 F1 值为 0.5528，与 Random Forest 相近。这表明 Random Forest 和 LightGBM 在 NSS 任务中能够较好地平衡精度与召回率，而其他模型在这一任务中表现较为一致，且 F1 值较低，显示出在处理 NSS 标签预测时存在一定的局限性。

精度分析

在 NSS 任务中，Random Forest 和 LightGBM 的精度分别为 0.5529 和 0.5528，表现出较好的分类能力。XGBoost 的精度为 0.4973，略低于 Random Forest 和 LightGBM，但仍高于其他模型（Logistic Regression、Softmax Regression、SVM、KNN、MLP 均为 0.3280）。这

表明 Random Forest 和 LightGBM 在 NSS 任务中能够有效减少假阳性，提高分类的准确性。

召回率分析

在召回率方面，Random Forest 和 XGBoost 表现较为突出，分别为 0.5542 和 0.5528。LightGBM 的召回率也达到 0.5528，显示出其在识别正类样本方面的良好能力。其他模型（Logistic Regression、Softmax Regression、SVM、KNN、MLP）的召回率均为 0.3333，显示出在 NSS 任务中这些模型在捕捉正类样本方面存在一定的不足。

综合分析

在 NSS 标签预测目标下，Random Forest 和 LightGBM 依然表现出色，尤其是在 F1 值、精度和召回率上，与 MCS 目标下的表现相似，显示出其不同任务中的稳定性和适应性。XGBoost 在 NSS 任务中的表现有所提升，尤其是在 F1 值和召回率上，但其精度略低于 Random Forest 和 LightGBM。其他模型在 NSS 任务中的表现较为一致，F1 值、精度和召回率均为 0.3306 和 0.3280，显示出在该任务中这些模型的性能受限，可能需要进一步的优化和调整。

值得注意的是，尽管所有模型在 NSS 任务中的准确率均接近于 1，但只有 Random Forest 和 LightGBM 在 F1 值、精度和召回率上表现出较高的数值。这可能意味着，在 NSS 任务中，虽然模型能够正确分类绝大多数样本，但在处理少数类样本时，Random Forest 和 LightGBM 仍然能够保持较高的识别能力，而其他模型则可能在少数类样本的识别上存在较大困难。

在 MCS 标签预测目标下，Random Forest 和 LightGBM 表现出较高的准确率、F1 值、精度和召回率，显示出其在处理此类任务时的优势。相比之下，NSS 任务下几乎所有模型的准确率均达到 0.9840，表明 NSS 任务可能具有更为平衡或易于建模的数据特征。然而，只有 Random Forest 和 LightGBM 在 F1 值、精度和召回率上表现出较高的数值，其他模型尽管准确率高，但在处理少数类样本时表现欠佳。

这种差异可能源于 MCS 和 NSS 标签预测目标在数据分布、样本特征等方面的不同。MCS 任务下，数据可能存在较大的不平衡或复杂的特征关系，导致简单模型（如 Logistic Regression、Softmax Regression 和 SVM）难以有效捕捉，而复杂模型（如 Random Forest 和 LightGBM）则能够通过集成学习和树模型的优势，更好地处理数据的复杂性和不平衡性。NSS 任务下，尽管准确率普遍较高，但在处理细粒度分类时，只有部分模型能够保持较高的 F1 值和召回率，显示出其在细节处理上的差异。

- **Random Forest 和 LightGBM 的优势突出：**在 MCS 和 NSS 两种任务中，Random Forest 和 LightGBM 均表现出较高的准确率、F1 值、精度和召回率，显示出其在处理不同类型的标签预测任务中的适应性和稳定性。这可能归因于它们强大的集成学习能力和处理复杂数据特征的优势。

复制

- **复杂模型优于简单模型：** Logistic Regression、Softmax Regression 和 SVM 等简单模型在 MCS 和 NSS 任务中的表现相对较差，尤其是在 F1 值和召回率方面。这表明在面对复杂或不平衡的数据时，这些模型的表达能力和分类性能存在明显不足。
- **XGBoost 在 MCS 任务中表现不佳，但在 NSS 任务中有所提升：** XGBoost 在 MCS 任务中的准确率和 F1 值极低，几乎无法有效分类，但在 NSS 任务中，其性能有了显著提升。这可能表明 XGBoost 在处理某些类型的数据分布时更为有效，但在面对高度不平衡或复杂的 MCS 任务时，可能需要进一步的参数调优或数据预处理。
- **KNN 和 MLP 的表现中等：** KNN 在 MCS 任务中表现较为中等，具有较高的准确率和 F1 值，而在 NSS 任务中，其性能与其他简单模型相似。MLP 在 MCS 任务中表现稍好，但在 NSS 任务中仍未能显著提升。这表明，尽管这些模型具有一定的非线性表达能力，但在当前任务和数据集下，其优势尚未充分发挥。

基于以上结论，未来的研究可以从以下几个方面进行改进和拓展：

1. **模型优化与调参：** 对表现不佳的模型（如 Logistic Regression、SVM、XGBoost）进行更深入的调参和优化，探索其在不同超参数设置下的性能变化，以期提升其在特定任务中的表现。
2. **数据预处理与特征工程：** 进一步优化数据预处理流程，如处理数据不平衡问题、特征选择与提取等，以提升模型的输入质量，从而增强模型的分类能力。
3. **集成学习与模型融合：** 探索将多个模型进行集成或融合的方法，如 Bagging、Boosting 或 Stacking，以充分利用各模型的优势，提升整体分类性能。
4. **深度学习模型的应用：** 除了 MLP 之外，尝试应用更复杂的深度学习模型，如卷积神经网络（CNN）或递归神经网络（RNN），以提升模型在处理复杂数据特征时的表现。
5. **跨任务的模型适应性研究：** 进一步研究模型在不同标签预测任务中的适应性和泛化能力，探索其在不同数据分布和特征空间下的性能变化规律，以指导模型选择和应用。

综上所述，本次实验通过对 MCS 和 NSS 两种不同标签预测目标下八个模型的性能评估，揭示了各模型在不同任务中的优劣势，为后续的模型选择与优化提供了重要的参考依据。未来的研究将在现有基础上，进一步深化模型优化与数据处理策略，以提升整体分类性能和应用价值。

7.2.4 测试结果分析

1. 2 AP 测试结果分析：

1. MCS 预测分布（MCS Prediction Distribution）：

左上图显示了 2 AP 测试中，MCS 预测值的分布。MCS=11 的频率最高，超过了 35 次，MCS=4 的频率紧随其后，约为 20 次。其他 MCS 值的频率相对较低。这个分布表明在 2

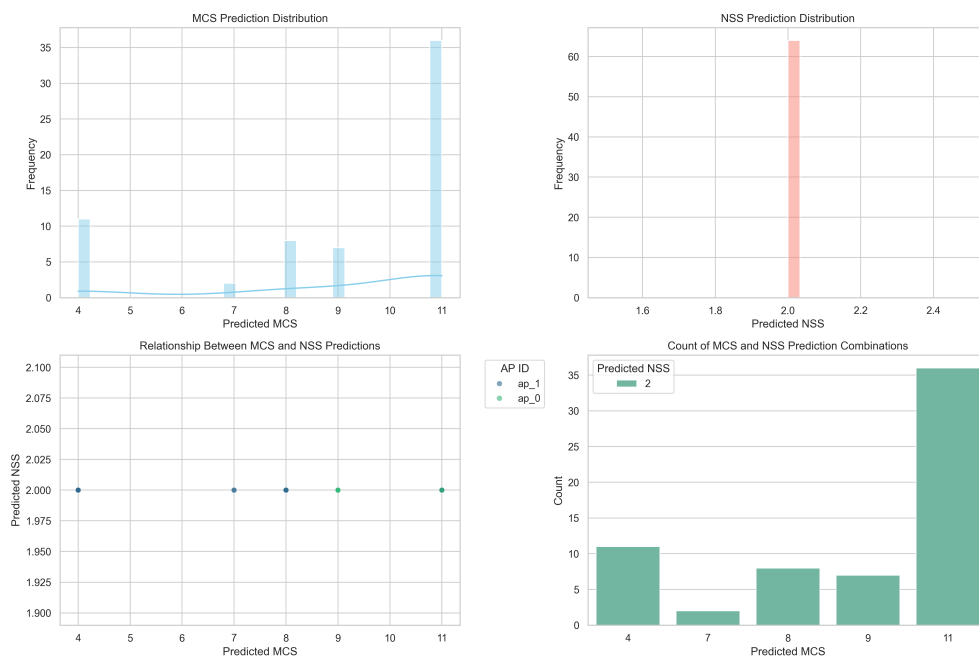


图 7.52 2AP 下测试结果分析

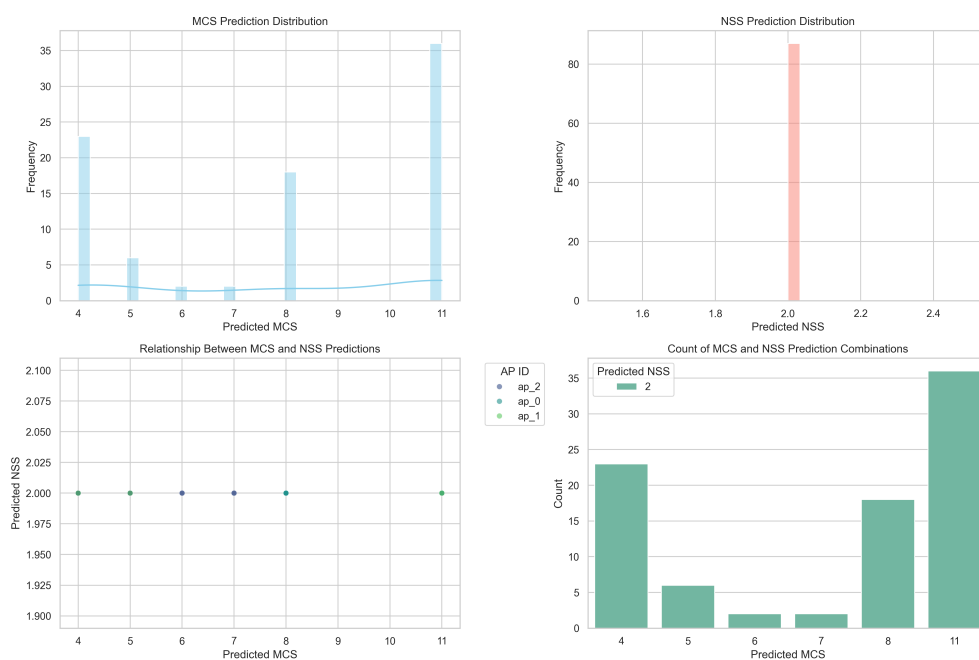


图 7.53 3AP 下测试结果分析

AP 的网络环境下，较高的 MCS 值（特别是 11）占据主导，表示该测试网络的性能较好，能维持高效的编码模式。

2. NSS 预测分布 (NSS Prediction Distribution):

右上图显示在 2 AP 测试中，NSS 预测值完全集中在 2，没有其他 NSS 值的出现。这表明在 2 AP 网络中，系统始终采用两个空间流来传输数据，可能是由于硬件限制或网络

配置的缘故。

3. MCS 与 NSS 预测关系 (Relationship Between MCS and NSS Predictions):

左下图展示了 2 AP 测试中, MCS 和 NSS 之间的关系。所有的 NSS 值均为 2, MCS 值在 4 到 11 之间变化。不同 AP 的数据点之间, NSS 并未发生变化, 始终保持为 2, 说明在该网络下, NSS 值固定在 2, 无论 MCS 值如何变化。

4. MCS 与 NSS 组合的计数 (Count of MCS and NSS Prediction Combinations):

右下图中, MCS=11 与 NSS=2 的组合出现最频繁, 次数接近 40, MCS=4 与 NSS=2 的组合约为 20 次, 其他 MCS 值与 NSS=2 的组合频次较少。这进一步确认了测试中 MCS=11 是最常见的情况。

在 2 AP 测试中, NSS 值始终固定为 2, 表明网络使用两个空间流。MCS=11 的组合出现最频繁, 表明在这个网络环境下, 能够经常保持高效的 MCS 编码模式, 代表较好的网络性能。

2. 3 AP 测试结果分析:

1. MCS 预测分布 (MCS Prediction Distribution):

左上图显示了 3 AP 测试中, MCS 的预测值分布。与 2 AP 测试类似, MCS=11 的频率最高, 接近 35 次, MCS=4 的频率也较高, 约为 15 次, 但总体上, MCS 值的分布更加分散, 有更多中间值出现。这表明在 3 AP 的网络环境下, 虽然较高的 MCS 仍占主导, 但 MCS 值的变化范围更广。

2. NSS 预测分布 (NSS Prediction Distribution):

右上图展示了 3 AP 测试中的 NSS 预测分布。类似于 2 AP 测试, NSS 值仍然集中在 2, 未出现其他 NSS 值的情况。说明在 3 AP 测试中, 系统依然只使用两个空间流, 可能与测试设备或网络配置相关。

3. MCS 与 NSS 预测关系 (Relationship Between MCS and NSS Predictions):

左下图显示了 3 AP 测试中, MCS 和 NSS 的关系。NSS 仍然保持为 2, 而 MCS 值在 4 到 11 之间波动。与 2 AP 测试类似, NSS 值未随着 MCS 的变化而改变, NSS 值固定为 2, 表明无论 MCS 如何调整, 系统使用的空间流数没有变化。

4. MCS 与 NSS 组合的计数 (Count of MCS and NSS Prediction Combinations):

右下图展示了 3 AP 测试中, MCS=11 与 NSS=2 的组合最为频繁, 接近 35 次, MCS=4 与 NSS=2 的组合出现约 15 次, 但在 3 AP 的情况下, 其他 MCS 值的组合也有一定的频次, 分布相对更加分散。

在 3 AP 测试中, NSS 值同样固定为 2, 表明无论增加了多少 AP, 空间流数依然不变。但与 2 AP 相比, MCS 值的分布更加分散, 尽管 MCS=11 仍是主流组合, 其他中间值的出现频率有所增加, 表明在 3 AP 网络下, MCS 的动态调整更加频繁, 网络性能的波动较 2 AP 测试更加明显。

7.3 问题三结果分析

首先对训练数据集中各个特征之间的相关性进行可视化及分析，如图7.54所示，eirp与seq_time（发送数据帧序列的总时长）相关性最强，其次是nav，protocol与loc_id。

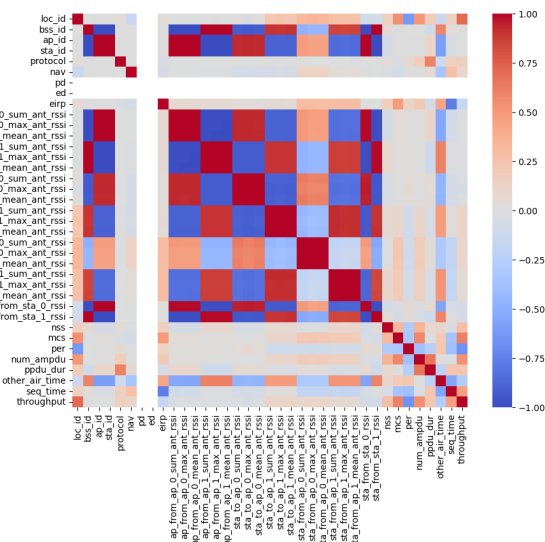


图 7.54 2AP 热力图

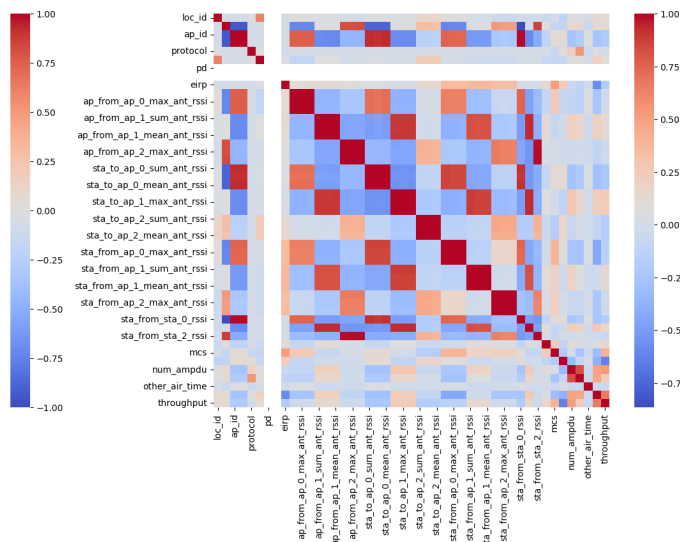


图 7.55 3AP 热力图

分析特征相关性热力图可以发现，在 2AP 结构下，与 seq_time 相关性最强的几个特征为 eirp，nav，protocol 和 loc_id，在 3AP 结构下，与 seq_time 相关性最强的几个特征为 eirp，nav，protocol 和 loc_id。

7.3.1 变量间的相关性分析

为进一步比较相关性，我们通过随机森林和 Xgboost 两种方法对特征相关性进行进一步评估，结果如图7.62和7.65所示。

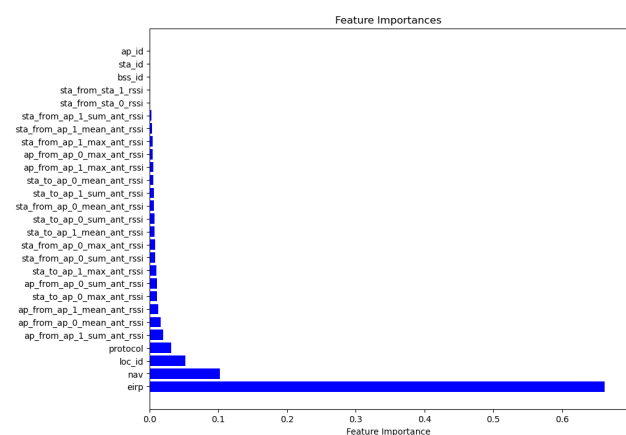


图 7.56 2AP 随机森林变量重要性排序

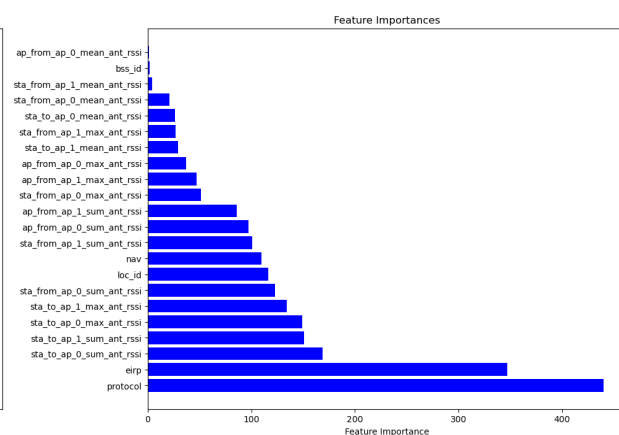


图 7.57 2AP xgboost 变量重要性排

可以进一步观察到，eirp 是相关性最强的特征，其次是 protocol，nav 与 loc_id。

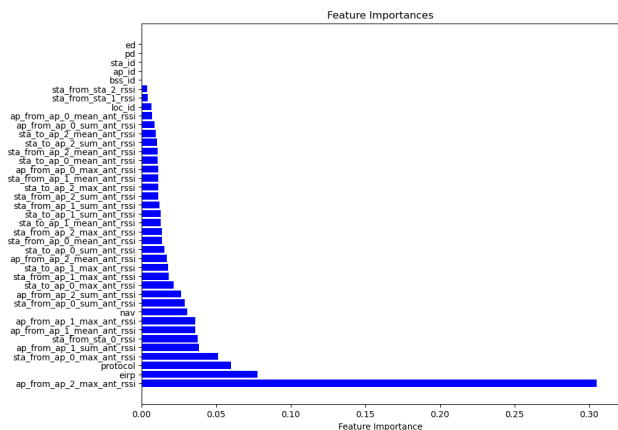


图 7.58 3AP 随机森林变量重要性排序

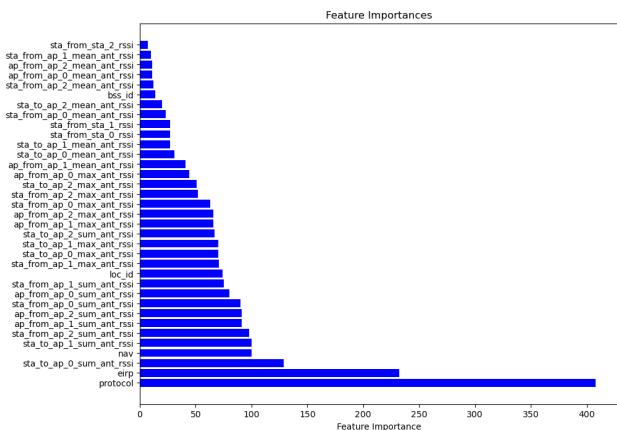


图 7.59 3AP xgboost 变量重要性排

可以进一步观察到,3AP 结构下参数之间的相关更为复杂,但 `eirp` 仍然是相关性很强的特征。

因为 AP 的发送功率 (`eirp`) 增加会直接导致其发射的信号强度 (RSSI) 提升。RSSI 是衡量信号强度的关键指标,高 RSSI 意味着信号强、干扰小,从而增加了数据传输成功的概率。即同步传输的次数更多,而同步传输时,每个 AP 交替抢到信道,偶然同时发送,发送时长基本上各占一半。因此, `eirp` 与 `seq_time` 存在较强的负相关。这也验证了结果的合理性。

7.3.2 验证结果分析

本节进一步对 AP 发送数据帧序列的总时长预测模型,在训练过程中的验证结果进行分析,根据问题一中的比较结果,我们在问题三中值选用了随机森林和 XGBoost 模型进行比较。

- 随机森林: 是一种集成学习方法,它通过构建多个决策树并将它们的预测结果进行平均或投票来得到最终的预测结果。随机森林在构建每棵决策树时都会随机选择部分样本和特征进行训练,这使得每棵决策树都具有一定的差异性。通过集成多棵决策树的预测结果,随机森林能够显著降低模型的方差并提高预测的准确性。随机森林在处理高维数据、缺失数据以及具有复杂相互作用的变量时表现出色。
- XGBoost 是一种高效的集成学习算法,属于梯度提升树 (Gradient Boosting Decision Tree) 的变种。它通过迭代地训练一系列的决策树模型并将它们组合成一个强大的模型来实现预测。XGBoost 在损失函数中加入了正则化项以控制模型的复杂度并防止过拟合。同时, XGBoost 还采用了多种优化技术来提高训练速度和预测精度。XGBoost 在机器学习竞赛和工业应用中均取得了优异的性能表现。

表7.60和7.61分别为 9 种模型在 2AP 结构和 3AP 结构下的 MSE, MAE 以及 MAPE 指标的对比。

图 7.60 2AP 结构下验证结果

	随机森林	xgboost
MSE	1.926	2.144
MAE	0.894	0.923
MAPE	0.051	0.052

图 7.61 3AP 结构下验证结果

	随机森林	xgboost
MSE	3.255	3.412
MAE	1.200	1.245
MAPE	0.092	0.095

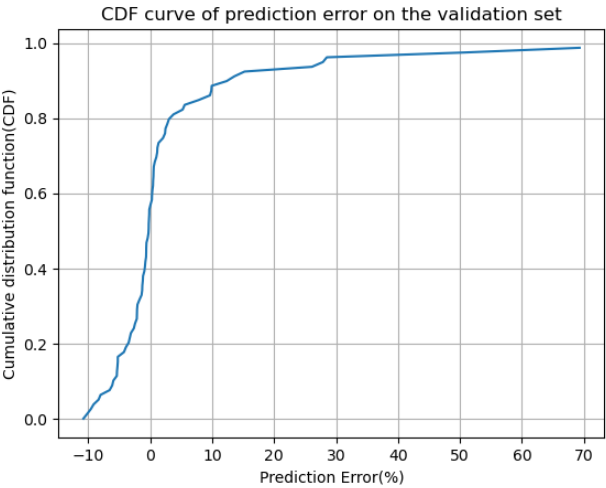


图 7.62 2ap 随机森林 CDF 曲线

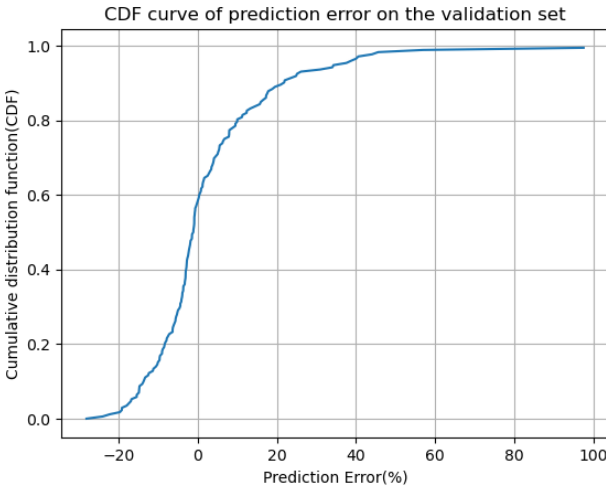


图 7.63 3ap 随机森林 CDF 曲线

从表7.60可以发现，随机森林和 XGBoost 的测试结果都较为优秀，随机森林的 MSE 最低为 10.421，XGBoost 的 MAE 和 MAPE 都最低，分别为 1.756 和 0.064。因此，在测试集的预测种，我们选用 XGBoost 来对 2AP 结构下的 seq_time 进行预测。

此外，我们还对验证结果的 CDF 进行了可视化，CDF 图如下所示：

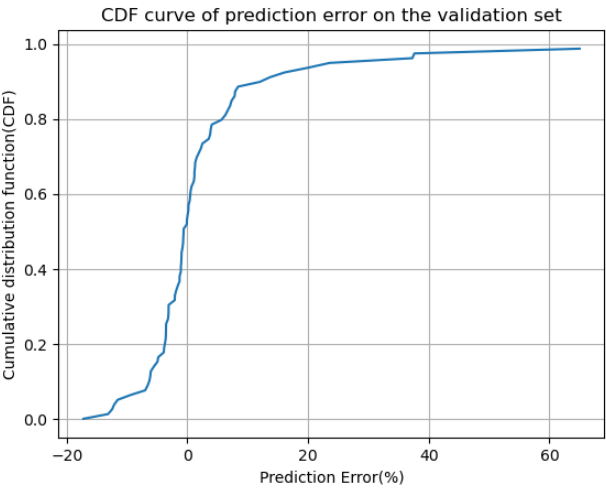


图 7.64 2apXGBoost CDF 曲线

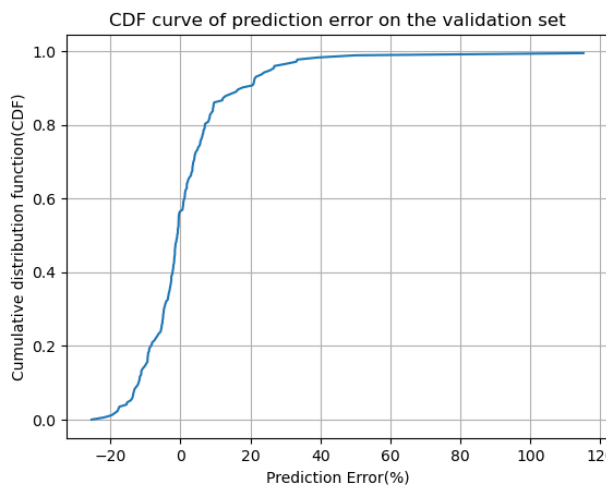


图 7.65 3apXGBoost CDF 曲线

7.3.3 测试结果分析

测试结果分别保存在附件：`test_set_1_2ap.csv` 以及 `test_set_1_3ap.csv` 当中。

8 模型评价

8.1 问题一模型评价

通过分析可知，问题一是回归问题。我们采用了线性回归、岭回归、支持向量机等多种回归模型。回归模型是一种统计方法，用于分析变量之间的关系。在预测发送机会的影响时，采用回归模型的优点在于：AP 发送机会和 WLAN 的吞吐量会受到网络拓扑、业务流量等多种影响。回归模型能够识别和量化多种变量之间的关系，其结果通常较为直观，可以通过回归系数清晰地看出各个因素具体影响及强弱关系。此外，部分回归模型不需要对数据进行预处理。进行训练和优化的数据是现实收集到的实际数据，这确保了模型的适用性和可靠性。在建模过程中，我们发现的回归模型也存在有缺点。我们收集到的 WLAN 通信数据都是实际数据，会掺杂噪音。当使用的回归模型对异常值和噪声敏感时，可能导致结果偏差。

8.2 问题二模型评价

在问题二中，模型需要根据输入特征（如 RSSI、门限等）来预测输出类别（即选择最合适的 MCS 和 NSS 组合），这是一种典型的分类任务。为了保证结果的准确性，我们采用了 Logistic 回归、Softmax 回归、随机森林等八种分类模型。在预测（MCS, NSS）时，使用分类模型的优点有：抗过拟合能力强：部分回归模型采用集成学习的方式，这样大大减小了过拟合风险，提高了模型的泛化能力。特别是针对 WLAN 数据通信影响因素较多、特征数量大的情况，更容易出现过拟合问题；高维特征处理：许多分类模型可以有效处理高维数据，适应 RSSI、门限等多种重要特征。不仅如此，对于 WLAN 数据通信实测到的多种复杂数据处理，也可表现出良好鲁棒性。同样，分类模型也存在部分缺点，主要体现在：模型解释性差：在 WLAN 应用中，对网络进行优化需要理解模型决策的依据。然而部分分类模型对（MCS, NSS）的预测表现良好性能，但是在解释性上相对较差，难以理解其内部决策过程；计算开销大。并且复杂模型（如随机森林或深度学习模型）在处理 WLAN 的实时数据时，计算资源需求较高。

8.3 问题三模型评价

问题三也是回归问题，我们采取了同问题一一样的八种模型进行回归建模。其目的在于对系统吞吐量进行建模，并实现对吞吐量的预测。采用回归模型的优势在于：在影响 WLAN 的吞吐量的多个特征中，部分特征如：距离、用户数量等与信号强度、吞吐量呈正相关，其他特征呈现非线性关系。回归模型既可以处理线性关系又可以处理非线性关系，能够适应各种数据特征和结构。另一方面，回归模型能够定量分析自变量（如 RSSI、用户数量、网络负载）与系统吞吐量之间的关系，为网络优化提供明确依据。问题三的回归建模缺点主要存在于：WLAN 的吞吐量受多种实时因素影响，如网络拥塞、设备数量等，简

单回归模型难以适应这些快速变化的环境条件。为了提高预测准确性，可能需要使用更复杂的回归方法（如多项式回归、岭回归等），但这也可能增加模型的理解和实现难度。

9 未来展望

当前的特征提取可能并未完全捕捉到吞吐量的所有影响因素。我们计划探索更多的高级特征，例如时序特征、历史吞吐量趋势等。此外我们将尝试构建多个模型的集成，如加权平均或模型堆叠，以进一步提高预测精度。另外，由于本研究的数据集规模有限，未来将引入更多的实测数据，以验证模型的通用性和鲁棒性。最后考虑到 WLAN 网络的动态变化特性，我们计划开发能够实时更新的在线学习模型，以适应网络环境的变化。

参考文献

- [1] 赵俊, IEEE802. 11 无线局域网性能分析和优化 [D], 中国科学院研究生院 (计算技术研究所), 2003.
- [2] Hung F Y, Marsic I, Performance analysis of the IEEE 802.11 DCF in the presence of the hidden stations, *Computer Networks*, 54(15):2674-2687, 2010.
- [3] Bellalta B, IEEE 802.11 ax: High-efficiency WLANs, *IEEE wireless communications*, 23 (1):38-46, 2016.
- [4] Surendranathan U, Wi-Fi 6: performance analysis of IEEE 802.11 AX, 2019.
- [5] 谢希仁, 计算机网络教授, 计算机网络, 电子工业出版社, 2003.
- [6] Shafique K, Khawaja B A, Sabir F, et al., Internet of things (IoT) for next-generation smart systems: A review of current challenges, future trends and prospects for emerging 5G-IoT scenarios, *Ieee Access*, 8:23022-23040, 2020.
- [7] Yuan X, Qin X, Tian F, et al., Coexistence between Wi-Fi and LTE on unlicensed spectrum: A human-centric approach, *IEEE Journal on Selected areas in Communications*, 35(4):964-977, 2017.
- [8] Aime M D, Calandriello G, Liyo A, Dependability in wireless networks: Can we rely on WiFi?, *IEEE Security & Privacy*, 5(1):23-29, 2007.
- [9] 罗浪, 无线局域网 CSMA/CA 协议的研究与实现, 成都理工大学, 2016.
- [10] Paul U, Kashyap A, Maheshwari R, et al., Passive measurement of interference in wifi networks with application in misbehavior detection, *IEEE transactions on mobile computing*, 12(3):434-446, 2011.
- [11] Cirik A C, Taghizadeh O, Lampe L, et al., Linear transceiver design for full-duplex multi-cell MIMO systems, *IEEE Access*, 4:4678-4689, 2016.
- [12] Cao X, Yang B, Wang K, et al., AI-Empowered Multiple Access for 6G: A Survey of Spectrum Sensing, Protocol Designs, and Optimizations, *arXiv preprint arXiv:2406.13335*, 2024.
- [13] Raschellà A, Bouhafs F, Mackay M, et al., AP selection algorithm based on a potential game for large IEEE 802.11 WLANs, *NOMS 2018-2018 IEEE/IFIP Network Operations and Management Symposium*, IEEE, 1-7, 2018.
- [14] Mahesh B, Machine learning algorithms-a review, *International Journal of Science and Research (IJSR)*. [Internet], 9(1):381-386, 2020.

附录 A Python 源程序

A.1 第 1 问程序

q1 preprocessing data.py

```
import pandas as pd
import numpy as np

# 计算同步、异步、混合状态
def process_transmission_data_2ap(df):
    # 创建新列 'Transmission Status' 和 'NAV Status'
    df['Transmission Status'] = 0 # 1: 同步, 2: 异步, 3: 混合
    # df['NAV Status'] = 1 # 1: 有效, 0: 无效

    # 处理每一行的 RSSI 值, 并根据逻辑判断状态
    for index, row in df.iterrows():
        apid = row["ap_id"]
        rssi0_max = row['ap_from_ap_0_max_ant_rssi']
        rssi1_max = row['ap_from_ap_1_max_ant_rssi']
        rssi0_mean = row['ap_from_ap_0_mean_ant_rssi']
        rssi1_mean = row['ap_from_ap_1_mean_ant_rssi']

        # 动态获取每一行的 PD, ED, NAV
        pd_value = row['pd']
        ed_value = row['ed']
        nav_value = row['nav']

        if apid == 1:
            # 判断传输状态
            if (rssi0_max > ed_value):
                df.at[index, 'Transmission Status'] = 1 # 同步
            elif ((rssi0_max < ed_value) and (rssi0_mean > nav_value)):
                df.at[index, 'Transmission Status'] = 2 # 混合
            elif rssi0_mean < nav_value:
                df.at[index, 'Transmission Status'] = 3 # 异步
            else:
                # 判断传输状态
                if (rssi1_max > ed_value):
                    df.at[index, 'Transmission Status'] = 1 # 同步
                elif ((rssi1_max < ed_value) and (rssi1_mean > nav_value)):
```

```

df.at[index, 'Transmission Status'] = 2 # 混合
elif rssi1_mean < nav_value:
df.at[index, 'Transmission Status'] = 3 # 异步

return df

def process_transmission_data_3ap(df):
# 创建新列 'Transmission Status' 和 'NAV Status'
df['Transmission Status'] = 0 # 1: 同步, 2: 异步, 3: 混合
# df['NAV Status'] = 1 # 1: 有效, 0: 无效

# 处理每一行的 RSSI 值, 并根据逻辑判断状态
for index, row in df.iterrows():
apid = row["ap_id"]
rssi0_max = row['ap_from_ap_0_max_ant_rssi']
rssi1_max = row['ap_from_ap_1_max_ant_rssi']
rssi2_max = row['ap_from_ap_2_max_ant_rssi']
rssi0_mean = row['ap_from_ap_0_mean_ant_rssi']
rssi1_mean = row['ap_from_ap_1_mean_ant_rssi']
rssi2_mean = row['ap_from_ap_2_mean_ant_rssi']

# 动态获取每一行的 PD, ED, NAV
pd_value = row['pd']
ed_value = row['ed']
nav_value = row['nav']

if apid == 1:
# 判断传输状态
if ((rssi0_max > ed_value) and (rssi2_max > ed_value)):
df.at[index, 'Transmission Status'] = 1 # 同步
elif ((rssi0_mean < nav_value) and (rssi2_mean < nav_value)):
:
df.at[index, 'Transmission Status'] = 3 # 异步
else:
df.at[index, 'Transmission Status'] = 2 # 混合
elif apid == 2:
# 判断传输状态
if ((rssi0_max > ed_value) and (rssi1_max > ed_value)):

```

```

df.at[index, 'Transmission Status'] = 1 # 同步
elif ((rssi0_mean < nav_value) and (rssi1_mean < nav_value))
:
df.at[index, 'Transmission Status'] = 3 # 异步
else:
df.at[index, 'Transmission Status'] = 2 # 混合
else:
# 判断传输状态
if ((rssi1_max > ed_value) and (rssi2_max > ed_value)):
df.at[index, 'Transmission Status'] = 1 # 同步
elif ((rssi1_mean < nav_value) and (rssi2_mean < nav_value))
:
df.at[index, 'Transmission Status'] = 3 # 异步
else:
df.at[index, 'Transmission Status'] = 2 # 混合

# 返回处理后的数据
return df

def ap2(name):
# 原始数据
odata = pd.read_csv("../.." + name)
# print(odata.head())

# 需要的特征
features = ["loc_id", "bss_id", "ap_id", "sta_id",
"protocol",
"nav", "pd", "ed",
"eirp",

"ap_from_ap_0_sum_ant_rssi", "ap_from_ap_0_max_ant_rssi", "
    ap_from_ap_0_mean_ant_rssi", "ap_from_ap_1_sum_ant_rssi",
    "ap_from_ap_1_max_ant_rssi", "ap_from_ap_1_mean_ant_rssi",
    ",
"sta_to_ap_0_sum_ant_rssi", "sta_to_ap_0_max_ant_rssi", "
    sta_to_ap_0_mean_ant_rssi", "sta_to_ap_1_sum_ant_rssi", "
    sta_to_ap_1_max_ant_rssi", "sta_to_ap_1_mean_ant_rssi",
"sta_from_ap_0_sum_ant_rssi", "sta_from_ap_0_max_ant_rssi",
    "sta_from_ap_0_mean_ant_rssi",

```

```

"sta_from_ap_1_sum_ant_rssi", "sta_from_ap_1_max_ant_rssi",
    "sta_from_ap_1_mean_ant_rssi",
"sta_from_sta_0_rssi", "sta_from_sta_1_rssi",
# "seq_time"
]

# 需要特征对应的数据
need_data = odata[features]

# 把字符串映射为数字
# 创建一个字典来映射字符串到数字
string_to_number = {
    "loc0":10, "loc1":1, "loc2":2, "loc3":3, "loc30":30, "
    loc31":31, "loc32":32, "loc33":33, "loc4":4,
    "ap_0":10, "ap_1":1, "ap_2":2,
    "sta_0":10, "sta_1":1, "sta_2":2,
    "tcp":1, "udp":2,
}
data = need_data.replace(string_to_number)

# 把NAN换成0
data = data.fillna(0)

# 把数组修改为列表，取平均
import ast
# 遍历需要修改的位置
rows, cols = data.iloc[:, 9:-3].shape[0], data.iloc[:,
    9:-3].shape[1]
for row in range(rows):
    for col in range(9,cols+9+1):
        da = data.iloc[row, col]
        # 先将字符转换为数组
        if isinstance(da, str):
            # data.iloc[row, col] = ast.literal_eval(da)
            the_list = ast.literal_eval(da)
            if len(the_list) <= 2: # 数据小于3个取平均
                data.iloc[row, col] = sum(the_list) / len(the_list)
            else: # 否则去掉一个最高值去掉一个最低值，求平均
                # print("before",len(the_list))

```

```

the_list.remove(max(the_list))
the_list.remove(min(the_list))
# print("after", len(the_list))
data.iloc[row, col] = sum(the_list) / len(the_list)

data = process_transmission_data_2ap(data)
data["seq_time"] = odata["seq_time"]
print(data)

data.to_csv(name, index=False)

"""将几个csv文件合并为1个"""
def merge_data(csv_files, name):
merged_df = pd.DataFrame() # 初始化一个空的DataFrame用于合并

for file in csv_files:
df = pd.read_csv(file)
# 如果merged_df是空的, 直接赋值
if merged_df.empty:
merged_df = df
else:
merged_df = pd.concat([merged_df, df], axis=0)

# 查看合并后的DataFrame
print(merged_df)

# 如果你想要保存合并后的DataFrame到新的CSV文件
merged_df.to_csv(name, index=False)

def ap3(name):
# 原始数据
odata = pd.read_csv("../.." + name)
# print(odata.head())

```



```

# 需要的特征
features = ["loc_id", "bss_id", "ap_id", "sta_id",
"protocol",
"nav", "pd", "ed",
"eirp",

"ap_from_ap_0_sum_ant_rssi", "ap_from_ap_0_max_ant_rssi", "
    ap_from_ap_0_mean_ant_rssi",
"ap_from_ap_1_sum_ant_rssi", "ap_from_ap_1_max_ant_rssi", "
    ap_from_ap_1_mean_ant_rssi",
"ap_from_ap_2_sum_ant_rssi", "ap_from_ap_2_max_ant_rssi", "
    ap_from_ap_2_mean_ant_rssi",
"sta_to_ap_0_sum_ant_rssi", "sta_to_ap_0_max_ant_rssi", "
    sta_to_ap_0_mean_ant_rssi",
"sta_to_ap_1_sum_ant_rssi", "sta_to_ap_1_max_ant_rssi", "
    sta_to_ap_1_mean_ant_rssi",
"sta_to_ap_2_sum_ant_rssi", "sta_to_ap_2_max_ant_rssi", "
    sta_to_ap_2_mean_ant_rssi",
"sta_from_ap_0_sum_ant_rssi", "sta_from_ap_0_max_ant_rssi",
    "sta_from_ap_0_mean_ant_rssi",
"sta_from_ap_1_sum_ant_rssi", "sta_from_ap_1_max_ant_rssi",
    "sta_from_ap_1_mean_ant_rssi",
"sta_from_ap_2_sum_ant_rssi", "sta_from_ap_2_max_ant_rssi",
    "sta_from_ap_2_mean_ant_rssi",
"sta_from_sta_0_rssi", "sta_from_sta_1_rssi", "
    sta_from_sta_2_rssi",

# "seq_time"
]

# 需要特征对应的数据
need_data = odata[features]

# 把字符串映射为数字
# 创建一个字典来映射字符串到数字
string_to_number = {
    "loc0":10, "loc1":1, "loc2":2, "loc3":3, "loc30":30, "
        loc31":31, "loc32":32, "loc33":33, "loc4":4,
    "ap_0":10, "ap_1":1, "ap_2":2,

```

```

        "sta_0":10, "sta_1":1, "sta_2":2,
        "tcp":1, "udp":2,
    }
    data = need_data.replace(string_to_number)

    # 把NAN换成0
    data = data.fillna(0)

    # 把数组修改为列表，取平均
    import ast
    # 遍历需要修改的位置
    rows, cols = data.iloc[:, 9:-3].shape[0], data.iloc[:,
        9:-3].shape[1]
    for row in range(rows):
        for col in range(9,cols+9+1):
            da = data.iloc[row, col]
            # 先将字符转换为数组
            if isinstance(da, str):
                # data.iloc[row, col] = ast.literal_eval(da)
                the_list = ast.literal_eval(da)
                if len(the_list) <= 2: # 数据小于3个取平均
                    data.iloc[row, col] = sum(the_list) / len(the_list)
                else: # 否则去掉一个最高值去掉一个最低值，求平均
                    # print("before",len(the_list))
                    the_list.remove(max(the_list))
                    the_list.remove(min(the_list))
                    # print("after", len(the_list))
                    data.iloc[row, col] = sum(the_list) / len(the_list)

    data = process_transmission_data_3ap(data)
    data["seq_time"] = odata["seq_time"]
    print(data)

    data.to_csv(name, index=False)

    if __name__ == '__main__':
        # 2AP
        names = ["training_set_2ap_loc0_nav82.csv", "

```

```

        training_set_2ap_loc0_nav86.csv", "
        training_set_2ap_loc1_nav82.csv", "
        training_set_2ap_loc1_nav86.csv", "
        training_set_2ap_loc2_nav82.csv"]
    for name in names:
        ap2(name)
    merge_data(names, 'merged_2ap.csv')

# 3AP
names = [
    "training_set_3ap_loc30_nav82.csv", "
        training_set_3ap_loc30_nav86.csv", "
        training_set_3ap_loc31_nav82.csv", "
        training_set_3ap_loc31_nav86.csv",
    "training_set_3ap_loc32_nav82.csv", "
        training_set_3ap_loc32_nav86.csv", "
        training_set_3ap_loc33_nav82.csv", "
        training_set_3ap_loc33_nav88.csv"
]
    for name in names:
        ap3(name)
    merge_data(names, 'merged_3ap.csv')

# test-2ap
names = ["test_set_1_2ap.csv"]
    for name in names:
        ap2(name)

# test-3ap
names = ["test_set_1_3ap.csv"]
    for name in names:
        ap3(name)

```

q1 feature ranking.py

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

```

# 随机森林特征重要性排序
def rf(X, y, feature_names, test_data):
    from sklearn.ensemble import RandomForestRegressor

    # 假设X是特征矩阵，y是目标变量
    model = RandomForestRegressor().fit(X, y)
    importances = model.feature_importances_

    # 排序
    sorted_indices = np.argsort(importances)[::-1]
    print(sorted_indices)
    sorted_features = [feature_names[i] for i in sorted_indices]
    sorted_importances = importances[sorted_indices]

    # 打印结果
    for feature, importance in zip(sorted_features,
                                   sorted_importances):
        print(f"{feature}: {importance}")

    # 可视化特征重要性
    plt.figure(figsize=(10, 8))
    plt.barh(sorted_features, sorted_importances, color='b', align='center')
    plt.xlabel('Feature Importance')
    plt.title('Feature Importances')
    plt.show()

    test_X = test_data.iloc[:, :-1]

    # 使用模型进行预测
    y_pred = model.predict(test_X)

    # 打印预测结果
    print("Predicted value:", y_pred)

    # 将预测结果保存到CSV文件中
    predictions_df = pd.DataFrame(y_pred, columns=['seq_time'])
    predictions_df.to_csv('predictions.csv', index=False)

```

```

def xgb_tree(X, y, feature_names, test_data):
    import xgboost as xgb

    # 假设DMatrix是XGBoost的数据结构
    dtrain = xgb.DMatrix(X, label=y)
    params = {
        'max_depth': 5,
        'eta': 0.1,
        'objective': 'reg:squarederror', # 对于分类问题, 使用 '
            binary:logistic' 等
    }
    num_round = 100
    bst = xgb.train(params, dtrain, num_round)

    importances = bst.get_fscore()
    # 将重要性字典转换为列表, 以便排序和绘图
    importances_list = [(k, v) for k, v in importances.items()]
    print(importances_list)
    sorted_importances = sorted(importances_list, key=lambda x: x
        [1], reverse=True)
    # sorted_features = [feature_names[i] for i in sorted_indices]

    # 可视化特征重要性
    plt.figure(figsize=(10, 8))
    plt.barh(range(len(sorted_importances)), [imp[1] for imp in
        sorted_importances], color='b')
    plt.yticks([i for i in range(len(sorted_importances))], [imp[0]
        for imp in sorted_importances])
    plt.xlabel('Feature Importance')
    plt.title('Feature Importances')
    plt.show()

    # 打印结果
    for feature, importance in sorted_importances:
        print(f"{feature}: {importance}")

    test_X = test_data.iloc[:, :-1]

```

```

# 将测试集转换为DMatrix对象
dtest = xgb.DMatrix(test_X)

# 使用模型进行预测
y_pred = bst.predict(dtest)

# 打印预测结果（可选）
print("Predictions:", y_pred)


if __name__ == '__main__':
# 读取合并的数据集
label= 2

# 读取合并的数据集
if label==2:
data = pd.read_csv("merged_2ap.csv")
test = pd.read_csv("test_set_1_2ap.csv")
features = ["loc_id", "bss_id", "ap_id", "sta_id",
"protocol",
"nav",
"eirp",

"ap_from_ap_0_sum_ant_rssi", "ap_from_ap_0_max_ant_rssi", "
    ap_from_ap_0_mean_ant_rssi", "ap_from_ap_1_sum_ant_rssi", "
    ap_from_ap_1_max_ant_rssi", "ap_from_ap_1_mean_ant_rssi",
"sta_to_ap_0_sum_ant_rssi", "sta_to_ap_0_max_ant_rssi", "
    sta_to_ap_0_mean_ant_rssi", "sta_to_ap_1_sum_ant_rssi", "
    sta_to_ap_1_max_ant_rssi", "sta_to_ap_1_mean_ant_rssi",
"sta_from_ap_0_sum_ant_rssi", "sta_from_ap_0_max_ant_rssi", "
    sta_from_ap_0_mean_ant_rssi",
"sta_from_ap_1_sum_ant_rssi", "sta_from_ap_1_max_ant_rssi", "
    sta_from_ap_1_mean_ant_rssi",
"sta_from_sta_0_rssi", "sta_from_sta_1_rssi",

"seq_time"

```

```

]
else:
data = pd.read_csv("merged_3ap.csv")
test = pd.read_csv("test_set_1_3ap.csv")
features = ["loc_id", "bss_id", "ap_id", "sta_id",
"protocol",
"nav",
"eirp",

"ap_from_ap_0_sum_ant_rssi", "ap_from_ap_0_max_ant_rssi", "
    ap_from_ap_0_mean_ant_rssi",
"ap_from_ap_1_sum_ant_rssi", "ap_from_ap_1_max_ant_rssi", "
    ap_from_ap_1_mean_ant_rssi",
"ap_from_ap_2_sum_ant_rssi", "ap_from_ap_2_max_ant_rssi", "
    ap_from_ap_2_mean_ant_rssi",
"sta_to_ap_0_sum_ant_rssi", "sta_to_ap_0_max_ant_rssi", "
    sta_to_ap_0_mean_ant_rssi",
"sta_to_ap_1_sum_ant_rssi", "sta_to_ap_1_max_ant_rssi", "
    sta_to_ap_1_mean_ant_rssi",
"sta_to_ap_2_sum_ant_rssi", "sta_to_ap_2_max_ant_rssi", "
    sta_to_ap_2_mean_ant_rssi",
"sta_from_ap_0_sum_ant_rssi", "sta_from_ap_0_max_ant_rssi", "
    sta_from_ap_0_mean_ant_rssi",
"sta_from_ap_1_sum_ant_rssi", "sta_from_ap_1_max_ant_rssi", "
    sta_from_ap_1_mean_ant_rssi",
"sta_from_ap_2_sum_ant_rssi", "sta_from_ap_2_max_ant_rssi", "
    sta_from_ap_2_mean_ant_rssi",
"sta_from_sta_0_rssi", "sta_from_sta_1_rssi", "
    sta_from_sta_2_rssi",

"seq_time"
]

X = data.iloc[:, :-1]#.values
y = data.iloc[:, -1]#.values

# 随机森林
print("随机森林重要性排序:")

```

```

rf(X, y, features, test)

# xgboost
print("xgboost重要性排序:")
xgb_tree(X, y, features, test)

# transformer

```

q1 prediction.py

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error,
    mean_absolute_error, mean_absolute_percentage_error

def cdf_draw(y_pred, y_val):
    # 计算误差并绘制CDF曲线
    errors = y_pred - y_val
    errors_percentage = (errors / y_val) * 100 # 直接与y_val进行运算，不需要转换为list

    # 计算误差的CDF
    sorted_errors = np.sort(errors_percentage)
    cdf = np.arange(len(sorted_errors)) / float(len(sorted_errors))

    # 绘制平滑的CDF曲线
    plt.figure()
    plt.plot(sorted_errors, cdf) # 使用连线和平滑点
    plt.xlabel('Prediction Error(%)')
    plt.ylabel('Cumulative distribution function(CDF)')
    plt.title('CDF curve of prediction error on the validation set')
    plt.grid(True)
    plt.show()

    # 计算误差的90%分位数
    error_90 = np.percentile(np.abs(errors_percentage), 90)
    model_accuracy = 1 - error_90 / 100

```



```

print(f"模型在验证集上的精度为: {model_accuracy:.2f}")

def cdf(y_pred, y_val, color, label):
    # 计算误差并绘制CDF曲线
    errors = y_pred - y_val
    errors_percentage = (errors / y_val) * 100 # 直接与y_val进行运算, 不需要转换为list

    # 计算误差的CDF
    sorted_errors = np.sort(errors_percentage)
    cdf = np.arange(len(sorted_errors)) / float(len(sorted_errors))

    # 绘制平滑的CDF曲线
    # plt.figure()
    plt.plot(sorted_errors, cdf, color=color, label=label) # 使用连线和平滑点
    # plt.xlabel('Prediction Error(%)')
    # plt.ylabel('Cumulative distribution function(CDF)')
    # plt.title('CDF curve of prediction error on the validation set')
    # plt.grid(True)
    # plt.show()

    # 计算误差的90%分位数
    error_90 = np.percentile(np.abs(errors_percentage), 90)
    model_accuracy = 1 - error_90 / 100
    print(f"模型在验证集上的精度为: {model_accuracy:.2f}")
    return model_accuracy

# 线性回归
def LR(X_train, y_train, X_test, y_test, color):
    from sklearn.linear_model import LinearRegression

    # 创建模型
    model = LinearRegression()

    # 训练模型
    model.fit(X_train, y_train)

```

```

# 预测
predictions = model.predict(X_test)
# 输出预测结果
# print("Predicted value:", predictions)

# 输出评价指标
mse = mean_squared_error(y_test, predictions)
print(f"Mean Squared Error: {mse}")
mae = mean_absolute_error(y_test, predictions)
print(f"Mean Absolute Error: {mae}")
mape = mean_absolute_percentage_error(y_test, predictions)
print(f"Mean Squared Error: {mape}")

# cdf_draw(predictions, y_test)
model_accuracy = cdf(predictions, y_test, color, label="Linear
    Regression")
return mse, model_accuracy, "lr"

# 岭回归
def RR(X_train, y_train, X_test, y_test, color):
    from sklearn.linear_model import Ridge

# 使用相同的数据
model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
# 输出预测结果
# print("Predicted value:", predictions)

# 输出评价指标
mse = mean_squared_error(y_test, predictions)
print(f"Mean Squared Error: {mse}")
mae = mean_absolute_error(y_test, predictions)
print(f"Mean Absolute Error: {mae}")
mape = mean_absolute_percentage_error(y_test, predictions)
print(f"Mean Squared Error: {mape}")

```

```

model_accuracy = cdf(predictions, y_test, color, label="Ridge")
return mse, model_accuracy, "rr"

# 拉索回归
def lasso(X_train, y_train, X_test, y_test, color):
    from sklearn.linear_model import Lasso

    model = Lasso(alpha=0.1)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    # 输出预测结果
    # print("Predicted value:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")
    mae = mean_absolute_error(y_test, predictions)
    print(f"Mean Absolute Error: {mae}")
    mape = mean_absolute_percentage_error(y_test, predictions)
    print(f"Mean Squared Error: {mape}")

    model_accuracy = cdf(predictions, y_test, color, label="lasso")
    return mse, model_accuracy, "lasso"

# 弹性网回归
def elastic(X_train, y_train, X_test, y_test, color):
    from sklearn.linear_model import ElasticNet

    model = ElasticNet(alpha=1.0, l1_ratio=0.7)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    # 输出预测结果
    # print("Predicted value:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")

```

```

mae = mean_absolute_error(y_test, predictions)
print(f"Mean Absolute Error: {mae}")
mape = mean_absolute_percentage_error(y_test, predictions)
print(f"Mean Squared Error: {mape}")

model_accuracy = cdf(predictions, y_test, color, label="
    ElasticNet")
return mse, model_accuracy, "elastic"

# 多项式回归
def pr(X_train, y_train, X_test, y_test, color):
    from sklearn.preprocessing import PolynomialFeatures
    from sklearn.linear_model import LinearRegression

    poly_features = PolynomialFeatures(degree=2)
    X_poly = poly_features.fit_transform(X_train)

    model = LinearRegression()
    model.fit(X_poly, y_train)

    X_test_poly = poly_features.transform(X_test)
    predictions = model.predict(X_test_poly)
    # 输出预测结果
    # print("Predicted value:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")
    mae = mean_absolute_error(y_test, predictions)
    print(f"Mean Absolute Error: {mae}")
    mape = mean_absolute_percentage_error(y_test, predictions)
    print(f"Mean Squared Error: {mape}")

    model_accuracy = cdf(predictions, y_test, color, label="
        PolynomialFeatures")
    return mse, model_accuracy, "pr"

# KNN

```

```

def knn(X_train, y_train, X_test, y_test, color):
    from sklearn.neighbors import KNeighborsRegressor

    model = KNeighborsRegressor(n_neighbors=3)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    # 输出预测结果
    # print("Predicted value:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")
    mae = mean_absolute_error(y_test, predictions)
    print(f"Mean Absolute Error: {mae}")
    mape = mean_absolute_percentage_error(y_test, predictions)
    print(f"Mean Squared Error: {mape}")

    model_accuracy = cdf(predictions, y_test, color, label="KNN")
    return mse, model_accuracy, "knn"

# 决策树
def dt(X_train, y_train, X_test, y_test, color):
    from sklearn.tree import DecisionTreeRegressor

    model = DecisionTreeRegressor(random_state=42)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    # 输出预测结果
    # print("Predicted value:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")
    mae = mean_absolute_error(y_test, predictions)
    print(f"Mean Absolute Error: {mae}")
    mape = mean_absolute_percentage_error(y_test, predictions)
    print(f"Mean Squared Error: {mape}")

```

```

model_accuracy = cdf(predictions, y_test, color, label="
    DecisionTreeRegressor")
return mse, model_accuracy, "dt"

# 随机森林
def rf(X_train, y_train, X_test, y_test, color):
    from sklearn.ensemble import RandomForestRegressor

    # 假设x是特征矩阵, y是目标变量
    model = RandomForestRegressor().fit(X_train, y_train)

    # 使用模型进行预测
    predictions = model.predict(X_test)
    # 打印预测结果
    # print("Predicted value:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")
    mae = mean_absolute_error(y_test, predictions)
    print(f"Mean Absolute Error: {mae}")
    mape = mean_absolute_percentage_error(y_test, predictions)
    print(f"Mean Squared Error: {mape}")

    model_accuracy = cdf(predictions, y_test, color, label="Random
        Forest")

    # 将预测结果保存到CSV文件中
    # predictions_df = pd.DataFrame(predictions, columns=['seq_time
        '])
    # predictions_df.to_csv('predictions.csv', index=False)
    return mse, model_accuracy, "rf"

# Gradient Boosting Regression Trees
def GBRT(X_train, y_train, X_test, y_test, color):
    from sklearn.ensemble import GradientBoostingRegressor

```

```

model = GradientBoostingRegressor(n_estimators=100,
    learning_rate=0.1, max_depth=1, random_state=42)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
# 打印预测结果
# print("Predicted value:", predictions)

# 输出评价指标
mse = mean_squared_error(y_test, predictions)
print(f"Mean Squared Error: {mse}")
mae = mean_absolute_error(y_test, predictions)
print(f"Mean Absolute Error: {mae}")
mape = mean_absolute_percentage_error(y_test, predictions)
print(f"Mean Squared Error: {mape}")

model_accuracy = cdf(predictions, y_test, color, label="Gradient
    Boosting Regression Trees")
return mse, model_accuracy, "gbdt"

# XGBoost
def xgb_tree(X_train, y_train, X_test, y_test, color):
import xgboost as xgb

# 假设DMatrix是XGBoost的数据结构
dtrain = xgb.DMatrix(X_train, label=y_train)
params = {
    'max_depth': 5,
    'eta': 0.1,
    'objective': 'reg:squarederror', # 对于分类问题, 使用 '
        binary:logistic' 等
}
num_round = 100
bst = xgb.train(params, dtrain, num_round)

# 将测试集转换为DMatrix对象
dtest = xgb.DMatrix(X_test)

# 使用模型进行预测

```

```

predictions = bst.predict(dtest)
# 输出预测结果
# print("Predictions:", predictions)

# 输出评价指标
mse = mean_squared_error(y_test, predictions)
print(f"Mean Squared Error: {mse}")
mae = mean_absolute_error(y_test, predictions)
print(f"Mean Absolute Error: {mae}")
mape = mean_absolute_percentage_error(y_test, predictions)
print(f"Mean Squared Error: {mape}")

model_accuracy = cdf(predictions, y_test, color, label="XGBoost"
    )

# 将预测结果保存到CSV文件中
# predictions_df = pd.DataFrame(predictions, columns=['seq_time
    '])
# predictions_df.to_csv('predictions.csv', index=False)
return mse, model_accuracy, "xgb"

# SVM
def svm(X_train, y_train, X_test, y_test, color):
    from sklearn.svm import SVR

    model = SVR(kernel='rbf', C=100, gamma=0.1)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    # 输出预测结果
    # print("Predictions:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")
    mae = mean_absolute_error(y_test, predictions)
    print(f"Mean Absolute Error: {mae}")
    mape = mean_absolute_percentage_error(y_test, predictions)
    print(f"Mean Squared Error: {mape}")

```



```

model_accuracy = cdf(predictions, y_test, color, label="SVM")
return mse, model_accuracy, "svm"

def RegressionNet(X_train, y_train, X_test, y_test, color):
import torch
import torch.nn as nn
import torch.optim as optim

# 定义神经网络模型
class RegressionNet(nn.Module):
def __init__(self, input_size, hidden_size, output_size):
super(RegressionNet, self).__init__()
self.fc1 = nn.Linear(input_size, hidden_size)
self.relu = nn.ReLU()
self.fc2 = nn.Linear(hidden_size, output_size)

def forward(self, x):
out = self.fc1(x)
out = self.relu(out)
out = self.fc2(out)
return out

# 将numpy数组转换为torch张量
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32)

# 实例化模型
input_size = X_train_tensor.shape[1]
hidden_size = 50
output_size = 1
model = RegressionNet(input_size, hidden_size, output_size)

# 定义损失函数和优化器
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练模型

```

```

num_epochs = 1000
for epoch in range(num_epochs):
    # 前向传播
    outputs = model(X_train_tensor)
    loss = criterion(outputs.squeeze(), y_train_tensor)

    # 反向传播和优化
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # 打印每100个epoch的损失
    if (epoch+1) % 100 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}'
              )

    # 测试模型
    model.eval() # 设置模型为评估模式
    with torch.no_grad():
        predictions = model(X_test_tensor).squeeze()

    # 输出预测结果
    # print("Predictions:", predictions)

    # 输出评价指标
    mse = mean_squared_error(y_test, predictions)
    print(f"Mean Squared Error: {mse}")
    mae = mean_absolute_error(y_test, predictions)
    print(f"Mean Absolute Error: {mae}")
    mape = mean_absolute_percentage_error(y_test, predictions)
    print(f"Mean Squared Error: {mape}")

    model_accuracy = cdf(predictions, y_test, color, label="Neural
        Networks")
    return mse, model_accuracy, "nn"

if __name__ == '__main__':
    label= 3

```

```

# 读取合并的数据集
if label==2:
data = pd.read_csv("merged_2ap.csv")
else:
data = pd.read_csv("merged_3ap.csv")

X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

plt.figure(figsize=(8, 6))

final_mse = 100
final_accuracy = 0
final_name = ["a", "b"]
# 线性回归
print("线性回归:")
mse, model_accuracy, name = LR(X_train, y_train, X_test, y_test,
    color = "blue")
if mse < final_mse:
final_mse = mse
final_name[0] = name
if model_accuracy > final_accuracy:
final_accuracy = model_accuracy
final_name[1] = name

# 岭回归
print("岭回归:")
mse, model_accuracy, name = LR(X_train, y_train, X_test, y_test,
    color = "blue")
if mse < final_mse:
final_mse = mse
final_name[0] = name
if model_accuracy > final_accuracy:
final_accuracy = model_accuracy
final_name[1] = name

```

```

# lasso回归
print("lasso回归:")
mse, model_accuracy, name = LR(X_train, y_train, X_test, y_test,
    color = "blue")
if mse < final_mse:
    final_mse = mse
    final_name[0] = name
if model_accuracy > final_accuracy:
    final_accuracy = model_accuracy
    final_name[1] = name

# # 弹性网回归
# print("弹性网回归:")
# result, name = elastic(X_train, y_train, X_test, y_test, color
    = "red")

# # 多项式回归
# print("多项式回归:")
# result, name = pr(X_train, y_train, X_test, y_test, color = "
    red")

# knn
print("knn:")
mse, model_accuracy, name = knn(X_train, y_train, X_test, y_test
    , color = "blue")
if mse < final_mse:
    final_mse = mse
    final_name[0] = name
if model_accuracy > final_accuracy:
    final_accuracy = model_accuracy
    final_name[1] = name

# # 决策树
# print("决策树:")
# result, name = dt(X_train, y_train, X_test, y_test, color = "
    red")

# 随机森林

```

```

print("随机森林:")
mse, model_accuracy, name = rf(X_train, y_train, X_test, y_test,
    color = "red")
if mse < final_mse:
    final_mse = mse
    final_name[0] = name
if model_accuracy > final_accuracy:
    final_accuracy = model_accuracy
    final_name[1] = name

# # Gradient Boosting Regression Trees
# print("GBRT:")
# result, name = GBRT(X_train, y_train, X_test, y_test, color =
    "red")

# xgboost
print("xgboost:")
mse, model_accuracy, name = xgb_tree(X_train, y_train, X_test,
    y_test, color = "red")
if mse < final_mse:
    final_mse = mse
    final_name[0] = name
if model_accuracy > final_accuracy:
    final_accuracy = model_accuracy
    final_name[1] = name

# svm
print("svm:")
mse, model_accuracy, name = svm(X_train, y_train, X_test, y_test
    , color = "blue")
if mse < final_mse:
    final_mse = mse
    final_name[0] = name
if model_accuracy > final_accuracy:
    final_accuracy = model_accuracy
    final_name[1] = name

# NN
print("NN:")

```

```

mse, model_accuracy, name = RegressionNet(X_train, y_train,
    X_test, y_test, color = "blue")
if mse < final_mse:
    final_mse = mse
    final_name[0] = name
if model_accuracy > final_accuracy:
    final_accuracy = model_accuracy
    final_name[1] = name

print("mse最小的方法为:", final_name[0])
print("accuracy最大的方法为:", final_name[1])

# plt.figure()
# plt.plot(sorted_errors, cdf) # 使用连线和平滑点
plt.xlabel('Prediction Error(%)')
plt.ylabel('Cumulative distribution function(CDF)')
plt.title('CDF curve of prediction error on the validation set')
plt.legend()
plt.grid(True)
plt.show()

```

A.2 第2问程序

q2 prediction.py

```

{
    "cells": [
        {
            "cell_type": "code",
            "execution_count": 4,
            "metadata": {},
            "outputs": [],
            "source": []
        },
        {
            "cell_type": "code",
            "execution_count": 11,
            "metadata": {},
            "outputs": [
                {
                    "name": "stdout",

```

```

        "output_type": "stream",
        "text": [
            "MCS模型验证准确率：0.78\n",
            "NSS模型验证准确率：0.98\n",
            "../test_set_2_2ap.csv的预测结果已保存到../\n",
            "    test_set_2_2ap_predictions.csv\n",
            "../test_set_2_3ap.csv的预测结果已保存到../\n",
            "    test_set_2_3ap_predictions.csv\n"
        ]
    }
],
"source": [
    "import pandas as pd\n",
    "import numpy as np\n",
    "from sklearn.model_selection import train_test_split\n",
    ,
    "from sklearn.ensemble import RandomForestClassifier\n",
    "import ast\n",
    "# 定义训练数据文件列表\n",
    "training_files = [\n",
    "    '../training_set_2ap_loc0_nav82.csv',\n",
    "    '../training_set_2ap_loc0_nav86.csv',\n",
    "    '../training_set_2ap_loc1_nav82.csv',\n",
    "    '../training_set_2ap_loc1_nav86.csv',\n",
    "    '../training_set_2ap_loc2_nav82.csv',\n",
    "    '../training_set_3ap_loc30_nav82.csv',\n",
    "    '../training_set_3ap_loc30_nav86.csv',\n",
    "    '../training_set_3ap_loc31_nav82.csv',\n",
    "    '../training_set_3ap_loc31_nav86.csv',\n",
    "    '../training_set_3ap_loc32_nav82.csv',\n",
    "    '../training_set_3ap_loc32_nav86.csv',\n",
    "    '../training_set_3ap_loc33_nav82.csv',\n",
    "    '../training_set_3ap_loc33_nav88.csv'\n",
    "]\n",
    "\n",
    "# 读取并合并所有训练数据\n",
    "df_list = []\n",
    "for file in training_files:\n",
    "    df = pd.read_csv(file)\n",

```

```

"    df_list.append(df)\n",
"\n",
"training_data = pd.concat(df_list, ignore_index=True)\n",
",
"\n",
"# 定义计算SINR的函数\n",
"def calculate_sinr(row):\n",
"    ap_id = int(row['ap_id'][-1:])\n",
"    sta_id = row['sta_id']\n",
"    # 获取AP到STA的信号功率\n",
"    own_ap_rssi_col = f'sta_from_ap_{ap_id}'\n",
"    _sum_ant_rssi'\n",
"    own_rssi_dBm = row.get(own_ap_rssi_col, np.nan)\n",
"    # 获取其他AP到STA的干扰功率\n",
"    interference_mW = 0\n",
"    for col in row.index:\n",
"        if col.startswith('sta_from_ap_') and col.\n",
"endswith('_sum_ant_rssi'):\n",
"            ap_x = col.split('_')[3]\n",
"            if ap_x != str(ap_id):\n",
"                interference_rssi_dBm = row[col]\n",
"                # print(interference_rssi_dBm)\n",
"                if not pd.isna(interference_rssi_dBm):\n",
"n",
"                    if isinstance(interference_rssi_dBm\n",
", str):\n",
"                        # data.iloc[row, col] = ast.\n",
"literal_eval(da)\n",
"                        interference_rssi_dBm = ast.\n",
"literal_eval(interference_rssi_dBm) \n",
"                        interference_mW += 10 ** ((np.\n",
"mean(interference_rssi_dBm) - 30) / 10)\n",
"    # 计算信号功率\n",
"    if isinstance(own_rssi_dBm, str):\n",
"        own_rssi_dBm = ast.literal_eval(own_rssi_dBm)\n",
",
"        own_rssi_dBm = np.mean(own_rssi_dBm)\n",
"    if not np.isnan(own_rssi_dBm):\n",
"        desired_mW = 10 ** ((own_rssi_dBm - 30) / 10)\n",

```



```

",
"    else:\n",
"        return np.nan\n",
"    # 噪声功率（假设为-95dBm）\n",
"    noise_floor_dBm = -95\n",
"    noise_mW = 10 ** ((noise_floor_dBm - 30) / 10)\n",
"    # 计算SINR\n",
"    sinr_linear = desired_mW / (interference_mW +
    noise_mW)\n",
"    sinr_dB = 10 * np.log10(sinr_linear)\n",
"    return sinr_dB\n",
"\n",
"# 计算SINR并添加到数据集中\n",
"training_data['sinr_dB'] = training_data.apply(
    calculate_sinr, axis=1)\n",
"# 去除缺失值\n",
"training_data.dropna(subset=['sinr_dB', 'mcs', 'nss'],
    inplace=True)\n",
"\n",
"# 特征和目标变量\n",
"features = ['sinr_dB', 'pd', 'ed', 'nav', 'eirp']\n",
"X = training_data[features]\n",
"y_mcs = training_data['mcs'].astype(int)\n",
"y_nss = training_data['nss'].astype(int)\n",
"\n",
"# 将数据划分为训练集和验证集\n",
"X_train, X_val, y_mcs_train, y_mcs_val =
    train_test_split(X, y_mcs, test_size=0.2,
        random_state=42)\n",
"X_train_nss, X_val_nss, y_nss_train, y_nss_val =
    train_test_split(X, y_nss, test_size=0.2,
        random_state=42)\n",
"\n",
"# 建立MCS的随机森林模型\n",
"clf_mcs = RandomForestClassifier(n_estimators=100,
    random_state=42)\n",
"clf_mcs.fit(X_train, y_mcs_train)\n",
"\n",
"# 建立NSS的随机森林模型\n",

```

```

"clf_nss = RandomForestClassifier(n_estimators=100,
    random_state=42)\n",
"clf_nss.fit(X_train_nss, y_nss_train)\n",
"\n",
"# 评估模型性能\n",
"from sklearn.metrics import accuracy_score\n",
"\n",
"y_mcs_pred = clf_mcs.predict(X_val)\n",
"accuracy_mcs = accuracy_score(y_mcs_val, y_mcs_pred)\n"
    ,
"print(f\"MCS模型验证准确率: {accuracy_mcs:.2f}\")\n",
"\n",
"y_nss_pred = clf_nss.predict(X_val_nss)\n",
"accuracy_nss = accuracy_score(y_nss_val, y_nss_pred)\n"
    ,
"print(f\"NSS模型验证准确率: {accuracy_nss:.2f}\")\n",
"\n",
"# 处理测试数据并进行预测\n",
"test_files = ['../test_set_2_2ap.csv', '../
    test_set_2_3ap.csv']\n",
"\n",
"for test_file in test_files:\n",
"    test_data = pd.read_csv(test_file)\n",
"    # 计算SINR\n",
"    test_data['sinr_dB'] = test_data.apply(
        calculate_sinr, axis=1)\n",
"    # 特征处理\n",
"    X_test = test_data[features].fillna(0)\n",
"    # 预测MCS和NSS\n",
"    test_data['mcs_pred'] = clf_mcs.predict(X_test)\n",
"    test_data['nss_pred'] = clf_nss.predict(X_test)\n",
"    # 保存预测结果\n",
"    output_file = test_file.replace('.csv', '
        _predictions.csv')\n",
"    test_data[['test_id', 'ap_id', 'sta_id', 'mcs_pred
        ', 'nss_pred']].to_csv(output_file, index=False)\n",
"    print(f\"{test_file}的预测结果已保存到{output_file
        }\")
]

```

```

    }
  ],
  "metadata": {
    "kernelspec": {
      "display_name": "py39torch",
      "language": "python",
      "name": "python3"
    },
    "language_info": {
      "codemirror_mode": {
        "name": "ipython",
        "version": 3
      },
      "file_extension": ".py",
      "mimetype": "text/x-python",
      "name": "python",
      "nbconvert_exporter": "python",
      "pygments_lexer": "ipython3",
      "version": "3.9.19"
    }
  },
  "nbformat": 4,
  "nbformat_minor": 2
}

```

A.3 第3问程序

q3 reprocessing data.py

```

import pandas as pd
import numpy as np

# 计算同步、异步、混合状态
def process_transmission_data(df):
    # 创建新列 'Transmission Status' 和 'NAV Status'
    df['Transmission Status'] = 0 # 1: 同步, 2: 异步, 3: 混合
    df['NAV Status'] = 1 # 1: 有效, 0: 无效

    # 处理每一行的 RSSI 值, 并根据逻辑判断状态
    for index, row in df.iterrows():
        rssi_ap_0 = row['ap_from_ap_0_max_ant_rssi']

```

```

rss_i_ap_1 = row['ap_from_ap_1_max_ant_rssi']

# 动态获取每一行的 PD, ED, NAV
pd_value = row['pd']
ed_value = row['ed']
nav_value = row['nav']

# 判断传输状态
if (rss_i_ap_0 > ed_value and rss_i_ap_1 > ed_value ):
df.at[index, 'Transmission Status'] = 1 # 同步
elif (
(pd_value < rss_i_ap_0 <= ed_value) or
(pd_value < rss_i_ap_1 <= ed_value)
):
df.at[index, 'Transmission Status'] = 3 # 混合
elif (
(nav_value < rss_i_ap_0 <= pd_value) and
(nav_value < rss_i_ap_1 <= pd_value)
):
df.at[index, 'Transmission Status'] = 2 # 异步

# 判断 NAV 状态
if (
(nav_value < rss_i_ap_0 <= pd_value) or
(nav_value < rss_i_ap_1 <= pd_value)
):
df.at[index, 'NAV Status'] = 0 # 无效
else:
df.at[index, 'NAV Status'] = 1 # 有效

# 返回处理后的数据
return df

def ap2(name):
# 原始数据
odata = pd.read_csv("../.." + name)
# print(odata.head())

```

```

# 需要的特征
features = ["loc_id", "bss_id", "ap_id", "sta_id",
"protocol",
"nav", "pd", "ed",
"eirp",

"ap_from_ap_0_sum_ant_rssi", "ap_from_ap_0_max_ant_rssi", "
    ap_from_ap_0_mean_ant_rssi", "ap_from_ap_1_sum_ant_rssi", "
    ap_from_ap_1_max_ant_rssi", "ap_from_ap_1_mean_ant_rssi",
"sta_to_ap_0_sum_ant_rssi", "sta_to_ap_0_max_ant_rssi", "
    sta_to_ap_0_mean_ant_rssi", "sta_to_ap_1_sum_ant_rssi", "
    sta_to_ap_1_max_ant_rssi", "sta_to_ap_1_mean_ant_rssi",
"sta_from_ap_0_sum_ant_rssi", "sta_from_ap_0_max_ant_rssi", "
    sta_from_ap_0_mean_ant_rssi",
"sta_from_ap_1_sum_ant_rssi", "sta_from_ap_1_max_ant_rssi", "
    sta_from_ap_1_mean_ant_rssi",
"sta_from_sta_0_rssi", "sta_from_sta_1_rssi",
"nss", "mcs", "per", "num_ampdu", "ppdu_dur", "other_air_time",
    "seq_time", "throughput"
]

# 需要特征对应的数据
need_data = odata[features]

# 把字符串映射为数字
# 创建一个字典来映射字符串到数字
string_to_number = {
    "loc0":10, "loc1":1, "loc2":2, "loc3":3, "loc30":30, "loc31":
        :31, "loc32":32, "loc33":33, "loc4":4,
    "ap_0":10, "ap_1":1, "ap_2":2,
    "sta_0":10, "sta_1":1, "sta_2":2,
    "tcp":1, "udp":2,
}

data = need_data.replace(string_to_number)

# 把NAN换成0
data = data.fillna(0)

# 把数组修改为列表，取平均

```

```

import ast
# 遍历需要修改的位置
rows, cols = data.iloc[:, 7:-4].shape[0], data.iloc[:, 7:-4].
    shape[1]
for row in range(rows):
for col in range(7, cols+7+1):
da = data.iloc[row, col]
# 先将字符转换为数组
if isinstance(da, str):
# data.iloc[row, col] = ast.literal_eval(da)
the_list = ast.literal_eval(da)
if len(the_list) <= 2: # 数据小于3个取平均
data.iloc[row, col] = sum(the_list) / len(the_list)
else: # 否则去掉一个最高值去掉一个最低值, 求平均
# print("before", len(the_list))
the_list.remove(max(the_list))
the_list.remove(min(the_list))
# print("after", len(the_list))
data.iloc[row, col] = sum(the_list) / len(the_list)

print(data)

data.to_csv(name, index=False)

"""将几个csv文件合并为1个"""
def merge_data(csv_files, name):
merged_df = pd.DataFrame() # 初始化一个空的DataFrame用于合并

for file in csv_files:
df = pd.read_csv(file)
# 如果merged_df是空的, 直接赋值
if merged_df.empty:
merged_df = df
else:
merged_df = pd.concat([merged_df, df], axis=0)

# 查看合并后的DataFrame
print(merged_df)

```

```

# 如果你想要保存合并后的DataFrame到新的CSV文件
merged_df.to_csv(name, index=False)

def ap3(name):
    # 原始数据
    odata = pd.read_csv("../.." + name)
    # print(odata.head())

    # 需要的特征
    features = ["loc_id", "bss_id", "ap_id", "sta_id",
                "protocol",
                "nav", "pd", "ed",
                "eirp",

                "ap_from_ap_0_sum_ant_rssi", "ap_from_ap_0_max_ant_rssi", "
                ap_from_ap_0_mean_ant_rssi",
                "ap_from_ap_1_sum_ant_rssi", "ap_from_ap_1_max_ant_rssi", "
                ap_from_ap_1_mean_ant_rssi",
                "ap_from_ap_2_sum_ant_rssi", "ap_from_ap_2_max_ant_rssi", "
                ap_from_ap_2_mean_ant_rssi",
                "sta_to_ap_0_sum_ant_rssi", "sta_to_ap_0_max_ant_rssi", "
                sta_to_ap_0_mean_ant_rssi",
                "sta_to_ap_1_sum_ant_rssi", "sta_to_ap_1_max_ant_rssi", "
                sta_to_ap_1_mean_ant_rssi",
                "sta_to_ap_2_sum_ant_rssi", "sta_to_ap_2_max_ant_rssi", "
                sta_to_ap_2_mean_ant_rssi",
                "sta_from_ap_0_sum_ant_rssi", "sta_from_ap_0_max_ant_rssi", "
                sta_from_ap_0_mean_ant_rssi",
                "sta_from_ap_1_sum_ant_rssi", "sta_from_ap_1_max_ant_rssi", "
                sta_from_ap_1_mean_ant_rssi",
                "sta_from_ap_2_sum_ant_rssi", "sta_from_ap_2_max_ant_rssi", "
                sta_from_ap_2_mean_ant_rssi",
                "sta_from_sta_0_rssi", "sta_from_sta_1_rssi", "
                sta_from_sta_2_rssi",

                "nss", "mcs", "per", "num_ampdu", "ppdu_dur", "other_air_time",
                "seq_time", "throughput"

```

```

]

# 需要特征对应的数据
need_data = odata[features]

# 把字符串映射为数字
# 创建一个字典来映射字符串到数字
string_to_number = {
    "loc0":10, "loc1":1, "loc2":2, "loc3":3, "loc30":30, "loc31":
        :31, "loc32":32, "loc33":33, "loc4":4,
    "ap_0":10, "ap_1":1, "ap_2":2,
    "sta_0":10, "sta_1":1, "sta_2":2,
    "tcp":1, "udp":2,
}
data = need_data.replace(string_to_number)

# 把NAN换成0
data = data.fillna(0)

# 把数组修改为列表，取平均
import ast
# 遍历需要修改的位置
rows, cols = data.iloc[:, 7:-4].shape[0], data.iloc[:, 7:-4].
    shape[1]
for row in range(rows):
    for col in range(7, cols+7+1):
        da = data.iloc[row, col]
        # 先将字符转换为数组
        if isinstance(da, str):
            # data.iloc[row, col] = ast.literal_eval(da)
            the_list = ast.literal_eval(da)
            if len(the_list) <= 2: # 数据小于3个取平均
                data.iloc[row, col] = sum(the_list) / len(the_list)
            else: # 否则去掉一个最高值去掉一个最低值，求平均
                # print("before", len(the_list))
                the_list.remove(max(the_list))
                the_list.remove(min(the_list))
                # print("after", len(the_list))

```



```

data.iloc[row, col] = sum(the_list) / len(the_list)

print(data)

data.to_csv(name, index=False)

if __name__ == '__main__':
    # 2AP
    names = ["training_set_2ap_loc0_nav82.csv", "
             training_set_2ap_loc0_nav86.csv", "
             training_set_2ap_loc1_nav82.csv", "
             training_set_2ap_loc1_nav86.csv", "
             training_set_2ap_loc2_nav82.csv"]
    for name in names:
        ap2(name)
    merge_data(names, 'merged_2ap.csv')

    # 3AP
    names = [
        "training_set_3ap_loc30_nav82.csv", "
         training_set_3ap_loc30_nav86.csv", "
         training_set_3ap_loc31_nav82.csv", "
         training_set_3ap_loc31_nav86.csv",
        "training_set_3ap_loc32_nav82.csv", "
         training_set_3ap_loc32_nav86.csv", "
         training_set_3ap_loc33_nav82.csv", "
         training_set_3ap_loc33_nav88.csv"
    ]
    for name in names:
        ap3(name)
    merge_data(names, 'merged_3ap.csv')

    # test-2ap
    names = ["test_set_1_2ap.csv"]
    for name in names:
        ap2(name)

    # test-3ap

```

```
names = ["test_set_1_3ap.csv"]
for name in names:
    ap3(name)
```

q3 prediction.py

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error,
    mean_absolute_error
import ast
import warnings
warnings.filterwarnings('ignore')

# 定义PHY Rate查找函数
def get_phy_rate(nss_list, mcs_list):
    phy_rate_table = {
        0: [8.6, 17.2],
        1: [17.2, 34.4],
        2: [25.8, 51.6],
        3: [34.4, 68.8],
        4: [51.6, 103.2],
        5: [68.8, 137.6],
        6: [77.4, 154.9],
        7: [86.0, 172.1],
        8: [103.2, 206.5],
        9: [114.7, 229.4],
        10: [129.0, 258.1],
        11: [143.4, 286.8],
    }
    phy_rates = []
    for nss, mcs in zip(nss_list, mcs_list):
        if mcs in phy_rate_table and nss in [1, 2]:
            index = nss - 1
            phy_rates.append(phy_rate_table[mcs][index])
        else:
            phy_rates.append(np.nan)
    return phy_rates
```

```

# 定义特征提取函数
def extract_features(df):
    features_df = df.copy()

    # 计算PHY Rate
    features_df['phy_rate'] = get_phy_rate(features_df['nss'],
        features_df['mcs'])

    # 计算有效数据率
    features_df['effective_rate'] = features_df['phy_rate'] * (1 -
        features_df['per'])

    # 提取自身AP的数字ID
    features_df['ap_id_num'] = features_df['ap_id'].str.extract('(\d
        +)').astype(int)

    # 提取自身AP到STA的RSSI
    features_df['sta_rssi'] = np.nan
    for idx, row in features_df.iterrows():
        sta_rssi_col = f'sta_from_ap_{row["ap_id_num"]}_sum_ant_rssi'
        # print(sta_rssi_col)
        if sta_rssi_col in features_df.columns:
            # print(np.mean(ast.literal_eval( row.get(sta_rssi_col, np.nan))
                ))
            features_df.at[idx, 'sta_rssi'] = np.mean(ast.literal_eval( row.
                get(sta_rssi_col, np.nan)))

    # 提取干扰AP的RSSI
    all_ap_id_nums = features_df['ap_id_num'].unique()
    features_df['interfering_rssi_mean'] = np.nan
    features_df['interfering_rssi_max'] = np.nan

    for idx, row in features_df.iterrows():
        interfering_rssi = []
        current_ap_id_num = row['ap_id_num']
        interfering_ap_ids = [ap_id for ap_id in all_ap_id_nums if ap_id
            != current_ap_id_num]

        for int_ap_id_num in interfering_ap_ids:

```

```

int_rssi_col = f'sta_from_ap_{int_ap_id_num}_sum_ant_rssi'
if int_rssi_col in features_df.columns:
    # print(row.get(int_rssi_col, np.nan))
    if isinstance(row.get(int_rssi_col, np.nan), str):
        rssi_value = np.mean(ast.literal_eval( row.get(int_rssi_col, np.
            nan)))
    if not pd.isna(rssi_value):
        interfering_rssi.append(rssi_value)

if interfering_rssi:
    features_df.at[idx, 'interfering_rssi_mean'] = np.mean(
        interfering_rssi)
    features_df.at[idx, 'interfering_rssi_max'] = np.max(
        interfering_rssi)
else:
    features_df.at[idx, 'interfering_rssi_mean'] = 0
    features_df.at[idx, 'interfering_rssi_max'] = 0

# 判断AP之间是否互相“听到”
features_df['can_hear_pd'] = 0
features_df['can_hear_ed'] = 0
features_df['can_hear_nav'] = 0

for idx, row in features_df.iterrows():
    can_hear_pd = 0
    can_hear_ed = 0
    can_hear_nav = 0
    current_ap_id_num = row['ap_id_num']
    interfering_ap_ids = [ap_id for ap_id in all_ap_id_nums if ap_id
        != current_ap_id_num]
    pd_value = row['pd']
    ed_value = row['ed']
    nav_value = row['nav']

    for int_ap_id_num in interfering_ap_ids:
        ap_rssi_col_max = f'ap_from_ap_{int_ap_id_num}_max_ant_rssi'
        ap_rssi_col_mean = f'ap_from_ap_{int_ap_id_num}_mean_ant_rssi'
        if isinstance(row.get(ap_rssi_col_max, np.nan), str):
            ap_rssi_max = np.mean(ast.literal_eval( row.get(ap_rssi_col_max,

```

```

        np.nan)))
if isinstance(row.get(ap_rssi_col_mean, np.nan), str):
ap_rssi_mean = np.mean(ast.literal_eval( row.get(
    ap_rssi_col_mean, np.nan)))

# ap_rssi_max = row.get(ap_rssi_col_max, np.nan)
# ap_rssi_mean = row.get(ap_rssi_col_mean, np.nan)

if not pd.isna(ap_rssi_max):
if ap_rssi_max >= pd_value:
can_hear_pd += 1
if ap_rssi_max >= ed_value:
can_hear_ed += 1
if not pd.isna(ap_rssi_mean) and ap_rssi_mean >= nav_value:
can_hear_nav += 1

features_df.at[idx, 'can_hear_pd'] = can_hear_pd
features_df.at[idx, 'can_hear_ed'] = can_hear_ed
features_df.at[idx, 'can_hear_nav'] = can_hear_nav

# 删除辅助列
features_list=['ap_id_num', 'protocol', 'nss', 'mcs', 'pd', 'ed'
    , 'nav', 'seq_time', 'interfering_rssi_mean', '
    interfering_rssi_max', 'phy_rate', 'effective_rate', 'sta_rssi'
    , 'can_hear_pd', 'can_hear_ed', 'can_hear_nav', 'throughput']
# features_df.drop('ap_id_num', axis=1, inplace=True)

return features_df[features_list]

# 读取训练数据
training_files = [
'../training_set_2ap_loc0_nav82.csv',
'../training_set_2ap_loc0_nav86.csv',
'../training_set_2ap_loc1_nav82.csv',
'../training_set_2ap_loc1_nav86.csv',
'../training_set_2ap_loc2_nav82.csv',
'../training_set_3ap_loc30_nav82.csv',
'../training_set_3ap_loc30_nav86.csv',
'../training_set_3ap_loc31_nav82.csv',

```

```

'../training_set_3ap_loc31_nav86.csv',
'../training_set_3ap_loc32_nav82.csv',
'../training_set_3ap_loc32_nav86.csv',
'../training_set_3ap_loc33_nav82.csv',
'../training_set_3ap_loc33_nav88.csv'
]

# 读取并合并所有训练数据
df_list = []
for file in training_files:
    df = pd.read_csv(file)
    df_list.append(df)

training_data = pd.concat(df_list, ignore_index=True)

train_data = training_data
train_features = extract_features(train_data)
# train_features.drop('ap_id_num', axis=1, inplace=True)
print(train_data.shape, train_features.shape)


train_features.fillna(0, inplace=True)

# 准备特征矩阵和目标变量
# X = train_features.drop(['test_id', 'ap_id', 'throughput', '
    num_ampdu', 'ppdu_dur'], axis=1)
X = train_features.drop(['throughput'], axis=1)
y = train_features['throughput']
X.shape, y.shape

# 对分类特征进行编码
X = pd.get_dummies(X, columns=['protocol'])

# 划分训练集和验证集
X_train, X_val, y_train, y_val = train_test_split(X, y,
    test_size=0.2, random_state=42)

# 训练模型

```

```

regressor = GradientBoostingRegressor(n_estimators=100,
    learning_rate=0.1, random_state=42)
regressor.fit(X_train, y_train)

# 模型评估
y_pred = regressor.predict(X_val)
mse = mean_squared_error(list(y_val), y_pred)
# mae = mean_absolute_error(y_val, y_pred)
print('验证集均方误差 MSE:', mse)
# print('验证集平均绝对误差 MAE:', mae)

# 计算平均绝对百分比误差 MAPE
mape = np.mean(np.abs((list(y_val) - y_pred) / y_val)) * 100
print('验证集平均绝对百分比误差 MAPE:', mape, '%')

# 特征重要性分析
# feature_importances = pd.Series(regressor.feature_importances_,
#     index=X.columns)
# print('特征重要性:\n', feature_importances.sort_values(
#     ascending=False))

mse = mean_squared_error(list(y_val), y_pred)
print(f"验证集上的均方误差 (MSE) : {mse}")

# 计算误差并绘制CDF曲线
errors = y_pred - list(y_val)
# print('误差:', errors)

import matplotlib.pyplot as plt
# 计算误差并绘制CDF曲线
errors = y_pred - y_val
errors_percentage = (errors / y_val) * 100 # 直接与y_val进行运算，不需要转换为list

# 计算误差的CDF
sorted_errors = np.sort(errors_percentage)
cdf = np.arange(len(sorted_errors)) / float(len(sorted_errors))

```

```

# 绘制平滑的CDF曲线
plt.figure()
plt.plot(sorted_errors, cdf) # 使用连线和平滑点
plt.xlabel('预测误差 (%)')
plt.ylabel('累积分布函数 (CDF)')
plt.title('验证集上的预测误差CDF曲线')
plt.grid(True)
plt.show()

# 计算误差的90%分位数
error_90 = np.percentile(np.abs(errors_percentage), 90)
model_accuracy = 1 - error_90 / 100
print(f"模型在验证集上的精度为: {model_accuracy:.2f}")

# 读取测试数据并进行预测
test_files = ['./test_set_1_2ap.csv', './test_set_1_3ap.csv']

for test_file, seq_time in zip(test_files, ['./2ap_predictions.
    csv', './3ap_predictions.csv']):
    test_data = pd.read_csv(test_file)
    # print(seq_time)
    seq_time = pd.read_csv(seq_time)
    # 确保测试数据包含必要的特征
    # 如果测试数据中没有数据帧统计信息, 需要调整代码
    # test_features = ['seq_time', 'mcs', 'nss', 'per', 'eirp', 'pd
        ', 'ed', 'nav']# 'num_ampdu', 'ppdu_dur',
    test_data['seq_time']=seq_time['seq_time']
    # print(test_data.columns)
    test_features=extract_features(test_data)
    # 检查测试数据中是否包含所有需要的特征
    test_features = test_features.drop(['throughput'], axis=1)
    # y = train_features['throughput']

# 对分类特征进行编码
test_features = pd.get_dummies(test_features, columns=['protocol
    '])

# 预测吞吐量

```



```
test_features.fillna(0, inplace=True)
print(f"正在预测 {test_file} 的吞吐量...")
# print(X_test.head())
test_data['predict_throughput'] = regressor.predict(
    test_features)

# 保存预测结果
output_file = test_file.replace('.csv', '_throughput_predictions
    .csv')
test_data.to_csv(output_file, index=False)
print(f"{test_file} 的预测结果已保存到 {output_file}")
```